



清华大学计算机科学与技术系本科专业主修课

计算机系统结构

第5章 指令级并行及其开发—硬件方法

2026年春季学期

提升处理器性能=减少CPU时间

$$CPU\text{时间} = \left(\sum_{i=1}^n CPI_i \times IC_i \right) \times \text{时钟周期}$$

- 流水线利用时间并行性使指令重叠并行执行，提高了指令带宽（吞吐率），而不是单条指令的执行速度。
- IPC：每时钟周期执行的指令数。

CPI：每条指令执行花费的平均时钟周期数

$$CPI = \left(\sum_{i=1}^N CPI_i \right) / N \quad \text{越小越好}$$

IPC：每时钟周期执行的指令数

$$IPC = N / \left(\sum_{i=1}^N CPI_i \right) \quad \text{越大越好}$$

- 指令级并行：指令之间存在的一种并行性，利用它计算机可以并行执行两条或两条以上的指令。

(ILP: Instruction-Level Parallelism)

- 开发ILP的途径有两种

- 空间并行：重复设置多个处理部件，让它们同时执行相邻或相近的多条指令；
- 时间并行：采用流水线技术，使指令重叠并行执行。

- 开发ILP的方法可以分为两大类

- 基于硬件的动态开发方法
- 基于软件的静态开发方法

- 本章研究：如何利用各种技术来开发更多的指令级并行（硬件方法）

- 5. 1 [指令级并行的概念](#)
- 5. 2 [相关与指令级并行](#)
- 5. 3 [指令的动态调度](#)
- 5. 4 [动态分支预测技术](#)
- 5. 5 [多指令流出技术](#)

5.1 指令级并行的概念

5.1 指令级并行的概念

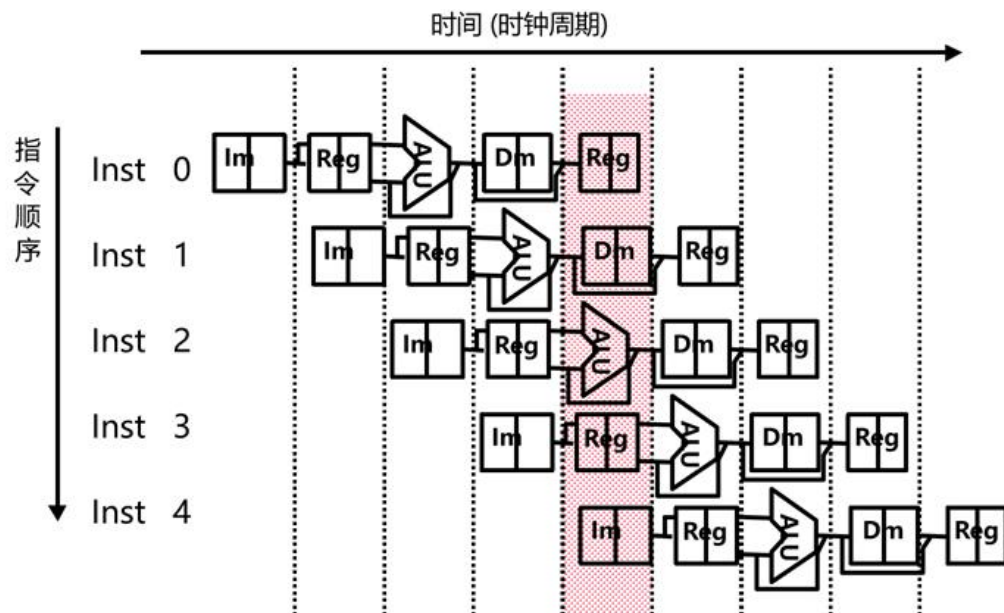
第 6 页

1. 流水线处理机的实际CPI分析

- 理想流水线的CPI加上各类停顿的时钟周期数：

$$CPI_{\text{流水线}} = CPI_{\text{理想}} + \text{停顿}_{\text{结构冲突}} + \text{停顿}_{\text{数据冲突}} + \text{停顿}_{\text{控制冲突}}$$

- $CPI_{\text{理想}}$ 是衡量流水线最高性能的一个指标。



■ 流水线结构方面：

- 结构相关：需要更多的硬件资源
- 数据相关：需要定向，编译器调度
- 控制相关：尽早检测条件，计算目标地址，延迟转移，预测

LD **R1**, 0 (R2)
DADD R4, **R1**, R5
AND R6, **R1**, R7

5.1 指令级并行的概念

第 7 页

1. 流水线处理机的实际CPI分析

- 理想流水线的CPI加上各类停顿的时钟周期数：

$$CPI_{\text{流水线}} = CPI_{\text{理想}} + \text{停顿}_{\text{结构冲突}} + \text{停顿}_{\text{数据冲突}} + \text{停顿}_{\text{控制冲突}}$$

- $CPI_{\text{理想}}$ 是衡量流水线最高性能的一个指标。

$CPI_{\text{流水线}} \approx CPI_{\text{理想}}$  思路：减少结构、数据与控制冲突

进一步减少 $CPI_{\text{流水线}}$  思路：并行执行两条或两条以上的指令 -> **指令级并行**

- 指令级并行指机器语言一级，如**存储器访问指令、整型指令、浮点指令之间的并行性**等。
- 指令级并行由处理器硬件和编译程序自动识别和利用，**不需要程序员对顺序程序作任何修改。**

5.1 指令级并行的概念

第 8 页

2. 指令级并行设计对象

- **基本程序块**：一串连续的代码（同时执行相邻或相近的多条指令），除了入口和出口以外没有其他的分支指令和转入点
- **分支指令和转入点**：程序平均每4 ~ 7条指令就会有一个分支

3. 循环级并行是指令级并行研究的重点之一

- 开发循环的不同叠代之间存在的并行性
- 循环展开（loop unrolling）技术
- 采用向量指令和向量数据表示

例如，考虑下述语句：

```
for (i=1; i<=500; i=i+1)
```

```
  a[i]=a[i] + s;
```

5.2 相关与指令级并行

5.2 相关与指令级并行

第 10 页

1. 指令相关与流水线冲突

一个简单基本程序块

DIV.D	F2, F8, F4
ADD.D	F8, F0, F12
SUB.D	F2, F8, F14

相关是程序固有的一种属性，反映了程序中的指令之间的相互依赖关系

■ 指令相关有三种类型：

- 数据相关
- 名相关（输出相关、反相关）
- 控制相关

■ 流水线冲突有三种类型：

- 结构冲突
- 数据冲突
- 控制冲突

具体的一次相关是否会导致实际冲突的发生以及该冲突会带来多长的停顿则是流水线的属性

5.2 相关与指令级并行

第 11 页

```
DIV.D F2, F8, F4
ADD.D F8, F0, F12
SUB.D F2, F8, F14
```

指令相关
流水线冲突

$$\left. \begin{array}{l} \text{指令相关} \\ \text{流水线冲突} \end{array} \right\} \text{CPI}_{\text{流水线}} \geq \text{CPI}_{\text{理想}} \left\{ \begin{array}{l} \text{消除相关: 代码变换} \\ \text{避免冲突: 指令调度} \end{array} \right\} \text{是否影响程序正确?}$$

代码变换与指令调度面临的两个问题:

Loop: 一个简单基本程序块

```
L.D    F0, 0(R1)    // F0为数组元素
ADD.D  F4, F0, F2    // 加上F2中的值
S.D    F4, 0(R1)    // 保存结果
DADDIU R1, R1, -8    // 数组指针递减8个字节
BNE    R1, R2, Loop  // 如果R1≠R2, 则分支
ADD.D  F8, F0, F6    // 加上F2中的值
```

程序顺序问题

控制相关问题

5.2 相关与指令级并行

第 12 页

DIV.D F2, F8, F4
ADD.D F8, F0, F12
SUB.D F2, F8, F14

指令相关
流水线冲突

$$\left. \begin{array}{l} \text{指令相关} \\ \text{流水线冲突} \end{array} \right\} \text{CPI}_{\text{流水线}} \geq \text{CPI}_{\text{理想}} \left\{ \begin{array}{l} \text{消除相关: 代码变换} \\ \text{避免冲突: 指令调度} \end{array} \right\} \text{是否影响程序正确?}$$

代码变换与指令调度面临的两个问题:

1. 程序顺序问题

- 定义: 程序顺序指由原来程序确定的在完全串行方式下指令的执行顺序。
- 问题: 解决相关与冲突过程中是否需要保持程序顺序? (**程序顺序可不可以改变?**)
- 原则: **程序顺序可以改变**, 只有在可能会导致错误的情况下才保持程序顺序。

2. 控制相关问题

- 定义: 一条指令要确定下一条指令的位置, 例如在执行分支、跳转、调用或返回指令时引起的相关。
- 问题: 解决相关与冲突过程中是否遵守控制相关的依赖关系? (**可不可以执行不该执行的指令?**)
- 原则: **执行不该执行的指令**不影响程序正确性, 则控制相关并不是一个必须严格保持的关键属性。

5.2 相关与指令级并行

第 13 页

```
DIV.D F2, F8, F4  
ADD.D F8, F0, F12  
SUB.D F2, F8, F14
```

指令相关
流水线冲突

$$\left. \begin{array}{l} \text{指令相关} \\ \text{流水线冲突} \end{array} \right\} \text{CPI}_{\text{流水线}} \geq \text{CPI}_{\text{理想}} \left\{ \begin{array}{l} \text{消除相关: 代码变换} \\ \text{避免冲突: 指令调度} \end{array} \right\} \text{是否影响程序正确?}$$

代码变换与指令调度必须保持的两个关键属性:

1. 数据流不变

- **定义:** 数据值从其产生者指令到其消费者指令的实际流动。
- **原则:** 分支指令使得数据流具有动态性, 不能影响数据流。一条指令有可能数据相关于多条先前指令, 分支指令的执行结果决定了哪条指令真正是所需数据的产生者。

2. 异常行为不变

- **定义:** 保持异常行为指原来程序中是怎么发生的, 改变执行顺序后还是怎么发生。
- **原则:** 无论怎么改变指令的执行顺序都不能改变程序中异常的发生情况, 也就是说改变指令执行顺序不能导致程序中发生新的异常属性。

5.2 相关与指令级并行

第 14 页

■指令调度原则：不遵守控制相关既不影响异常行为，也不改变数据流。

➤方法：把失败分支中的指令调度到分支指令之前。

举例：

DADDU		R1, R2, R3
BEQZ		R12, Skipnext
DSUBU	修改? →	R4, R5, R6
DADDU		R5, R4, R9
Skipnext: OR		R7, R8, R9

前提条件：

- 1、R4寄存器值在Skipnex后的程序不再使用；
- 2、DSUBU指令不会产生异常。

5.3 指令的动态调度

5.3 指令的动态调度

第 16 页

■ 静态调度

- 依靠编译器对代码进行静态调度，以减少相关和冲突。
- 它不是在程序执行的过程中、而是在编译期间进行代码调度和优化。
- 通过把相关的指令拉开距离来减少可能产生的停顿。

■ 动态调度

- 在程序的执行过程中，依靠专门硬件对代码进行调度，以减少数据相关导致的停顿。
- 优点：
 - ✓ 能够处理一些在编译时情况不明的相关（例如涉及到存储器访问的相关），并简化了编译器；
 - ✓ 能够使本来是面向某一流水线优化编译的代码在其它的流水线（动态调度）上也能高效地执行。
- 缺点：以硬件复杂性的显著增加为代价

5.3.1 动态调度的基本思想

第 17 页

1. 到目前为止我们所使用流水线的最大局限性:

- 指令是按序流出和按序执行的
- 考虑下面一段代码:

DIV.D F4, F0, F2

ADD.D F10, F4, F6 ;ADD.D指令与DIV.D指令关于F4相关, 导致流水线停顿。

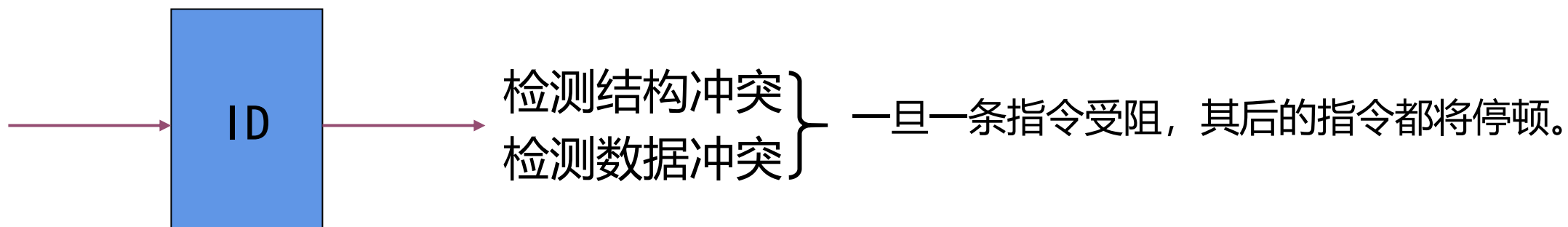
SUB.D F12, F6, F14 ;SUB.D指令与流水线中的任何指令都没有关系, 但也因此受阻

调度关键: SUB.D有没有可能提前执行?
如果提前执行会不会产生错误结果。

5.3.1 动态调度的基本思想

第 18 页

在顺序执行流水线中：



- 为了使上述指令序列中的SUB.D指令能继续执行下去，必须把指令流出的工作拆分为两步：
 - ✓ 检测结构冲突
 - ✓ 等待数据冲突消失
- 只要检测到没有结构冲突，就可以让指令流出，并且**流出后的指令一旦其操作数就绪就可以立即执行。**

5.3.1 动态调度的基本思想

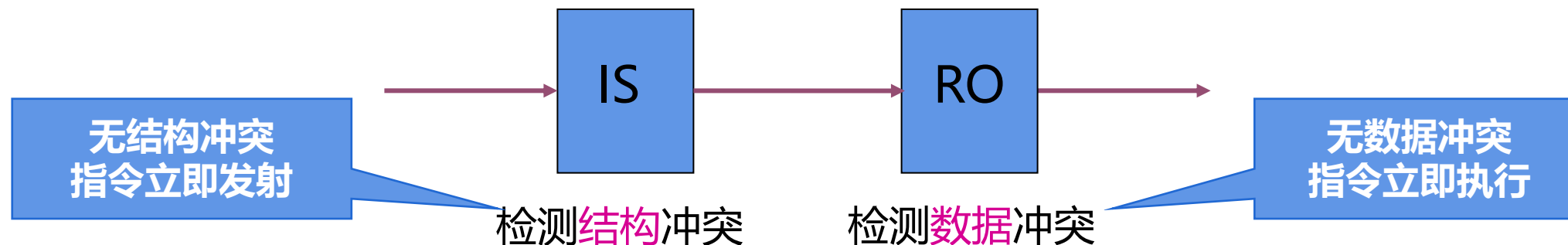
第 19 页

2. 乱序执行

- 指令的执行顺序与程序顺序不相同
- 指令的完成也是乱序完成的，即指令的完成顺序与程序顺序不相同。

3. 为了支持乱序执行，我们将5段流水线的译码阶段再分为两个阶段：

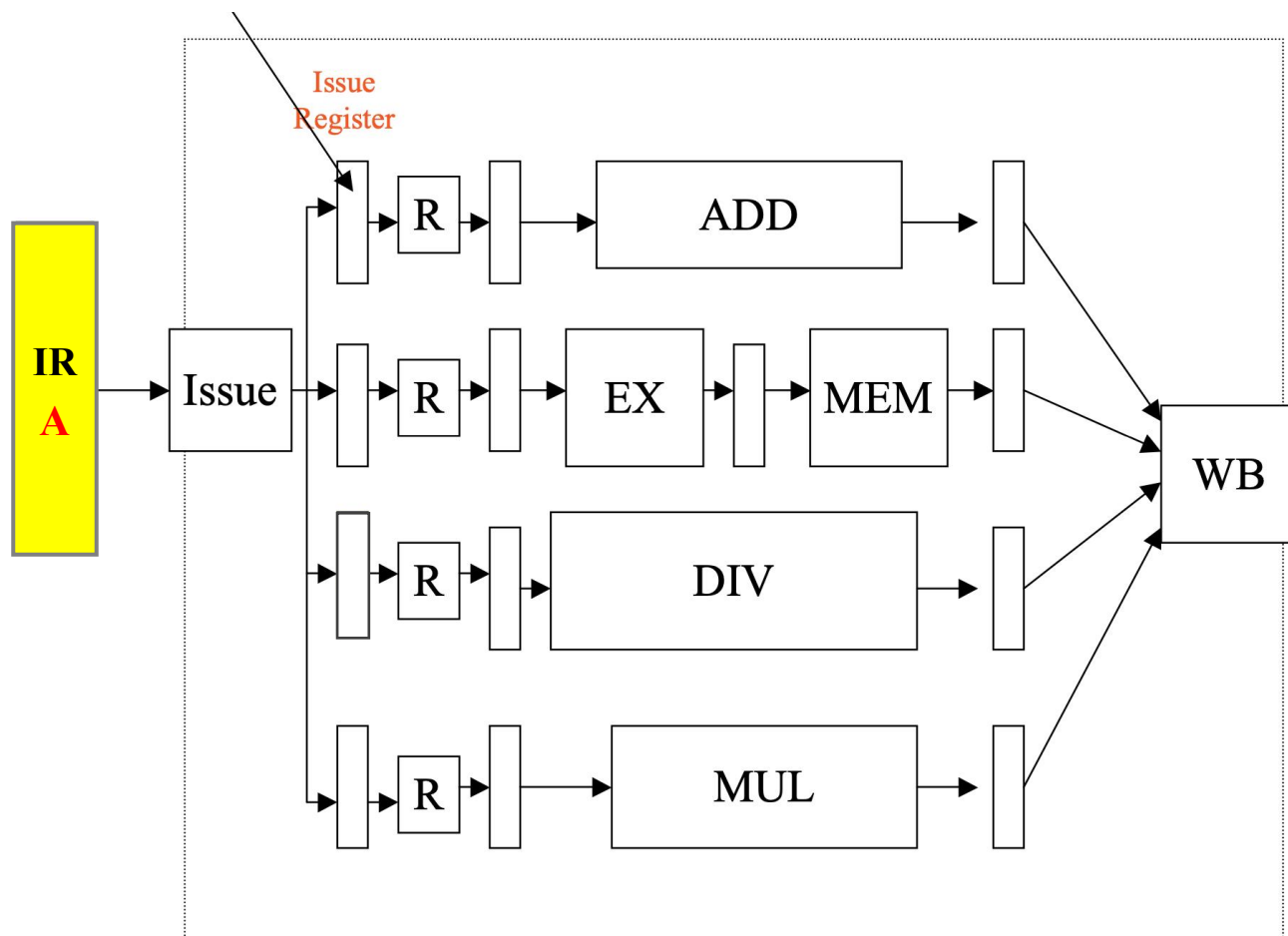
- 流出 (Issue, IS)：指令译码，检查是否存在结构冲突 (in-order issue)
- 读操作数 (Read Operands, RO)：等待数据冲突消失，然后读操作数 (out of order execution)



5.3.1 动态调度的基本思想

第 20 页

T1时间 分配区域/staging area



A: DIVD

F0, F2, F4

B: A D D D

F10, **F0**, F8

C: MULTD

F12, F8, F14

直接数据相关

IF (取指) : 1拍

IS (发射) : 1拍

R (取数) : 1拍

EX: MUL : 3拍;

ADD : 1拍

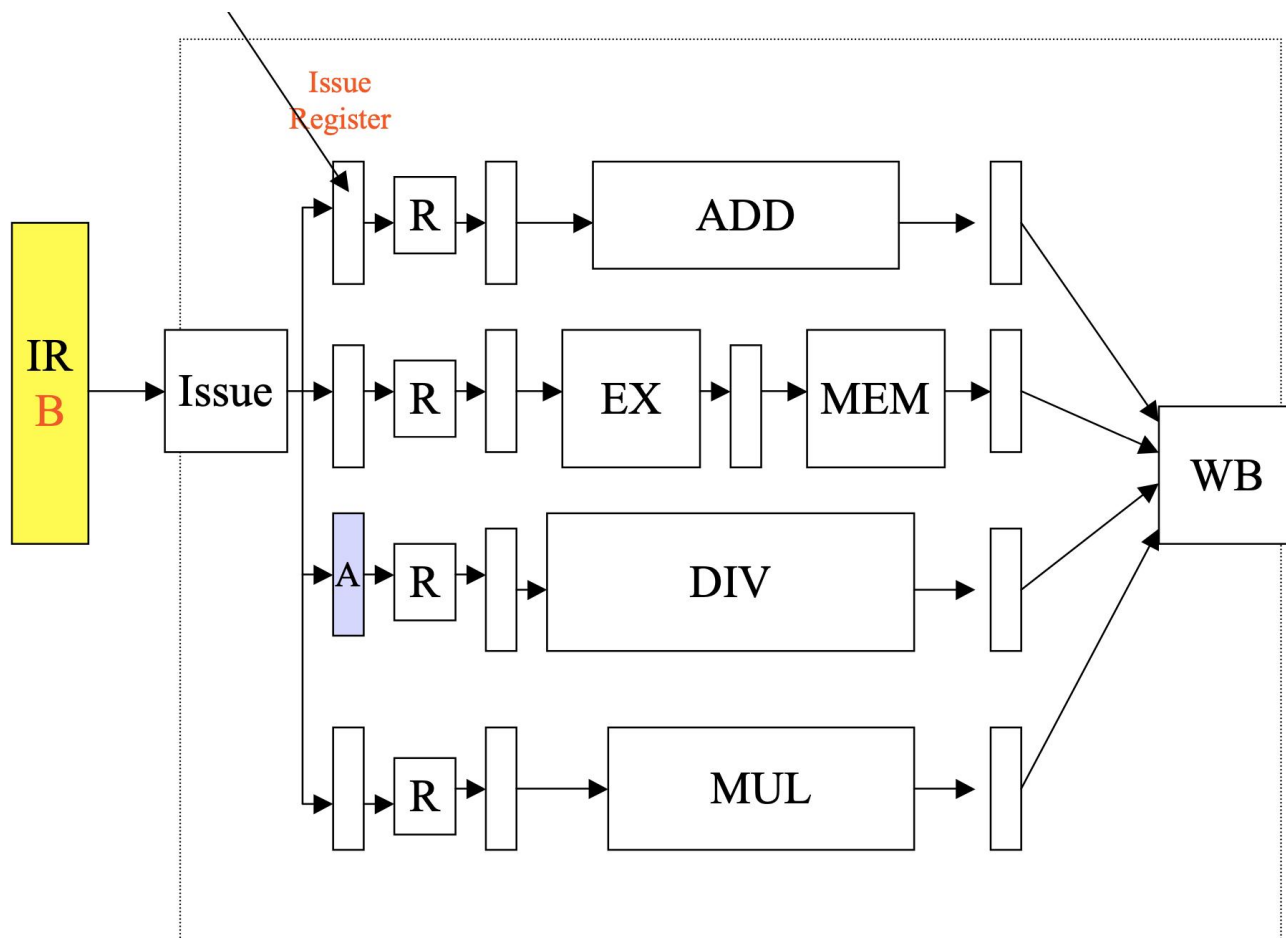
DIV : 3拍

WB (写回) : 1拍

5.3.1 动态调度的基本思想

第 21 页

T2时间 分配区域/staging area



A: DIVD **F0**, F2, F4
B: ADDD F10, **F0**, F8
C: MULTD F12, F8, F14

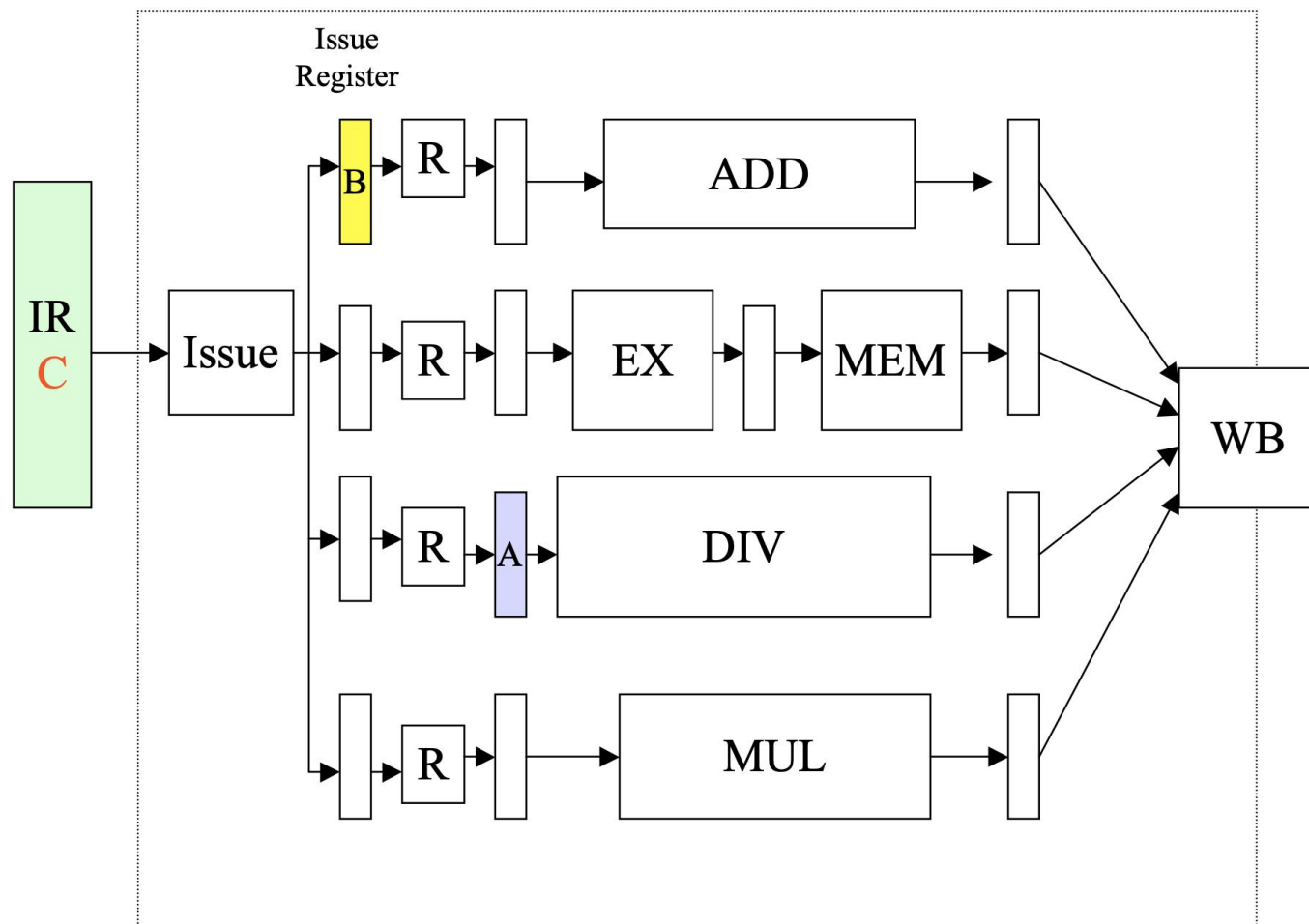
直接数据相关

IF (取指) : 1拍
IS (发射) : 1拍
R (取数) : 1拍
EX: MUL : 3拍;
 ADD : 1拍
 DIV : 3拍
WB (写回) : 1拍

5.3.1 动态调度的基本思想

第 22 页

T3时间



A: DIVD

B: ADDD

C: MULTD

F0, F2, F4

F10, **F0**, F8

F12, F8, F14

直接数据相关

IF (取指) : 1拍

IS (发射) : 1拍

R (取数) : 1拍

EX: MUL : 3拍;

ADD : 1拍

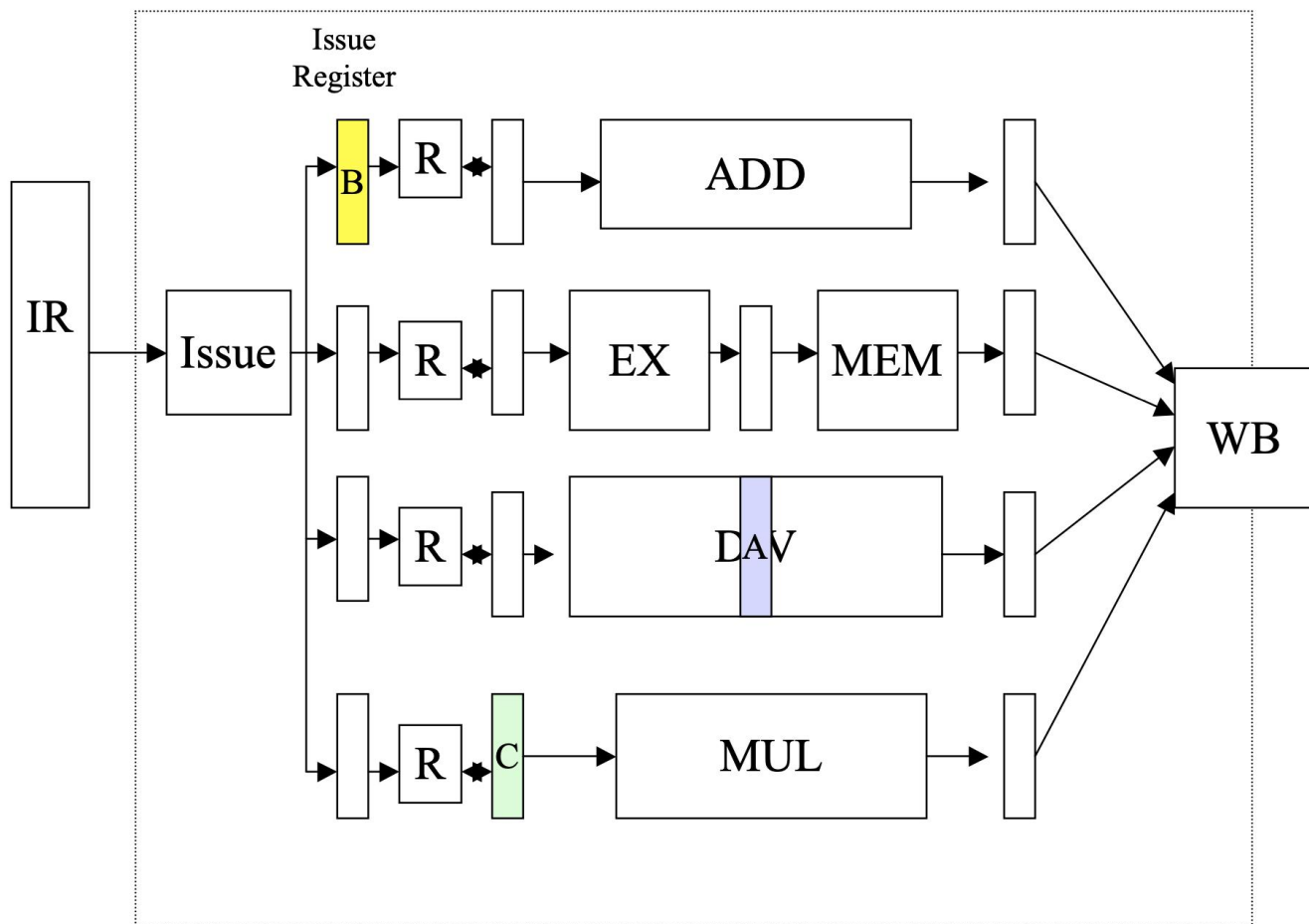
DIV : 3拍

WB (写回) : 1拍

5.3.1 动态调度的基本思想

第 24 页

T5时间



A: DIVD **F0**, F2, F4
B: ADDD F10, **F0**, F8
C: MULTD F12, F8, F14

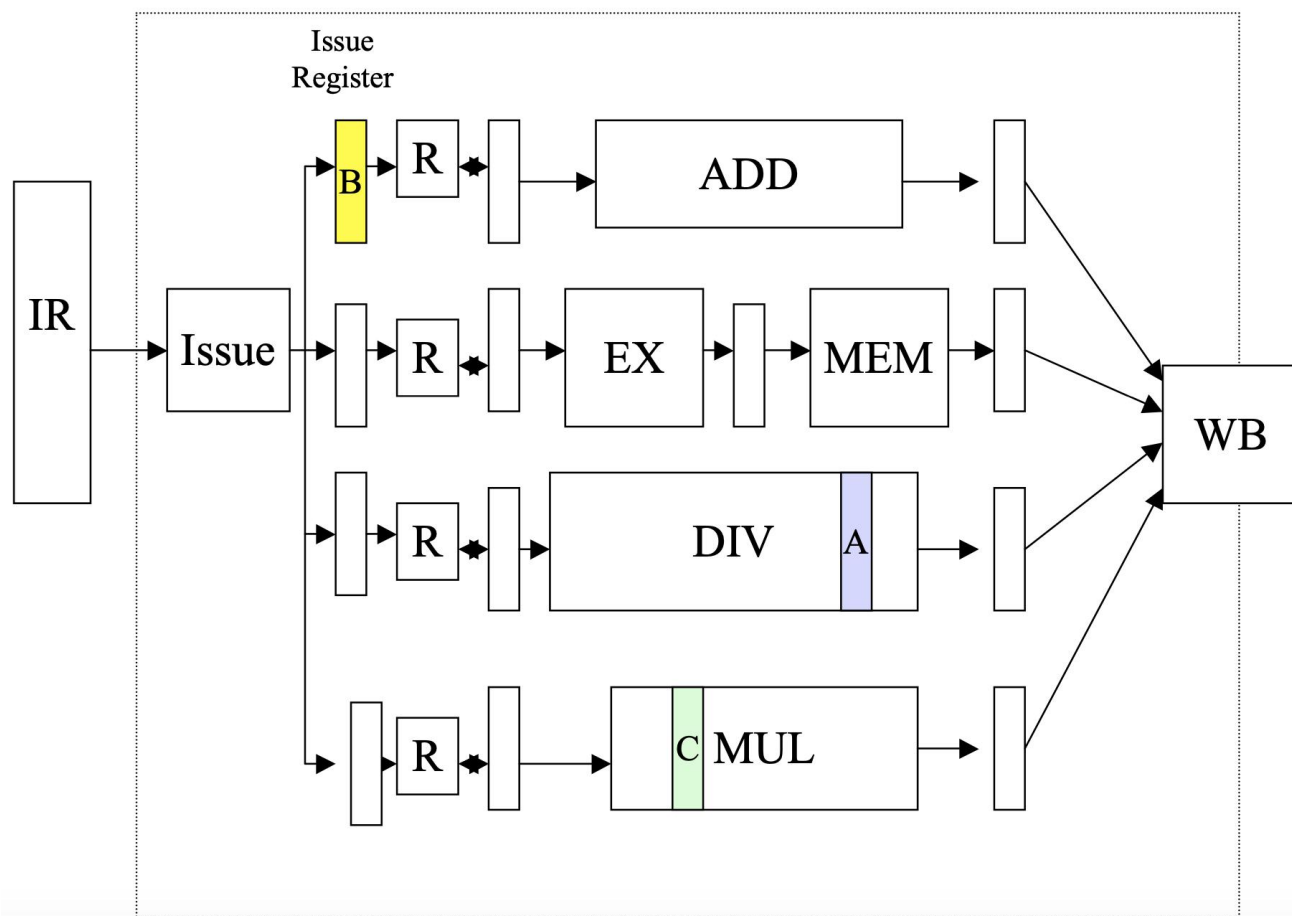
直接数据相关

IF (取指) : 1拍
IS (发射) : 1拍
R (取数) : 1拍
EX: MUL : 3拍;
 ADD : 1拍
 DIV : 3拍
WB (写回) : 1拍

5.3.1 动态调度的基本思想

第 25 页

T6时间



A: DIVD

F0, F2, F4

B: ADDD

F10, **F0**, F8

C: MULTD

F12, F8, F14

直接数据相关

IF (取指) : 1拍

IS (发射) : 1拍

R (取数) : 1拍

EX: MUL : 3拍;

ADD : 1拍

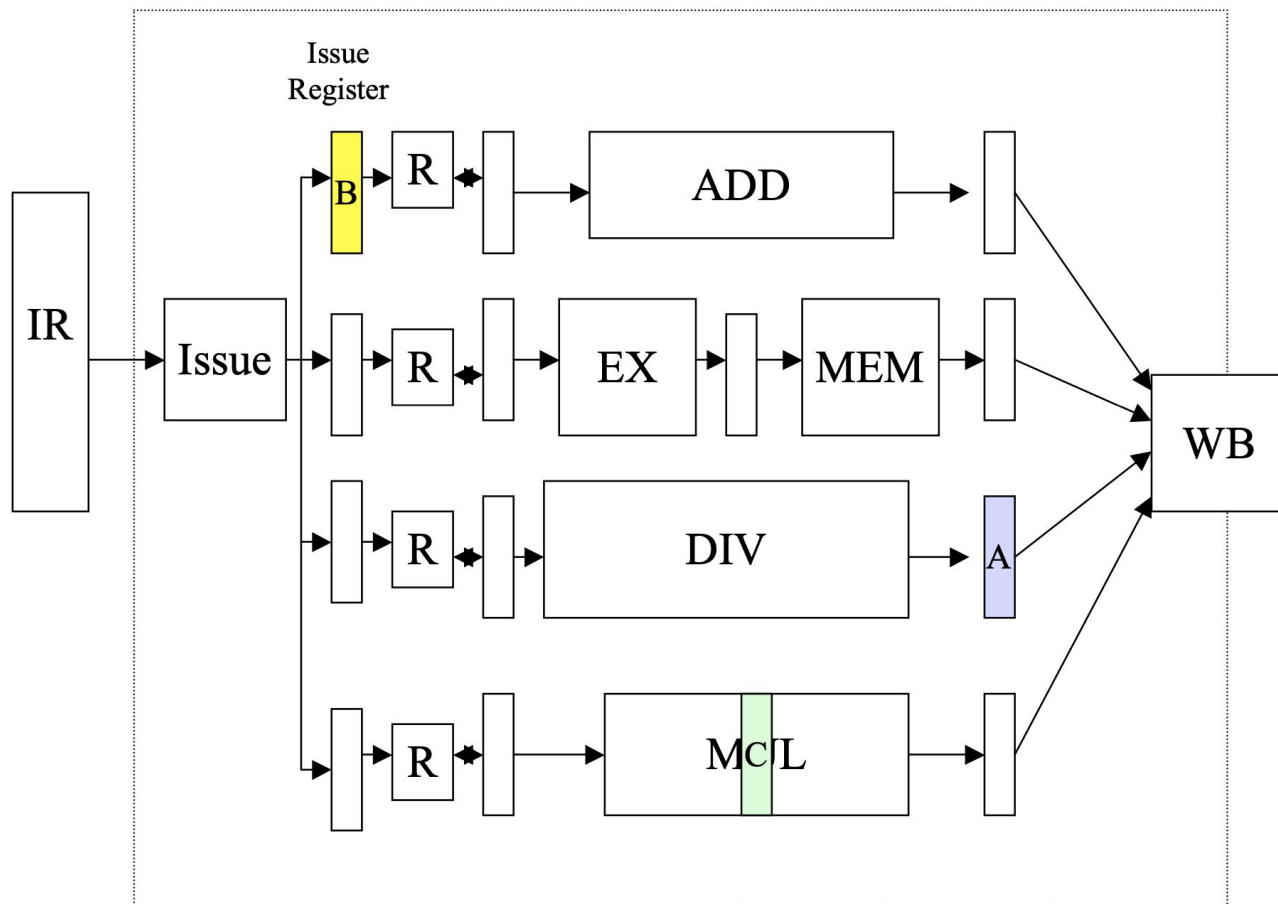
DIV : 3拍

WB (写回) : 1拍

5.3.1 动态调度的基本思想

第 26 页

T7时间



A: DIVD **F0**, F2, F4

B: ADDD F10, **F0**, F8

C: MULTD F12, F8, F14

直接数据相关

IF (取指) : 1拍

IS (发射) : 1拍

R (取数) : 1拍

EX: MUL : 3拍;

ADD : 1拍

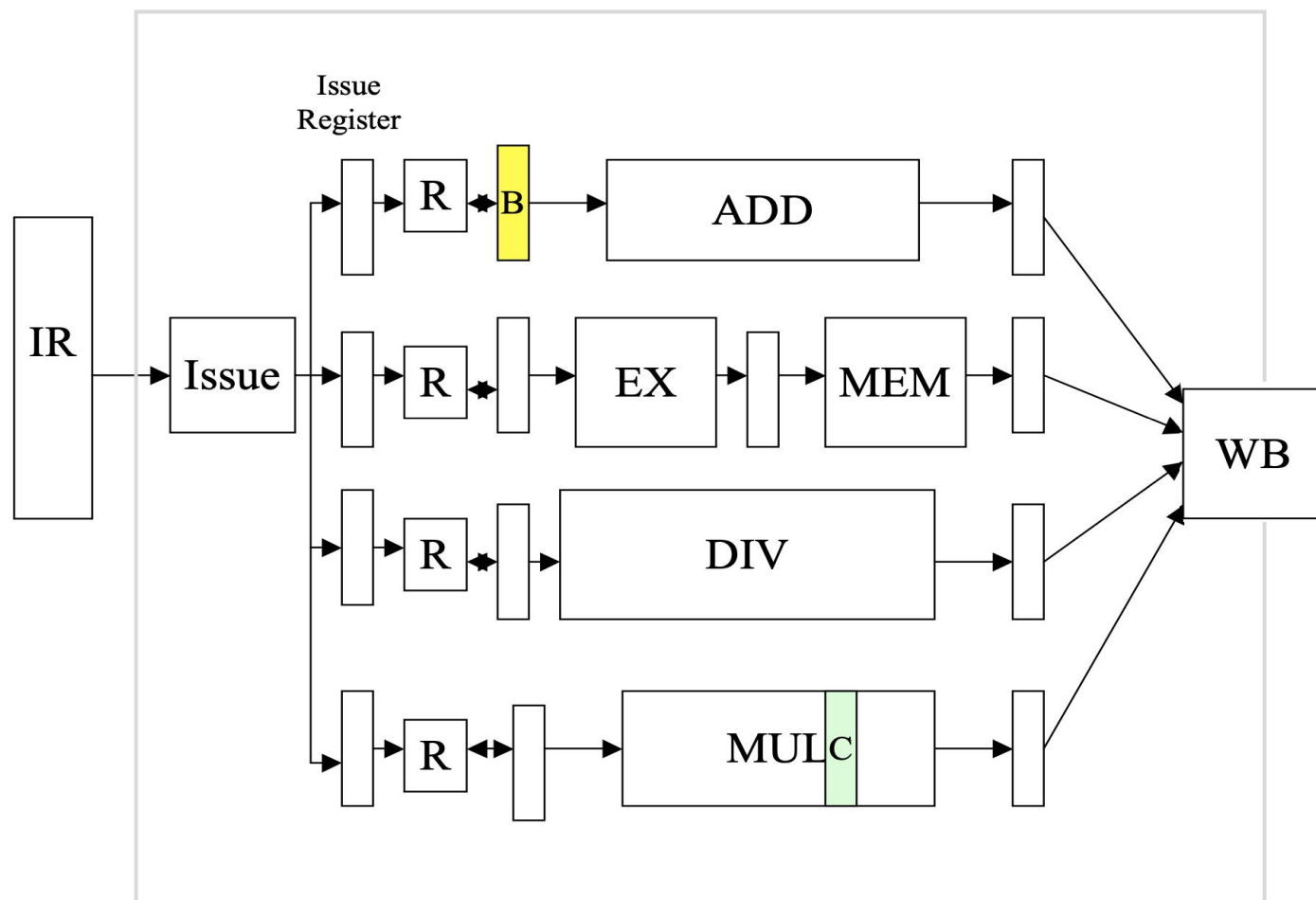
DIV : 3拍

WB (写回) : 1拍

5.3.1 动态调度的基本思想

第 27 页

T8时间



A: DIVD

B: ADDD

C: MULTD

直接数据相关

IF (取指) : 1拍

IS (发射) : 1拍

R (取数) : 1拍

EX: MUL : 3拍;

ADD : 1拍

DIV : 3拍

WB (写回) : 1拍

F0, F2, F4

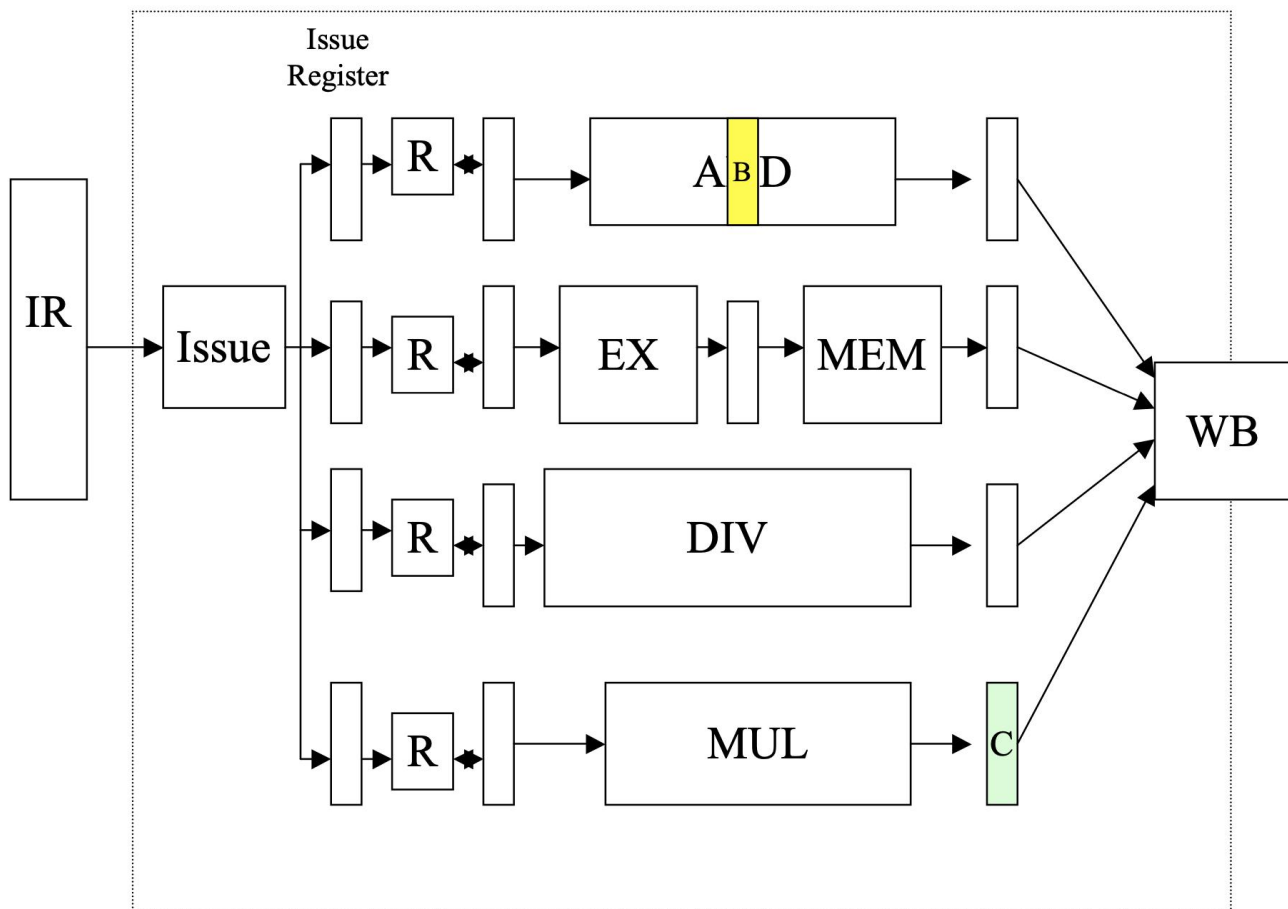
F10, **F0**, F8

F12, F8, F14

5.3.1 动态调度的基本思想

第 28 页

T9时间



A: DIVD

F0, F2, F4

B: ADDD

F10, **F0**, F8

C: MULTD

F12, F8, F14

直接数据相关

IF (取指) : 1拍

IS (发射) : 1拍

R (取数) : 1拍

EX: MUL : 3拍;

ADD : 1拍

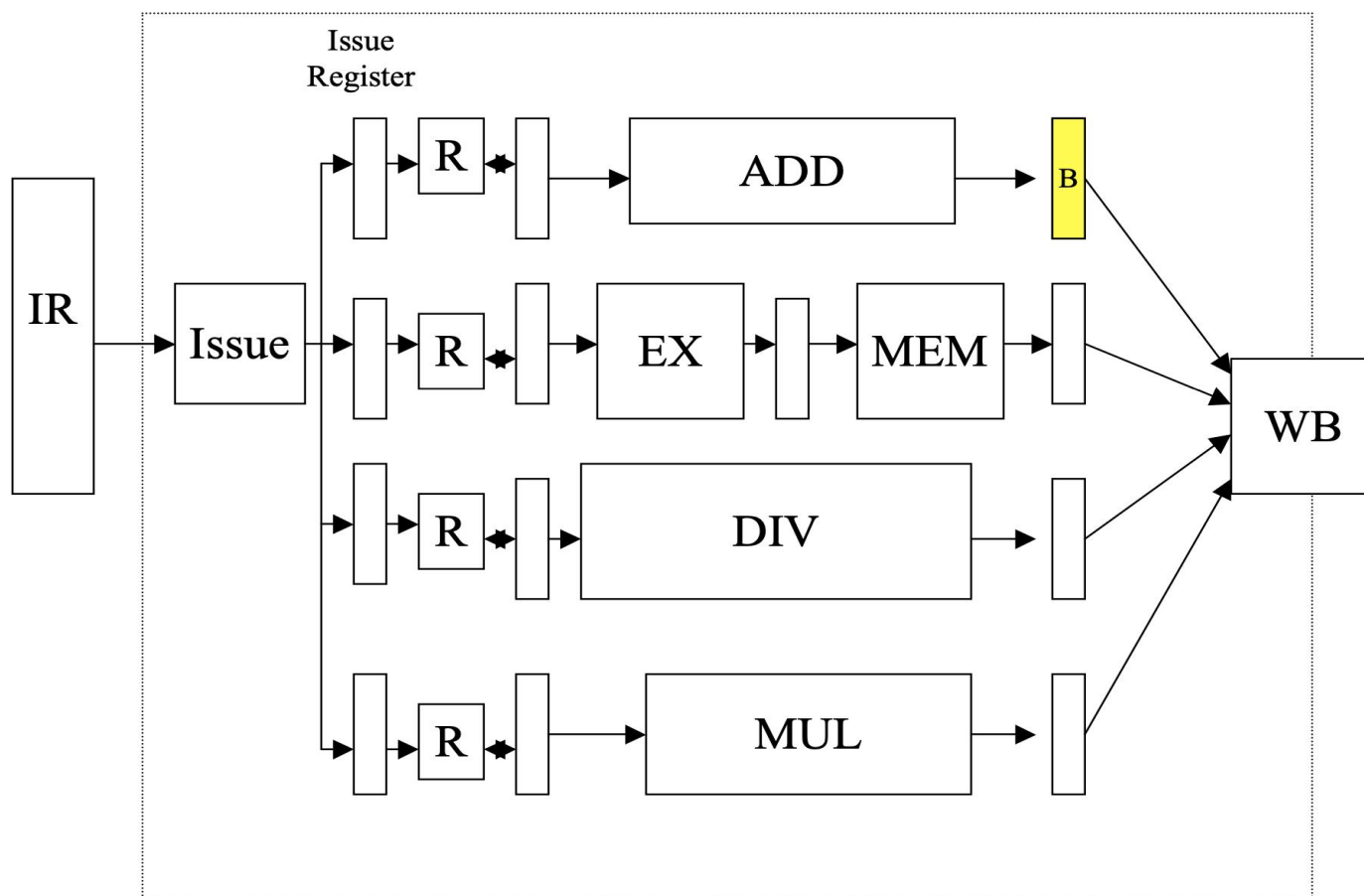
DIV : 3拍

WB (写回) : 1拍

5.3.1 动态调度的基本思想

第 29 页

T10时间



A: DIVD	F0 , F2, F4
B: ADDD	F10, F0 , F8
C: MULTD	F12, F8, F14

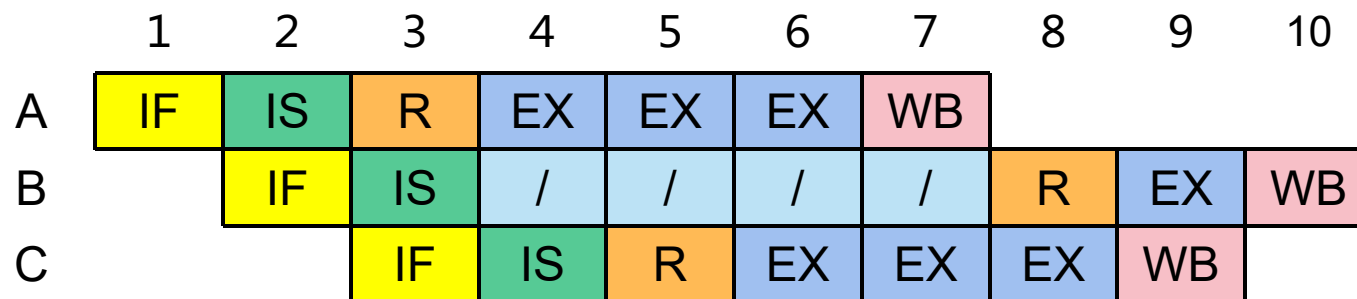
结论：适当且正确地改变程序顺序不会影响程序的正确性。

5.3.1 动态调度的基本思想

第 30 页

结论：适当且正确地改变程序顺序不会影响程序的正确性。

A: DIVD **F0**, F2, F4
B: ADDD F10, **F0**, F8
C: MULTD F12, F8, F14



乱序执行：一种将各条指令不按顺序拆散后执行的运行方式，即指令执行顺序是由输入数据可用性所决定的顺序，而不是由程序的原始数据所决定。

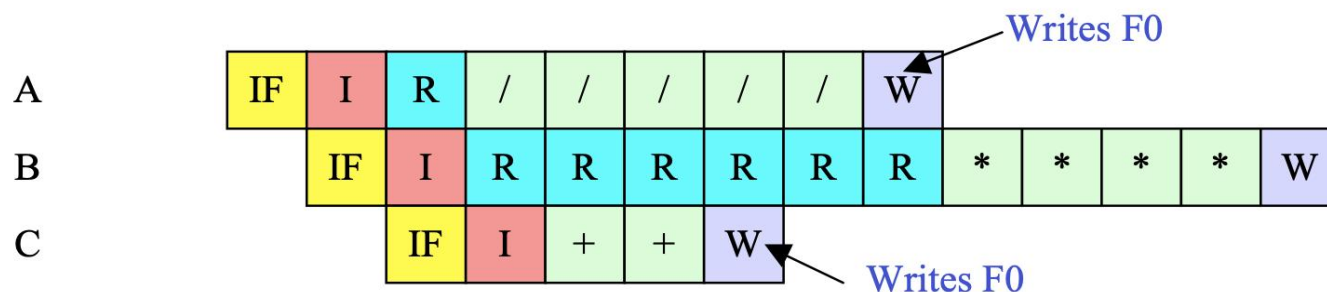
优点：可以避免因为获取下一条程序指令所引起的处理器等待，取而代之的处理下一条可以立即执行的指令。

存在问题：乱序执行会不会影响或引入新的**WAW冲突**和**WAR冲突**的？

5.3.1 动态调度的基本思想

第 31 页

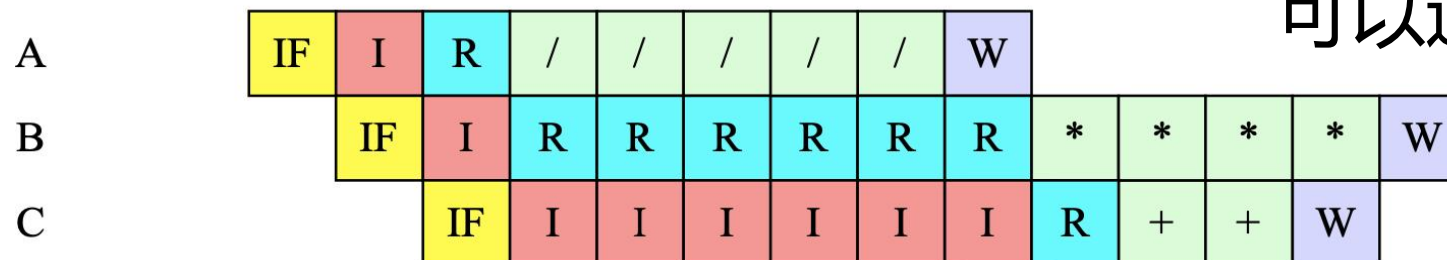
乱序执行流水线:



A: DIV.D F0, F2, F4
B: MUL.D F6, F0, F8
C: ADD.D F0, F10, F12

WAW如何处理?

若有WAW, 则不发射

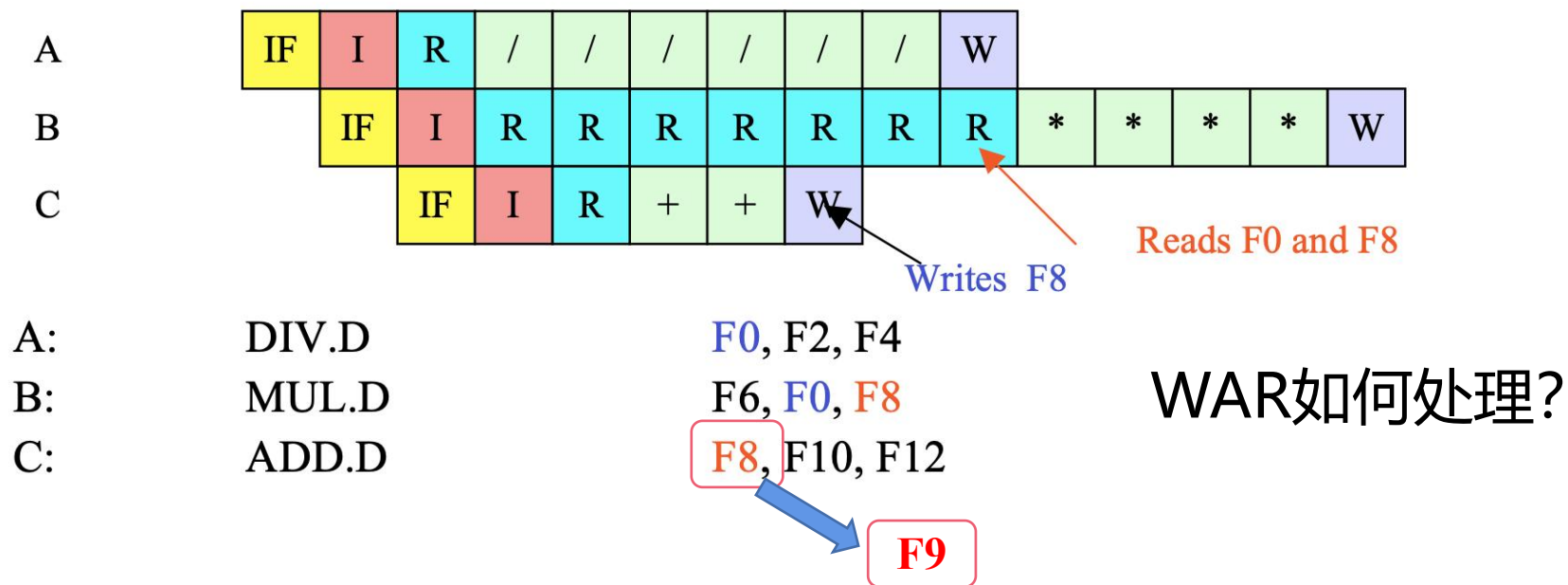


可以通过**指令调度**解决

5.3.1 动态调度的基本思想

第 32 页

乱序执行流水线:

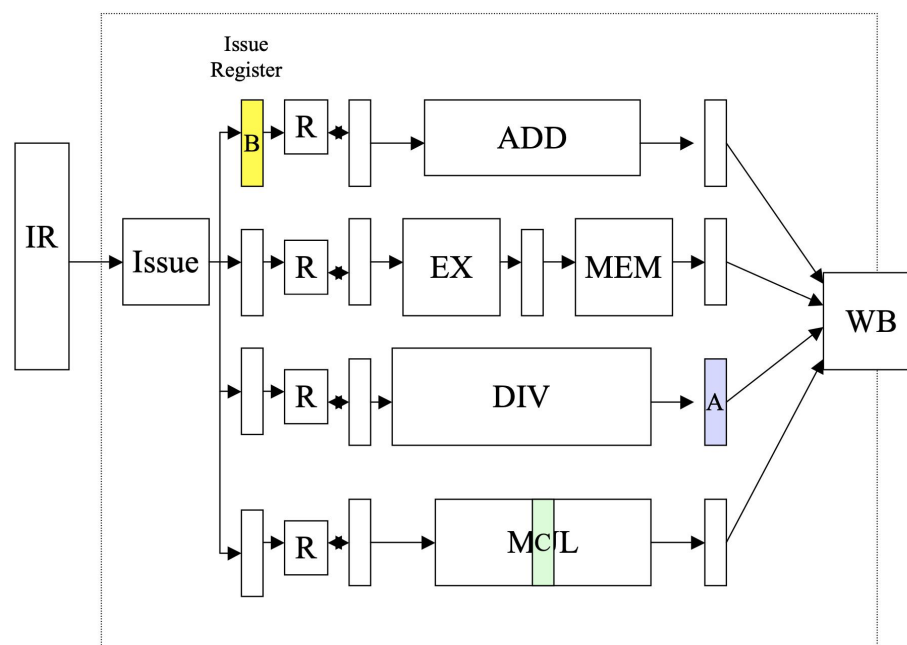


5.3.1 动态调度的基本思想

第 33 页

5. 动态调度的流水线支持多条指令同时处于执行当中。

- 前提要求：具有多个功能部件、或者功能部件流水化、或者兼而有之。
- 假设具有多个功能部件。



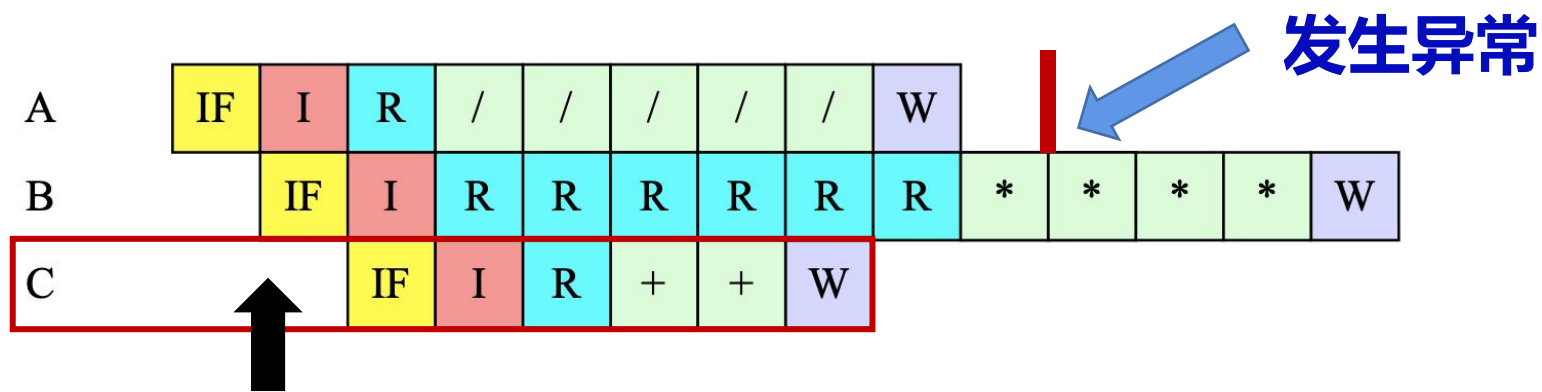
5.3.1 动态调度的基本思想

第 34 页

6. 指令乱序完成带来的最大问题：异常处理比较复杂

■ 动态调度的处理机要保持正确的异常行为

- 对于一条会产生异常的指令来说，只有当处理机确切地知道该指令将被执行时，才允许它产生异常。
- 即使保持了正确的异常行为，动态调度处理机仍可能发生不精确异常。



中断现场恢复时是否执行指令C?

5.3.1 动态调度的基本思想

第 35 页

7. 精确异常处理与不精确异常处理

- 精确异常：如果发生异常时处理机的现场跟严格按程序顺序执行时指令i的现场相同。

条件1：先于A发射的指令都已经执行完



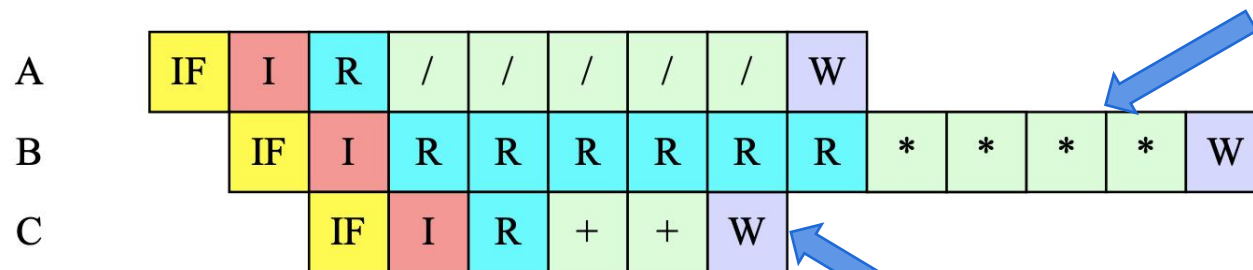
条件2：后于A发射的指令都没有执行完

5.3.1 动态调度的基本思想

第 36 页

7. 精确异常处理与不精确异常处理

- 不精确异常：当执行指令*i*导致发生异常时，处理机的**现场（状态）**与严格按程序顺序执行时指令*i*的**现场（状态）**不同。
- 发生不精确异常的原因：因为当发生异常（设为指令*i*）时，
 - ①流水线可能已经执行完按程序顺序是位于指令*i*之后的指令；
 - ②流水线可能还没完成按程序顺序是指令*i*之前的指令。



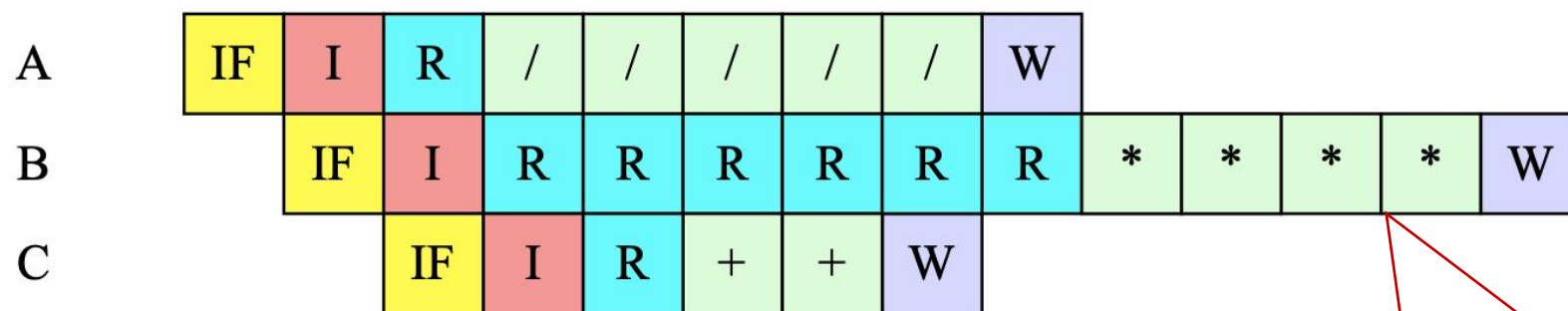
情形1：B指令发生中断，后指令C已经执行完

情形2：C指令发生中断，前指令B已经执行完

5.3.1 动态调度的基本思想

第 37 页

设计目标：提升CPI或IPC（性能），同时保证异常行为和数据流不变



问题1：如何调度指令执行？

原则：（1）无结构冲突，发射指令；
（2）无数据冲突，执行指令。

问题2：如何保证精确中断？

原则：（1）先中断的指令都执行完成；
（2）后中断的指令都不被执行。

记分牌算法和Tomasulo算法是两种比较典型的动态调度算法

5.3.2 记分牌动态调度算法

第 38 页

1. 基本概念

■ CDC 6600 (1963) 计算机最早采用一个称为记分牌 (Scoreboard, 记分板) 的硬件实现了对指令的动态调度。

■ **记分牌目标**: 在没有结构冲突时, 尽可能早地执行没有数据冲突的指令, 实现每个时钟周期执行一条指令

■ **记分牌实现**: 硬件维护着用于记录指令的三张表

A	IF	I	R	/	/	/	/	/	W					
B		IF	I	R	R	R	R	R	R	*	*	*	*	W
C			IF	I	R	+	+	W						

调度: (1) 无结构冲突, 发射指令;
(2) 无数据冲突, 执行指令。

➤ **执行状态表**: 当前流出的所有指令

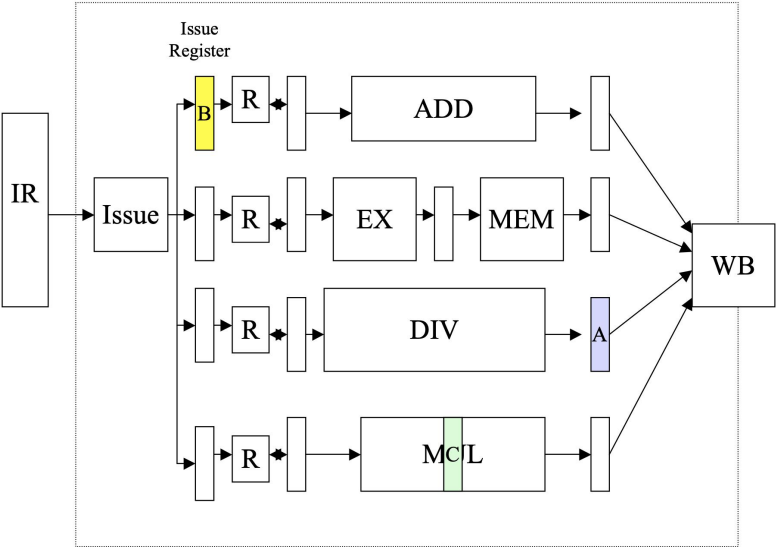
➤ **功能部件状态表**: 当前流出的所有指令是否存在**结构冲突**

➤ **寄存器状态表**: 当前流出的所有指令是否存在**数据冲突**

5.3.2 记分牌动态调度算法

2. 举例：采用了记分牌的MIPS处理器的基本结构

- 每条指令都要经过记分牌，记分牌负责相关检测并控制指令的流出和执行。
- 每条指令的执行过程分为4段（主要考虑浮点操作）：
 - ✓ 流出、读操作数、执行和写结果
- **指令状态表：**记录正在执行的各条指令已经进入到了哪一段。



指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	✓	✓	✓	
L.D	F2, 45(R3)	✓	✓		
MULT.D	F0, F2, F4	✓			
SUB.D	F8, F6, F2				
DIV.D	F10, F0, F6				
ADD.D	F6, F8, F2				

5.3.2 记分牌动态调度算法

第 40 页

与一般MIPS指令将指令周期分为取指令、指令译码、执行指令、存储器访问和分支完成以及写回五个周期不同，记分牌算法中将所有指令都划分为如下的四个阶段：

➤ 流出 <- 无资源 (WAW) 冲突 (执行部件和寄存器)

如果当前流出指令所需的功能部件空闲，并且所有其他正在执行的指令的目的寄存器与该指令的不同，记分牌就向功能部件流出该指令，并修改记分牌内部的记录表。

作用：可以解决了WAW冲突

➤ 读操作数 <- 无数据 (RAW) 冲突 (寄存器)

记分牌监测源操作数的可用性，如果数据可用，它就通知功能部件从寄存器中读出源操作数并开始执行。

作用：动态地解决了RAW冲突，并导致指令可能乱序开始执行。

5.3.2 记分牌动态调度算法

第 41 页

➤ 执行

取到操作数后，功能部件开始执行。当产生出结果后，就通知记分牌它已经完成执行。注意：在浮点流水线中，这一段可能要占用多个时钟周期。

➤ 写结果 <- 无数据 (WAR) 冲突 (寄存器)

记分牌一旦知道执行部件完成了执行，就检测是否存在WAR冲突。

✓ **如果不存在WAR冲突**，或者原有的WAR冲突已消失，记分牌就通知功能部件把结果写入目的寄存器，并释放该指令使用的所有资源。

✓ **如果检测到WAR冲突**，就不允许该指令将结果写到目的寄存器。这发生在以下情况：

- ❑ 前面的某条指令（按顺序流出）还没有读取操作数，而且其中某个源操作数寄存器与本指令的目的寄存器相同。在这种情况下，**记分牌必须等待，直到该冲突消失**。

5.3.2 记分牌动态调度算法

第 42 页

➤ **功能部件状态表：**记录各个功能部件的状态。每个功能部件有一项，每一项由9个字段组成。

一段程序： L.D F2, 34(R2)
 MULT.D F0, F2, F4

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	MULT.D	F0	F2	F4	Integer		No	Yes
Mult2									
Add			RAW					WAR	
Divide									

- Busy：忙标志，指出功能部件是否忙。初值为“no”；
- Op：该功能部件正在执行或将要执行的操作；
- F_i：目的寄存器编号；
- F_j, F_k：源寄存器编号；
- Q_j, Q_k：指出向源寄存器F_j、F_k写数据的功能部件；
- R_j, R_k：标志位，为“Yes”表示F_j, F_k中的操作数就绪且还未被取走；否则就被置为“No”。

5.3.2 记分牌动态调度算法

第 43 页

➤ **结果寄存器状态表：** 每个寄存器在该表中有一项，用于指出哪个功能部件（编号）将把结果写入该寄存器。

✓ 如果当前正在运行的指令都不以它为目的寄存器，则其相应项置为“no”。

✓ Result各项的初值为“no”（全0）。

一段程序： L.D F2, 34(R2)
 MULT.D F0, F2, F4

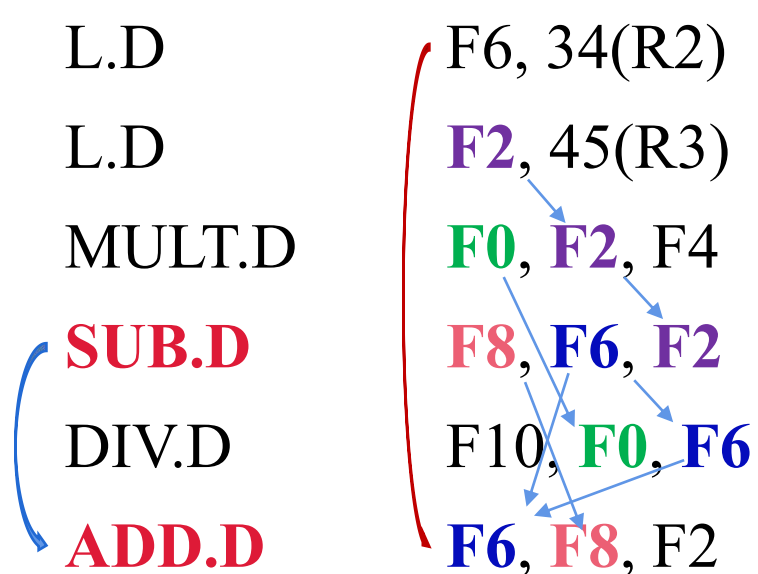
	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1	Integer						

WAW? 会不会指向两个部件？

5.3.2 记分牌动态调度算法

第 44 页

例5.1 假设浮点流水线中各部件的延迟如下：加法需2个时钟周期、乘法需10个时钟周期、除法需40个时钟周期，完成程序执行的记分牌状态。



解：代码段存在以下相关性

(1) 先写后读 (RAW) 相关：第二条L.D指令到MULT.D和SUB.D之间，MULT.D到DIV.D之间，SUB.D到ADD.D之间；

(2) 先读后写 (WAR) 相关：DIV.D和ADD.D之间，SUB.D和ADD.D之间；

(3) 写写 (WAW) 相关：L.D和ADD.D之间 (**可忽略**)；

(4) 结构相关：ADD.D和SUB.D指令关于浮点加法部件。

5.3.2 记分牌动态调度算法

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCKFU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)				
L.D F2, 45(R3)				
MULT.D F0, F2, F4		初始化		
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	No								
Mult1	No								
Mult2	No								
Add	No								
Divide	No								

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称								

5.3.2 记分牌动态调度算法

第 46 页

具体算法，约定：

- FU：表示当前指令所要用的功能部件；
- D：目的寄存器的名称；
- S_1 、 S_2 ：源操作数寄存器的名称；
- Op：要进行的操作；
- $F_j[FU]$ ：功能部件FU的 F_j 字段（其他字段依此类推）；
- Result[D]：结果寄存器状态表中与寄存器D相对应的内容。其中存放的是将把结果写入寄存器D的功能部件的名称。

5.3.2 记分牌动态调度算法

第 47 页

(1) 指令流出

进入条件:

`not Busy[FU] & not Result[D];` // **功能部件空闲且没有写后写 (WAW) 冲突**

记分牌内容修改:

`Busy[FU] ← yes;`

// 把当前指令的相关信息填入功能部件状态表, 功能部件状态表中各字段的含义见前面

`Op[FU] ← Op;` // 记录操作码

`Fi[FU] ← D;` // 记录目的寄存器编号

`Fj[FU] ← S1;` // 记录第一个源寄存器编号

关键点一：功能部件空闲和无WAW冲突决定指令是否发射？

5.3.2 记分牌动态调度算法

第 48 页

$F_k[FU] \leftarrow S_2;$	// 记录第二个源寄存器编号
$Q_j[FU] \leftarrow \text{Result}[S_1];$	// 记录将产生第一个源操作数的部件
$Q_k[FU] \leftarrow \text{Result}[S_2];$	// 记录将产生第二个源操作数的部件
$R_j[FU] \leftarrow \text{not } Q_j[FU];$	// 置第一个源操作数是否可用的标志。如果 $Q_j[FU]$ 为 “no” ,
就表示没有操作部件要写 S_1 ,	数据可用。置 $R_j[FU]$ 为 “yes” 。否则置 $R_j[FU]$ 为 “no”
$R_k[FU] \leftarrow \text{not } Q_k[FU];$	// 置第二个源操作数是否可用的标志
$\text{Result}[D] \leftarrow FU;$	// D是当前指令的目的寄存器。功能部件FU将把结果写入D

5.3.2 记分牌动态调度算法

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

发射第1个L.D
指令

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
1FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1			
L.D F2, 45(R3)				
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F6		R2				Yes
Mult1	No								
Mult2	No								
Add	No								
Divide	No								

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称				Integer				

5.3.2 记分牌动态调度算法

第 50 页

(2) 读操作数

进入条件:

$R_j[FU] \ \& \ R_k[FU];$ // 两个源操作数都已就绪

记分牌内容修改:

$R_j[FU] \leftarrow \text{no};$ // 已经读走了就绪的第一个源操作数

$R_k[FU] \leftarrow \text{no};$ // 已经读走了就绪的第二个源操作数

$Q_j[FU] \leftarrow 0;$ // 不再等待其他FU的计算结果

$Q_k[FU] \leftarrow 0;$

5.3.2 记分牌动态调度算法

第 51 页

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

是否发射第2个
L.D指令?

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
2
FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2		
L.D F2, 45(R3)				
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F6		R2				No
Mult1	No								
Mult2	No								
Add	No								
Divide	No								

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称				Integer				

5.3.2 记分牌动态调度算法

第 52 页

(3) 执行

结束条件：功能部件操作结束。

5.3.2 记分牌动态调度算法

第 53 页

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

是否发射
MULT.D指令?

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
3
FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	
L.D F2, 45(R3)				
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F6		R2			No	
Mult1	No								
Mult2	No								
Add	No								
Divide	No								

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称				Integer				

5.3.2 记分牌动态调度算法

第 54 页

(4) 写结果

进入条件：

$\forall f((F_j[f] \neq F_i[FU] \text{ or } R_j[f] = \text{no}) \& (F_k[f] \neq F_i[FU] \text{ or } R_k[f] = \text{no}));$ // **不存在WAR冲突**

记分牌内容修改：

$\forall f(\text{if } Q_j[f] = FU \text{ then } R_j[f] \leftarrow \text{yes});$ //如果有指令在等待该结果（作为第一源操作数），则将其Rj置为“yes”，表示数据可用

$\forall f(\text{if } Q_k[f] = FU \text{ then } R_k[f] \leftarrow \text{yes});$ //如果有指令在等待该结果（作为第二源操作数），则将其Rk置为“yes”，表示数据可用

$\text{Result}(F_i[FU]) \leftarrow 0;$ // 释放目的寄存器 $F_i[FU]$

$\text{Busy}[FU] = \text{No};$ // 释放功能部件FU

5.3.2 记分牌动态调度算法

第 55 页

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

可以发射第2个
L.D指令

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
4FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)				
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	No								
Mult2	No								
Add	No								
Divide	No								

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称								

5.3.2 记分牌动态调度算法

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

可以发射
MULT.D指令

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
5
FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5			
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F2	R3				Yes	
Mult1	No								
Mult2	No								
Add	No								
Divide	No								

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称		Integer						

5.3.2 记分牌动态调度算法

第 57 页

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

可以发射
SUB.D指令

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
6
FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6		
MULT.D F0, F2, F4	6			
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	No								
Divide	No								

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1	Integer						

5.3.2 记分牌动态调度算法

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

需要读两次F2?
读多个数据?
可以发射DIV.D
指令

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
7
FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	
MULT.D F0, F2, F4	6			
SUB.D F8, F6, F2	7			
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	No								

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1	Integer			Add			

5.3.2 记分牌动态调度算法

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

可以发射
ADD.D指令?

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
8a
FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	
MULT.D F0, F2, F4	6			
SUB.D F8, F6, F2	7			
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1	Integer			Add	Divide		

5.3.2 记分牌动态调度算法

第 60 页

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

可以发射
ADD.D指令?

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
8b
FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6			
SUB.D F8, F6, F2	7			
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2			Yes	Yes
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1				Add	Divide		

5.3.2 记分牌动态调度算法

第 61 页

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

可以发射
ADD.D指令?

Integer	10
Mult1	
Mult2	
Add	2
Divide	

结果寄存器状态:

CLOCK	FU
9	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9		
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes	Sub	F8	F6	F2			No	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1				Add	Divide		

5.3.2 记分牌动态调度算法

第 62 页

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

可以发射
ADD.D指令?

9	Integer
	Mult1
	Mult2
1	Add
	Divide

结果寄存器状态:

CLOCK	FU
10	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9		
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes	Sub	F8	F6	F2			No	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1				Add	Divide		

5.3.2 记分牌动态调度算法

第 63 页

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

可以发射
ADD.D指令?

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
11

FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9	11	
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes	Sub	F8	F6	F2			No	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1				Add	Divide		

5.3.2 记分牌动态调度算法

第 64 页

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

可以发射
ADD.D指令

7

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
12
FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes								
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1					Divide		

5.3.2 记分牌动态调度算法

第 65 页

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
6	Mult1
	Mult2
	Add
	Divide

结果寄存器状态:

CLOCK	FU
13	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2	13			

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes	Add	F6	F8	F2			Yes	Yes
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1			Add		Divide		

5.3.2 记分牌动态调度算法

第 66 页

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
5	Mult1
	Mult2
2	Add
	Divide

结果寄存器状态:

CLOCK	FU
14	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2	13	14		

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes	Add	F6	F8	F2			No	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1			Add		Divide		

5.3.2 记分牌动态调度算法

第 67 页

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
4	Mult1
	Mult2
1	Add
	Divide

结果寄存器状态:

CLOCK	FU
15	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2	13	14		

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes	Add	F6	F8	F2			No	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1			Add		Divide		

5.3.2 记分牌动态调度算法

第 68 页

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
3	Mult1
	Mult2
0	Add
	Divide

结果寄存器状态:

CLOCK	FU
16	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes	Add	F6	F8	F2			No	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1			Add		Divide		

5.3.2 记分牌动态调度算法

第 69 页

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2 F4	
SUB.D	F8	F6 F2	
DIV.D	F10	F0 F6	
ADD.D	F6	F8 F2	

功能部件状态:

	Integer
2	Mult1
	Mult2
	Add
	Divide

结果寄存器状态:

CLOCK	FU
17	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No					WAR冲突			
Add	Yes	Add	F6	F8	F2			No	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1			Add		Divide		

5.3.2 记分牌动态调度算法

第 70 页

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
1	Mult1
	Mult2
	Add
	Divide

结果寄存器状态:

CLOCK	FU
18	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes	Add	F6	F8	F2			No	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1			Add		Divide		

5.3.2 记分牌动态调度算法

第 71 页

指令状态:

		j	k
L.D	F6	34	+ R2
L.D	F2	45	+ R3
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
0	Mult1
	Mult2
	Add
	Divide

结果寄存器状态:

CLOCK	FU
19	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9	19	
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	Yes	Mult	F0	F2	F4			No	No
Mult2	No								
Add	Yes	Add	F6	F8	F2			No	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称	Mult1			Add		Divide		

5.3.2 记分牌动态调度算法

第 72 页

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

Integer
Mult1
Mult2
2 Add
Divide

结果寄存器状态:

CLOCK	FU
20	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9	19	20
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	No								
Mult2	No								
Add	Yes	Add	F6	F8	F2			No	No
Divide	Yes	Div	F10	F0	F6			Yes	Yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称				Add		Divide		

5.3.2 记分牌动态调度算法

第 73 页

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Mult1
	Mult2
1	Add
40	Divide

结果寄存器状态:

CLOCK	FU
21	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9	19	20
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8	21		
ADD.D F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	No								
Mult2	No								
Add	Yes	Add	F6	F8	F2			No	No
Divide	Yes	Div	F10	F0	F6			No	No

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称				Add		Divide		

5.3.2 记分牌动态调度算法

第 74 页

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Mult1
	Mult2
0	Add
39	Divide

结果寄存器状态:

CLOCK	FU
22	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9	19	20
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8	21		
ADD.D F6, F8, F2	13	14	16	22

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	No								
Mult2	No								
Add	No								
Divide	Yes	Div	F10	F0	F6			No	No

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称						Divide		

5.3.2 记分牌动态调度算法

第 75 页

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

Integer
Mult1
Mult2
Add
38→0 Divide

结果寄存器状态:

CLOCK	FU
23→61	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9	19	20
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8	21	61	
ADD.D F6, F8, F2	13	14	16	22

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	No								
Mult2	No								
Add	No								
Divide	Yes	Div	F10	F0	F6			No	No

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称						Divide		

5.3.2 记分牌动态调度算法

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK
62
FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9	19	20
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8	21	61	62
ADD.D F6, F8, F2	13	14	16	22

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	No								
Mult2	No								
Add	No								
Divide	No								

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称								

5.3.2 记分牌动态调度算法

第 77 页

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

Integer
Mult1
Mult2
Add
Divide

结果寄存器状态:

CLOCK	FU
62	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	按序发射 (issue) 乱序执行 (execute) 和提交(commit)		4
L.D F2, 45(R3)	5			8
MULT.D F0, F2, F4	6			20
SUB.D F8, F6, F2	7			12
DIV.D F10, F0, F6	8	21	61	62
ADD.D F6, F8, F2	13	14	16	21

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	No								
Mult2	No								
Add	No								
Divide	No								

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称								

5.3.2 记分牌动态调度算法

第 78 页

程序：

L.D	F6	34 + R2
L.D	F2	45 + R3
MULT.D	F0	F2 F4
SUB.D	F8	F6 F2
DIV.D	F10	F0 F6
ADD.D	F6	F8 F2

[illegible]

5.3.2 记分牌动态调度算法

第 79 页

程序:

```
L.D    F6    34 + R2
L.D    F2    45 + R3
MULT.D F0    F2    F4
SUB.D  F8    F6    F2
DIV.D  F10   F0    F6
ADD.D  F6    F8    F2
```

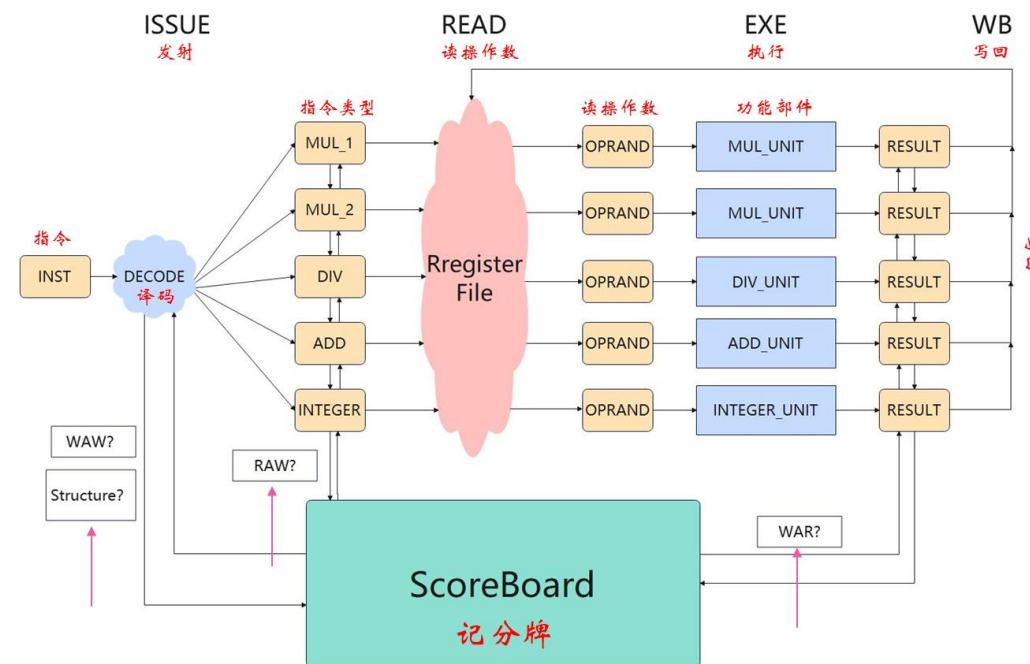
	11	12	13	14	15	16	17	18	19	20	21	22	~62
LD F6, 34(R2)													
LD F2, 45(R3)													
MULTD F0, F2, F4	M2	M3	M4	M5	M6	M7	M8	M9	M10	WB			
SUBD F8, F6, F2	A2	WB											
DIVD F10, F0, F6	ISS	ISS	ISS	ISS	ISS	ISS	ISS	ISS	ISS	ISS	RD	D1	WB
ADDD F6, F8, F2			ISS	RD	A1	A2	A2	A2	A2	A2	A2	WB	

5.3.2 记分牌动态调度算法

第 80 页

■ 记分牌是一种应用较早的方法

- CDC 6600 (1963) 计算机最早采用此功能
- 按序发射, 乱序执行, 乱序提交 (完成)
- 当时没有 Cache, 没有 Forwarding
- RAW: 等待写入寄存器, 下一周期读取
- **WAW: 在发射阶段等待**
- **WAR: 在提交阶段等前面操作数读毕**



5.3.2 记分牌动态调度算法

第 81 页

- 记分牌算法的优点：实现指令的乱序执行，解决了乱序执行过程中数据冒险问题，实现了指令的数据流式运行，即数据一旦准备好就开始运行，且实现简单。
- 记分牌算法的缺点：
 - 因WAR和WAW冒险而产生阻塞，一旦产生阻塞：
 1. 后续相同类型的指令就没办法继续发射（在乱序执行过程中记分牌规定每一条配置路线都只能同时存在一条指令）；
 2. 如果后续相同类型的指令没法发射，那么更后面的也许可以立马执行的指令也会被阻塞到，这对性能有很大的影响。

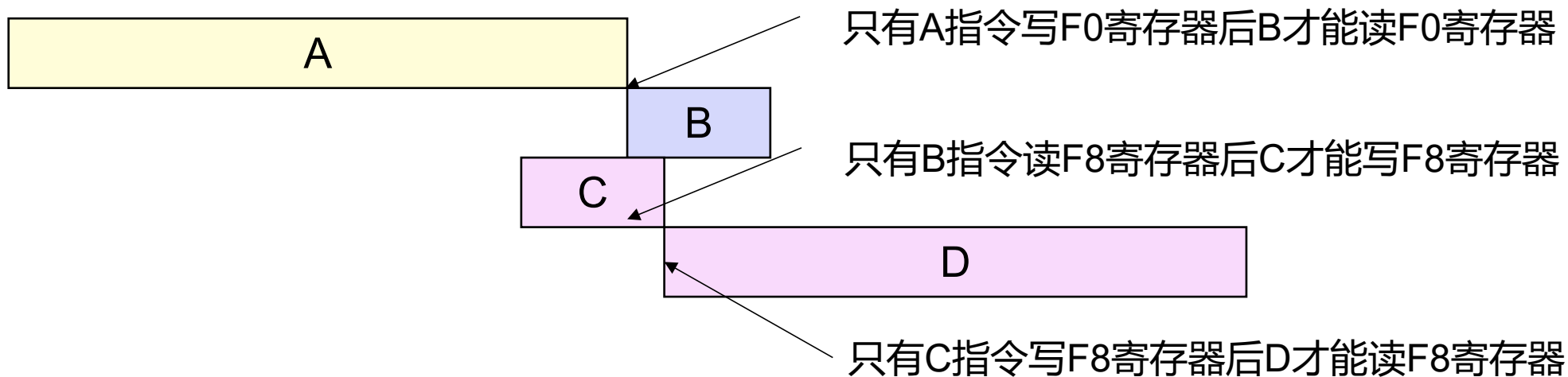
如何进一步解决WAR和WAW冒险问题？

5.3.3 显式动态寄存器重命名

第 82 页

■ WAR名相关

A: DIVD	F0 , F2, F4	X=Y/Z
B: ADDD	F6, F0 , F8	P=X-Q
C: SUBD	F8 , F10, F12	Q=R-S
D: DIVD	F14, F8 , F16	T=Q/U



5.3.3 显式动态寄存器重命名

第 83 页

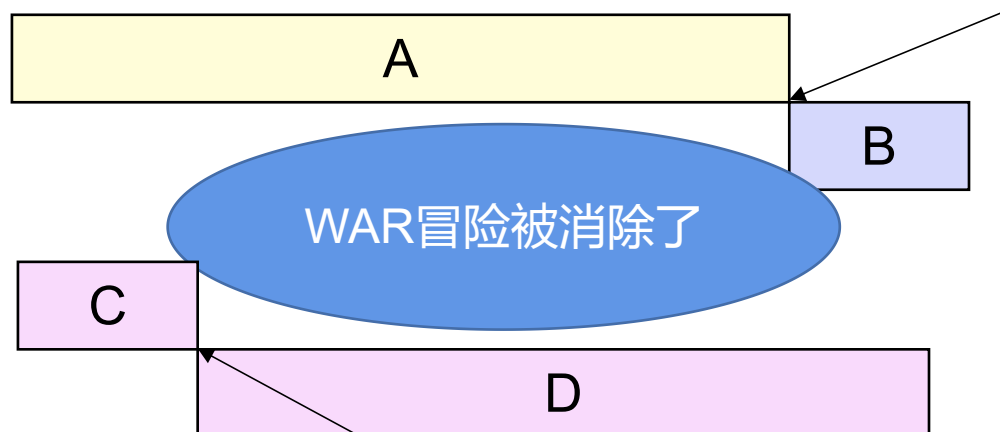
■ 通过寄存器重命名消除WAR名相关

A: DIVD	F0, F2, F4
B: ADDD	F6, F0, F8
C: SUBD	F8, F10, F12
D: DIVD	F14, F8, F16

X=Y/Z
P=X-Q
Q=R-S
T=Q/U

寄存器
重命名

A: DIVD	F0, F2, F4
B: ADDD	F6, F0, F8
C: SUBD	T1, F10, F12
D: DIVD	F14, T1, F16



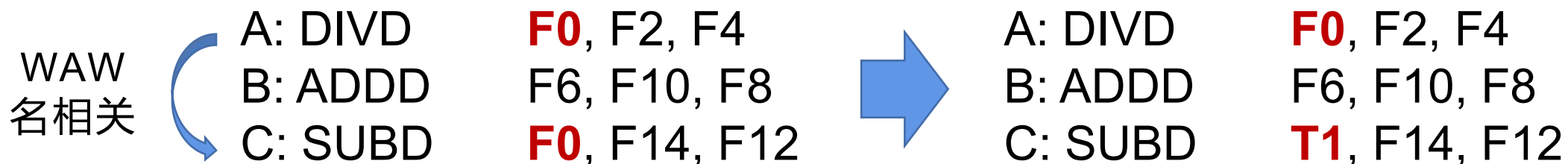
只有A指令写F0寄存器后B才能读F0寄存器

只有C指令写T1寄存器后D才能读T1寄存器

5.3.3 显式动态寄存器重命名

第 84 页

■ 通过寄存器重命名消除WAW名相关



RAW
ADD R3, R1, R2
AND R5, R3, R4

真相关

R3重命名
为R10

WAW

ADD	R3, R1, R2
AND	R5, R3, R4
AND	R3, R6, R7
ADD	R9, R3, R8



ADD	R3 , R1, R2
AND	R5, R3 , R4
AND	R10 , R6, R7
ADD	R9, R10 , R8

WAR

ADD	R1, R3, R2
AND	R3, R5, R4



ADD	R1, R3 , R2
AND	R10 , R5, R4

5.3.3 显式动态寄存器重命名

第 85 页

■ 一个复杂的LOOP实例

LOOP:

A: LD **F0**, 0(R0)

B: MUL **F4**, **F0**, F2

C: SD **F4**, 0(R0)

D: ADDI **R1**, **R1**, #8

E: BNE **R1**, R2, LOOP

| temp=x[i]

| temp=temp * a

| x[i]=temp

| i++

| branch if R1.ne.R2

RAW

← WAW

← WAR

←

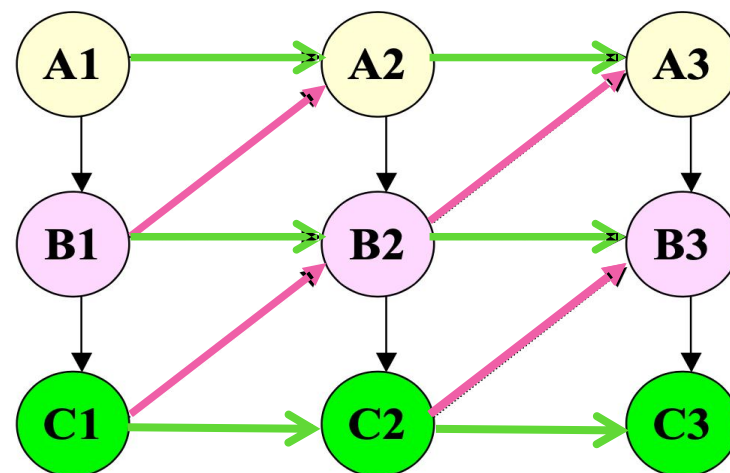
循环体内部数据相关与名相关 (RAW、WAR、WAW)

5.3.3 显式动态寄存器重命名

第 86 页

■ 一个复杂的LOOP实例

循环体 1	{	A1: LD F0 , 0(R0)
		B1: MUL F4 , F0 , F2
		C1: SD F4 , 0(R0)
		D1: ADDI R1 , R1, #8
循环体 2	{	A2: LD F0 , 0(R0)
		B2: MUL F4 , F0 , F2
		C2: SD F4 , 0(R0)
		D2: ADDI R1 , R1, #8
循环体 3	{	A2: LD F0 , 0(R0)
		B2: MUL F4 , F0 , F2
		C2: SD F4 , 0(R0)
		D2: ADDI R1 , R1, #8



5.3.3 显式动态寄存器重命名

第 87 页

■ 一个复杂的LOOP实例

循环体 1

- A1: LD **F0**, 0(R0)
- B1: MUL **F4**, **F0**, F2
- C1: SD **F4**, 0(R0)
- D1: ADDI **R1**, R1, #8

循环体 2

- A2: LD **F0**, 0(R0)
- B2: MUL **F4**, **F0**, F2
- C2: SD **F4**, 0(R0)
- D2: ADDI **R1**, R1, #8

循环体 3

- A2: LD **F0**, 0(R0)
- B2: MUL **F4**, **F0**, F2
- C2: SD **F4**, 0(R0)
- D2: ADDI **R1**, R1, #8



修改
输出
寄存器号

A1: LD **F0**, 0(R0)

B1: MUL **F4**, **F0**, F2

C1: SD **F4**, 0(R0)

D1: ADDI **R1**, R1, #8

A2: LD **F6**, 0(R0)

B2: MUL **F8**, **F6**, F2

C2: SD **F8**, 0(R0)

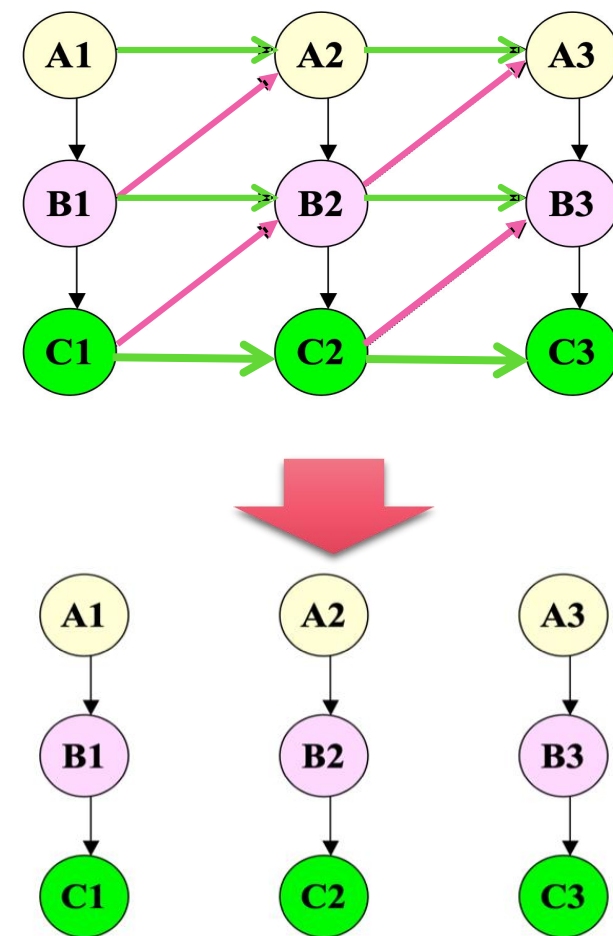
D2: ADDI **R1**, R1, #8

A2: LD **F10**, 0(R0)

B2: MUL **F12**, **F10**, F2

C2: SD **F12**, 0(R0)

D2: ADDI **R1**, R1, #8



5.3.3 显式动态寄存器重命名

第 88 页

■ 显式寄存器重命名

➤ 结构寄存器 (ISA Register, AR) —命名 (Name)

✓ 对软件可见

ISA Register File

→ ARF

✓ 程序编码使用

Physical Register File

→ PRF

➤ 物理寄存器 (Physical Register, PR) —地址 (Location)

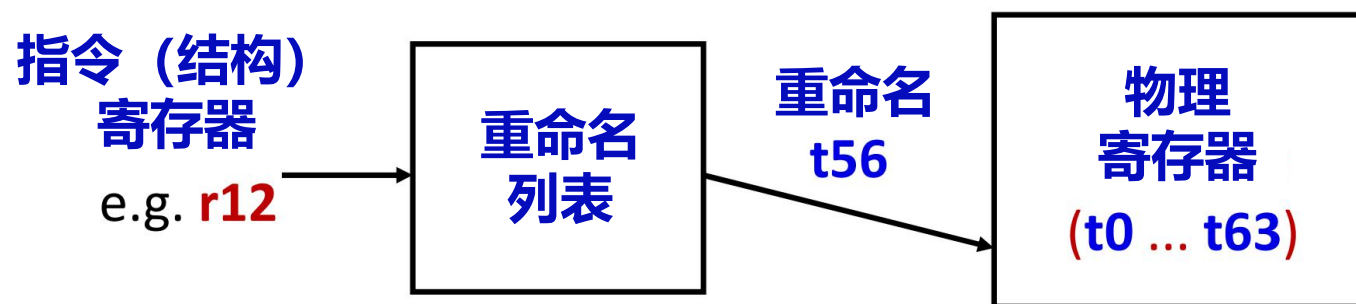
✓ 真正存放信息的寄存器

✓ 由硬件控制

➤ 重命名

✓ Name—Location

✓ AR→PR需要一个映射表 rename table



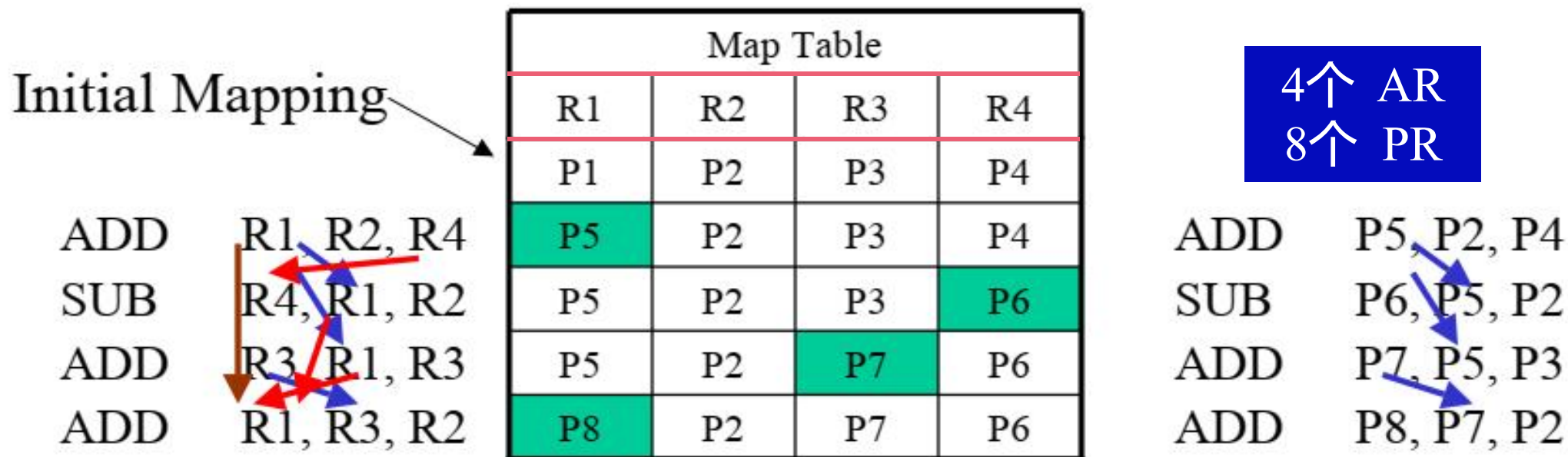
通过编译可以实现静态寄存器重命名, 可消除 WAR、WAW 名相关, 但是受限于寄存器数量

5.3.3 显式动态寄存器重命名

第 89 页

■ 显式动态寄存器重命名

- 硬件维护一个映射表 (Rename Table)
 - ✓ 写寄存器：为该结构寄存器 R_i (Name) 分配一个新的物理寄存器 P_j (Location) 建立映射关系；
 - ✓ 读寄存器：查找映射表，查找最近一次写寄存器 R_w

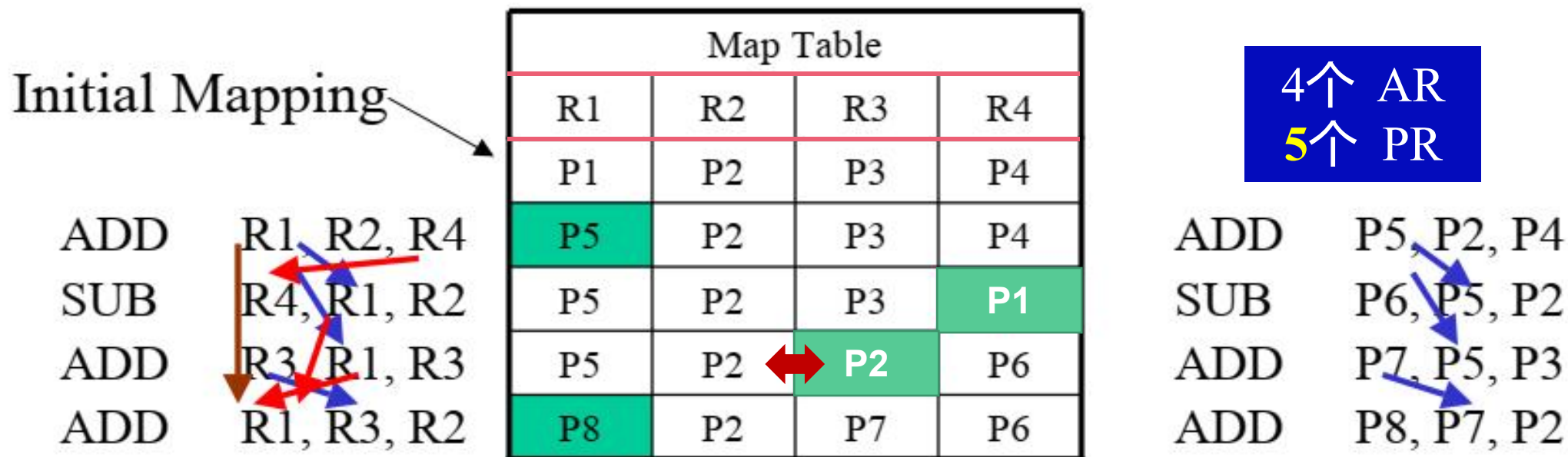


5.3.3 显式动态寄存器重命名

第 90 页

■ 显式动态寄存器重命名

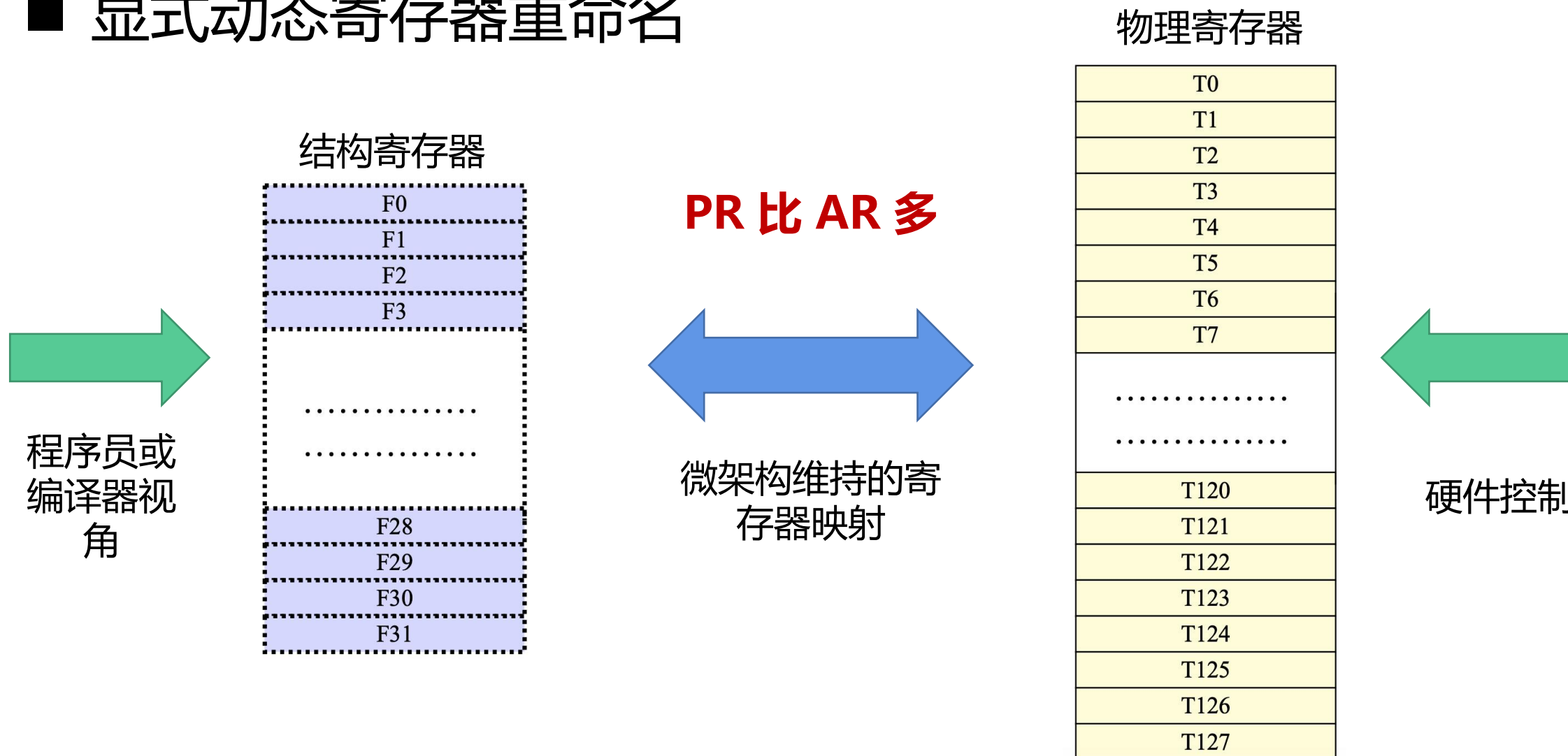
- 硬件维护一个映射表 (Rename Table)
 - ✓ 写寄存器：为该结构寄存器 R_i (Name) 分配一个新的物理寄存器 P_j (Location) 建立映射关系；
 - ✓ 读寄存器：查找映射表，查找最近一次写寄存器 R_w



5.3.3 显式动态寄存器重命名

第 91 页

■ 显式动态寄存器重命名



5.3.3 显式动态寄存器重命名

第 92 页

■ 显式动态寄存器重命名

空闲物理寄存器列表

T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15	T16	T17
----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----

重命名映射表

AR	PR
F0	T0
F1	T1
F2	T2
F3	T3
F4	T4
F5	T5
F6	T6
F7	T7
F8	T8

原程序：

A: DIVD F0, F2, F4
B: AD44 F6, F0, F8
C: SUBD F8, F3, F5
D: DIVD F0, F8, F0



初始状态

5.3.3 显式动态寄存器重命名

第 93 页

■ 显式动态寄存器重命名

空闲寄存器列表

T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15	T16	T17
----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----

原程序:

A: DIVD F0, F2, F4
B: AD44 F6, F0, F8
C: SUBD F8, F3, F5
D: DIVD F0, F8, F0

重命名映射表

AR	PR
F0	T9
F1	T1
F2	T2
F3	T3
F4	T4
F5	T5
F6	T6
F7	T7
F8	T8

A: DIVD **T9**, T2, T4

5.3.3 显式动态寄存器重命名

第 94 页

■ 显式动态寄存器重命名

空闲寄存器列表

T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15	T16	T17
----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----

原程序:

A: DIVD F0, F2, F4
B: AD44 F6, F0, F8
C: SUBD F8, F3, F5
D: DIVD F0, F8, F0

重命名映射表

AR	PR
F0	T9
F1	T1
F2	T2
F3	T3
F4	T4
F5	T5
F6	T10
F7	T7
F8	T8

A: DIVD **T9**, T2, T4
B: AD44 **T10**, T9, T8

5.3.3 显式动态寄存器重命名

第 95 页

■ 显式动态寄存器重命名

空闲寄存器列表

T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15	T16	T17
----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----

原程序:

A: DIVD F0, F2, F4
B: AD DD F6, F0, F8
C: SUBD F8, F3, F5
D: DIVD F0, F8, F0

重命名映射表

AR	PR
F0	T9
F1	T1
F2	T2
F3	T3
F4	T4
F5	T5
F6	T10
F7	T7
F8	T11

A: DIVD **T9**, T2, T4
B: AD DD **T10**, T9, T8
C: SUBD **T11**, T3, T5

5.3.3 显式动态寄存器重命名

第 96 页

■ 显式动态寄存器重命名

空闲寄存器列表

释放

T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15	T16	T17
----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----

原程序:

A: DIVD
B: AD44
C: SUBD
D: DIVD

F0, F2, F4
F6, F0, F8
F8, F3, F5
F0, F8, F0

重命名映射表

AR	PR
F0	T12
F1	T1
F2	T2
F3	T3
F4	T4
F5	T5
F6	T10
F7	T7
F8	T11

A: DIVD
B: AD44
C: SUBD
D: DIVD

T9, T2, T4
T10, T9, T8
T11, T3, T5
T12, T11, T9

5.3.3 显式动态寄存器重命名

第 97 页

例5.2 假设浮点流水线中各部件的延迟如下：加法需2个时钟周期、乘法需10个时钟周期、除法需40个时钟周期，完成程序执行的显式动态寄存器重命名记分牌状态。

L.D	F6, 34(R2)	解：代码段存在以下相关性 (1) 先写后读相关：第二条L.D指令到MULT.D和SUB.D之间， MULT.D到DIV.D之间， SUB.D到ADD.D之间； (2) 先读后写相关： DIV.D和ADD.D之间， SUB.D和ADD.D之间； (3) 结构相关： ADD.D和SUB.D指令关于浮点加法部件。
L.D	F2 , 45(R3)	
MULT.D	F0 , F2 , F4	
SUB.D	F8 , F6 , F2	
DIV.D	F10, F0 , F6	
ADD.D	F6 , F8 , F2	

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

- 初始状态
- 每条指令分配空闲寄存器
 - 类似于单赋值编译器转换

Integer
Integer
Mult
Add
Divide

寄存器重命名与结果:

CLOCKFU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)				
L.D F2, 45(R3)				
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1									
INT2									
MULT									
ADD									
Divide									

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P0	P2	P4	P6	P8	P10	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

Integer
Integer
Mult
Add
Divide

寄存器重命名与结果:

CLOCK
1FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1			
L.D F2, 45(R3)				
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	yes	load	P32		R2				yes
INT2	no								
MULT	no								
ADD	no								
Divide	no								

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P0	P2	P4	P32	P8	P10	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

Integer
Integer
Mult
Add
Divide

寄存器重命名与结果:

CLOCK
2FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2		
L.D F2, 45(R3)	2			
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	yes	load	P32		R2				no
INT2	yes	load	P34		R3				yes
MULT	no								
ADD	no								
Divide	no								

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P0	P34	P4	P32	P8	P10	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

Integer
Integer
Mult
Add
Divide

寄存器重命名与结果:

CLOCK
3FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	
L.D F2, 45(R3)	2	3		
MULT.D F0, F2, F4	3			
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	yes	load	P32		R2				no
INT2	yes	load	P34		R3				no
MULT	yes	mult	P36	P34	P4	INT2		No	yes
ADD	no								
Divide	no								

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P32	P8	P10	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

Integer
Integer
Mult
Add
Divide

寄存器重命名与结果:

CLOCK
4FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	
MULT.D F0, F2, F4	3			
SUB.D F8, F6, F2	4			
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	yes	load	P34		R3				yes
MULT	yes	mult	P36	P34	P4	INT2		no	yes
ADD	yes	sub	P38	P32	P34		INT2	yes	no
Divide	no								

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P32	P38	P10	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

Integer
Integer
Mult
Add
Divide

寄存器重命名与结果:

CLOCK
5FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3			
SUB.D F8, F6, F2	4			
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			yes	yes
ADD	yes	sub	P38	P32	P34		INT2	yes	yes
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P32	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Integer
10	Mult
2	Add
	Divide

寄存器重命名与结果:

CLOCK	FU
6	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6		
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	yes	sub	P38	P32	P34			no	no
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P32	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Integer
9	Mult
1	Add
	Divide

寄存器重命名与结果:

CLOCK	FU
7	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6		
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	yes	sub	P38	P32	P34			no	no
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P32	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Integer
8	Mult
0	Add
	Divide

寄存器重命名与结果:

CLOCK	FU
8	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6	8	
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	yes	sub	P38	P32	P34			no	no
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P32	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Integer
7	Mult
	Add
	Divide

寄存器重命名与结果:

CLOCK	FU
9	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6	8	9
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	no								
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P32	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

L.D	F6	j	k
L.D	F2	34 + R2	
MULT.D	F0	45 + R3	
SUB.D	F8	F2 F4	
DIV.D	F10	F6 F2	
ADD.D	F6	F0 F6	
		F8 F2	

功能部件状态:

虽然重命名表中未列出P32, 但P32仍活着, 不必重新分配

Integer
Integer
Mult
Add
Divide

寄存器重命名与结果:

CLOCK
10
FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6	8	9
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	10			

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	yes	add	P42	P38	P34			yes	yes
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P42	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Integer
5	Mult
2	Add
	Divide

寄存器重命名与结果:

CLOCK	FU
11	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6	8	9
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	10	11		

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	yes	add	P42	P38	P34			no	no
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P42	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Integer
4	Mult
1	Add
	Divide

寄存器重命名与结果:

CLOCK	FU
12	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6	8	9
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	10	11		

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	yes	add	P42	P38	P34			no	no
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P42	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Integer
3	Mult
0	Add
	Divide

寄存器重命名与结果:

CLOCK	FU
13	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6	8	9
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	10	11	13	

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	yes	add	P42	P38	P34			no	no
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P42	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Integer
2	Mult
	Add
	Divide

寄存器重命名与结果:

CLOCK	FU
14	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6	8	9
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	10	11	13	14

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	no								
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P42	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Integer
1	Mult
	Add
	Divide

寄存器重命名与结果:

CLOCK	FU
15	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6	8	9
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	10	11	13	14

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	no								
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P42	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

	Integer
	Integer
0	Mult
	Add
	Divide

寄存器重命名与结果:

CLOCK	FU
16	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6	16	
SUB.D F8, F6, F2	4	6	8	9
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	10	11	13	14

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	yes	mult	P36	P34	P4			no	no
ADD	no								
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P42	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

Integer
Integer
Mult
Add
Divide

寄存器重命名与结果:

CLOCK
17FU

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6	16	17
SUB.D F8, F6, F2	4	6	8	9
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	10	11	13	14

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	no								
ADD	no								
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P42	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

指令状态:

		j	k
L.D	F6	34 + R2	
L.D	F2	45 + R3	
MULT.D	F0	F2	F4
SUB.D	F8	F6	F2
DIV.D	F10	F0	F6
ADD.D	F6	F8	F2

功能部件状态:

Integer
Integer
Mult
Add
40 Divide

寄存器重命名与结果:

CLOCK	FU
18	

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
MULT.D F0, F2, F4	3	6	16	17
SUB.D F8, F6, F2	4	6	8	9
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	10	11	13	14

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
INT1	no								
INT2	no								
MULT	no								
ADD	no								
Divide	yes	divd	P40	P36	P32	MLIT		no	yes

	结果寄存器状态表								
	F0	F2	F4	F6	F8	F10	F12	...	F30
部件名称	P36	P34	P4	P42	P38	P40	P12	...	P30

5.3.3 显式动态寄存器重命名

第 117 页

■ 显式寄存器重命名的Scoreboard算法的缺点：

- 快速访问映射表
- 物理寄存器数量比结构寄存器多，没有空闲的物理寄存器⇒ 在issue段停顿
- 需要确定空闲的物理寄存器
- 记分牌算法在指令完成时不是顺序的（即写回的时候不按顺序），不按顺序完成指令会对程序的调试提出挑战
- 管理空闲寄存器复杂
- 难以精确中断

记分牌在GPU的thread scheduling中仍然在用，现代处理器一般采用改进的Tomasulo算法

5.3.4 Tomasulo算法

第 118 页

1. 核心思想

- 记录和检测指令相关，操作数一旦就绪就立即执行，把发生RAW冲突的可能性减少到最小；
- 通过寄存器换名来消除WAR冲突和WAW冲突。

2. IBM 360/91首先采用了Tomasulo算法

- IBM 360/91的设计目标是基于整个360系列的统一指令系统和编译器来实现高性能，而不是设计和利用专用的编译器来提高性能。



IBM 360

5.3.4 Tomasulo算法

第 119 页

1. 核心思想

- 记录和检测指令相关，操作数一旦就绪就立即执行，把发生RAW冲突的可能性减少到最小；
- 通过寄存器换名来消除WAR冲突和WAW冲突。

2. IBM 360/91首先采用了Tomasulo算法

- IBM 360/91的设计目标是基于整个360系列的统一指令系统和编译器来实现高性能，而不是设计和利用专用的编译器来提高性能。



IBM 360

5.3.4 Tomasulo算法

第 120 页

3. 寄存器换名可以消除WAR冲突和WAW冲突

考虑以下代码：

	DIV.D	F0, F2, F4	
反相关 (F8) 导致WAR冲突	{	ADD.D	F6, F0, F8
		S.D	F6, 0 (R1)
		SUB.D	F8, F10, F14
		MUL.D	F6, F10, F8
			输出相关 (F6) 导致WAW冲突

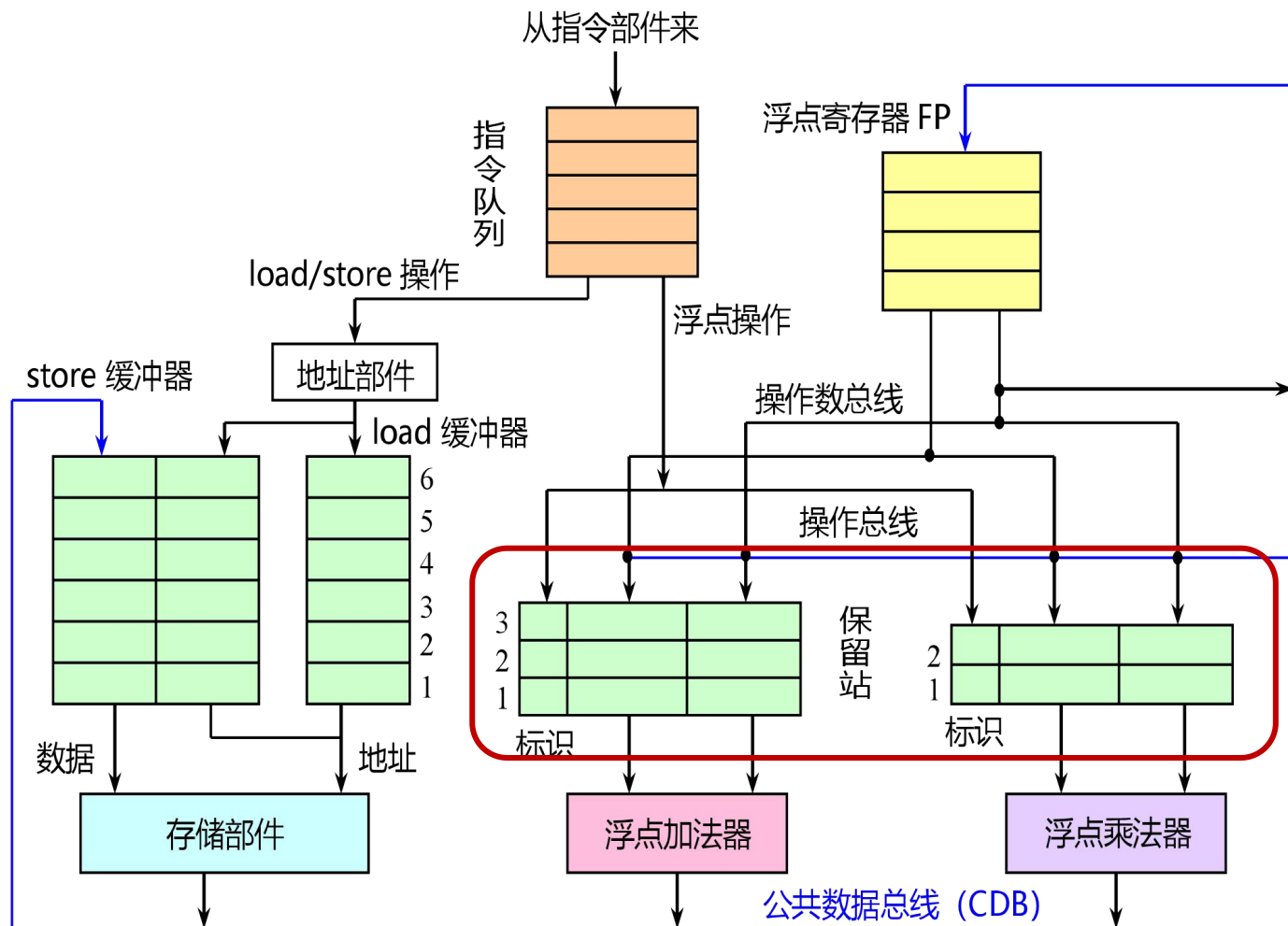
- 采用记分牌算法可能会导致长时间的停顿
- 可以采用消除名相关的办法，引入两个临时寄存器**S**和**T**

	DIV.D	F0, F2, F4	
两个F6都重命名为S	{	ADD.D	S, F0, F8
		S.D	S, 0 (R1)
两个F8都重命名为T	{	SUB.D	T, F10, F14
		MUL.D	F6, F10, T

5.3.4 Tomasulo算法

第 121 页

4. 基于Tomasulo算法的MIPS处理器浮点部件的基本结构



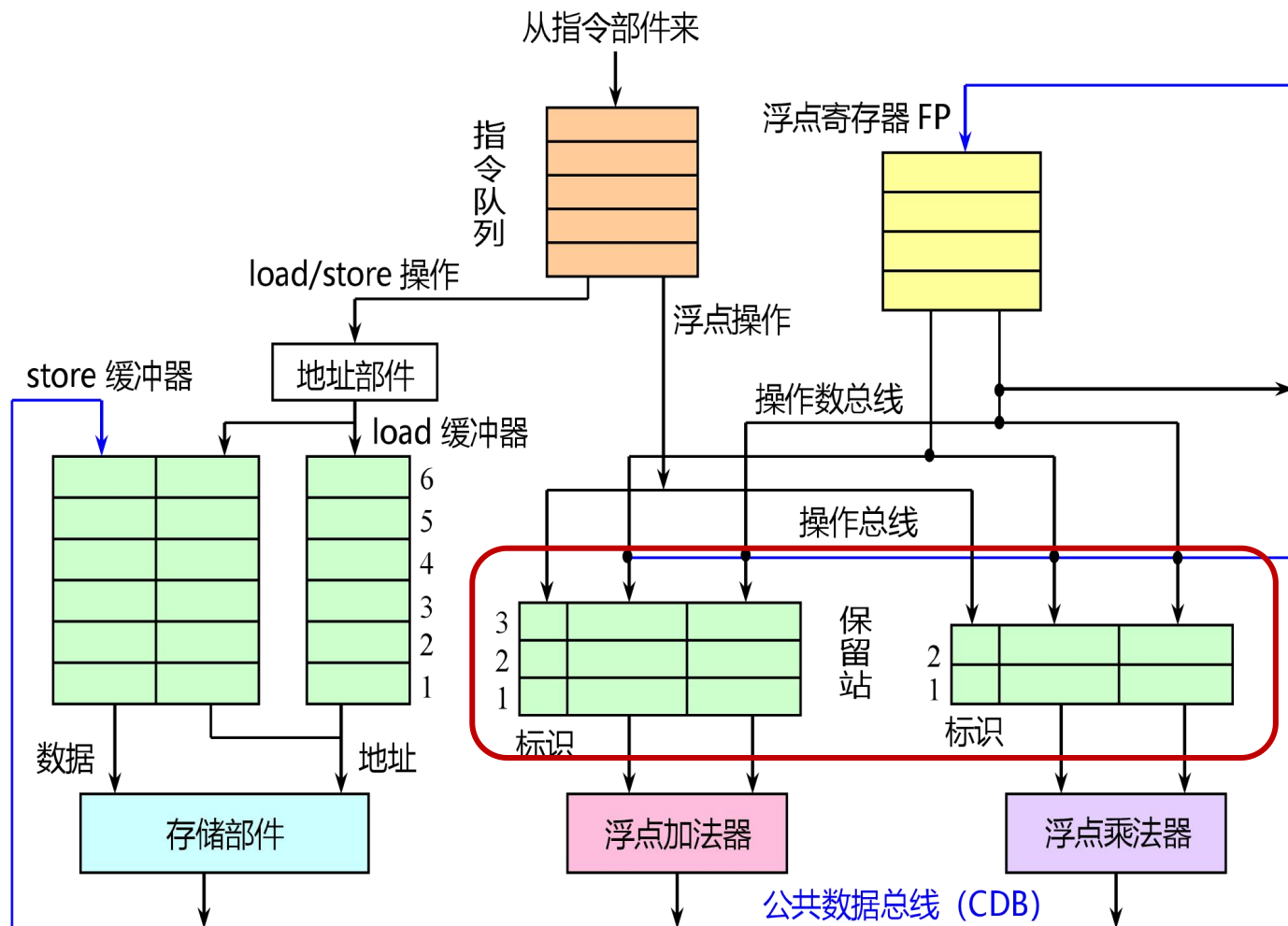
保留站 (Reservation Station)

- 每个保留站中保存一条已经流出并等待到本功能部件执行的指令（相关信息），包括：操作码、操作数以及用于检测 and 解决冲突的信息。
- 在一条指令流出到保留站的时候，如果该指令的源操作数已经在寄存器中就绪，则将之取到该保留站中。
- 如果操作数还没有计算出来，则在该保留站中记录将产生这个操作数的保留站的标识。

5.3.4 Tomasulo算法

第 122 页

4. 基于Tomasulo算法的MIPS处理器浮点部件的基本结构



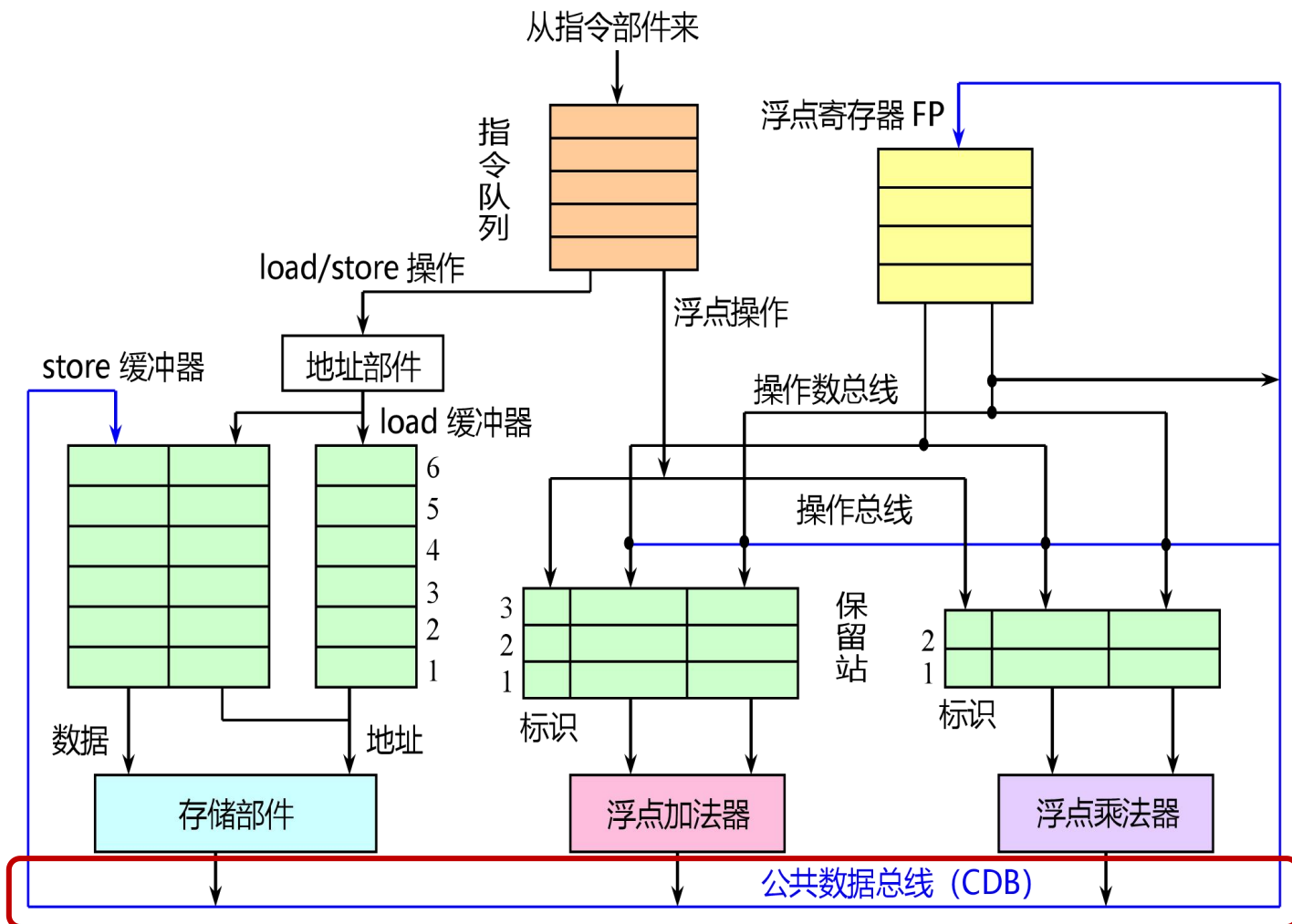
保留站 (Reservation Station)

- 浮点加法器有3个保留站: ADD1, ADD2, ADD3
- 浮点乘法器有两个保留站: MULT1, MULT2
- 每个保留站都有一个标识字段, 唯一地标识了该保留站。

5.3.4 Tomasulo算法

第 123 页

4. 基于Tomasulo算法的MIPS处理器浮点部件的基本结构



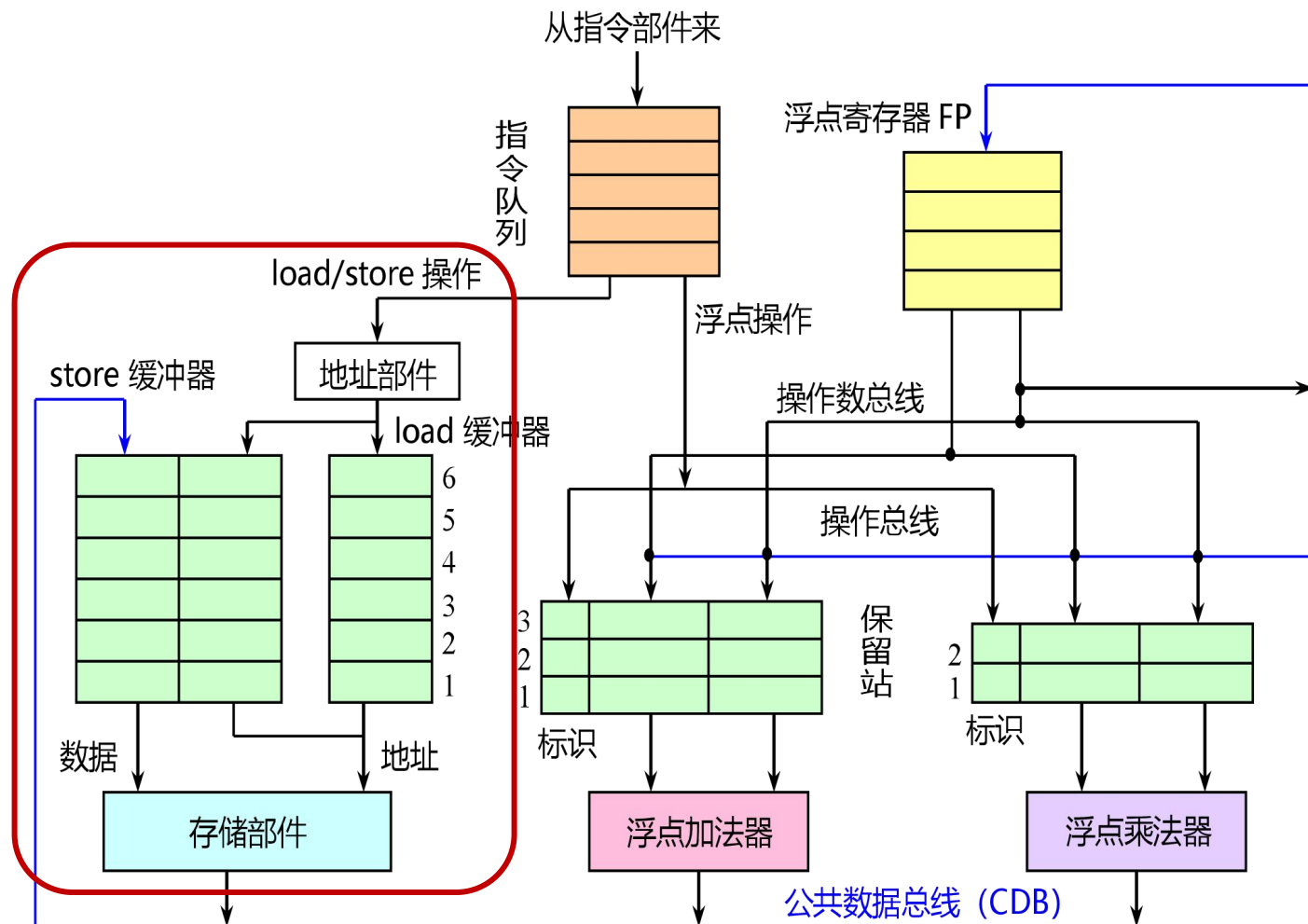
公共数据总线CDB （一条重要的数据通路）

- 所有功能部件的计算结果都是送到CDB上，由它把这些结果直接送到（播送到）各个需要该结果的地方。
- 在具有多个执行部件且采用多流出（即每个时钟周期流出多条指令）的流水线中，需要采用多条CDB。

5.3.4 Tomasulo算法

第 124 页

4. 基于Tomasulo算法的MIPS处理器浮点部件的基本结构



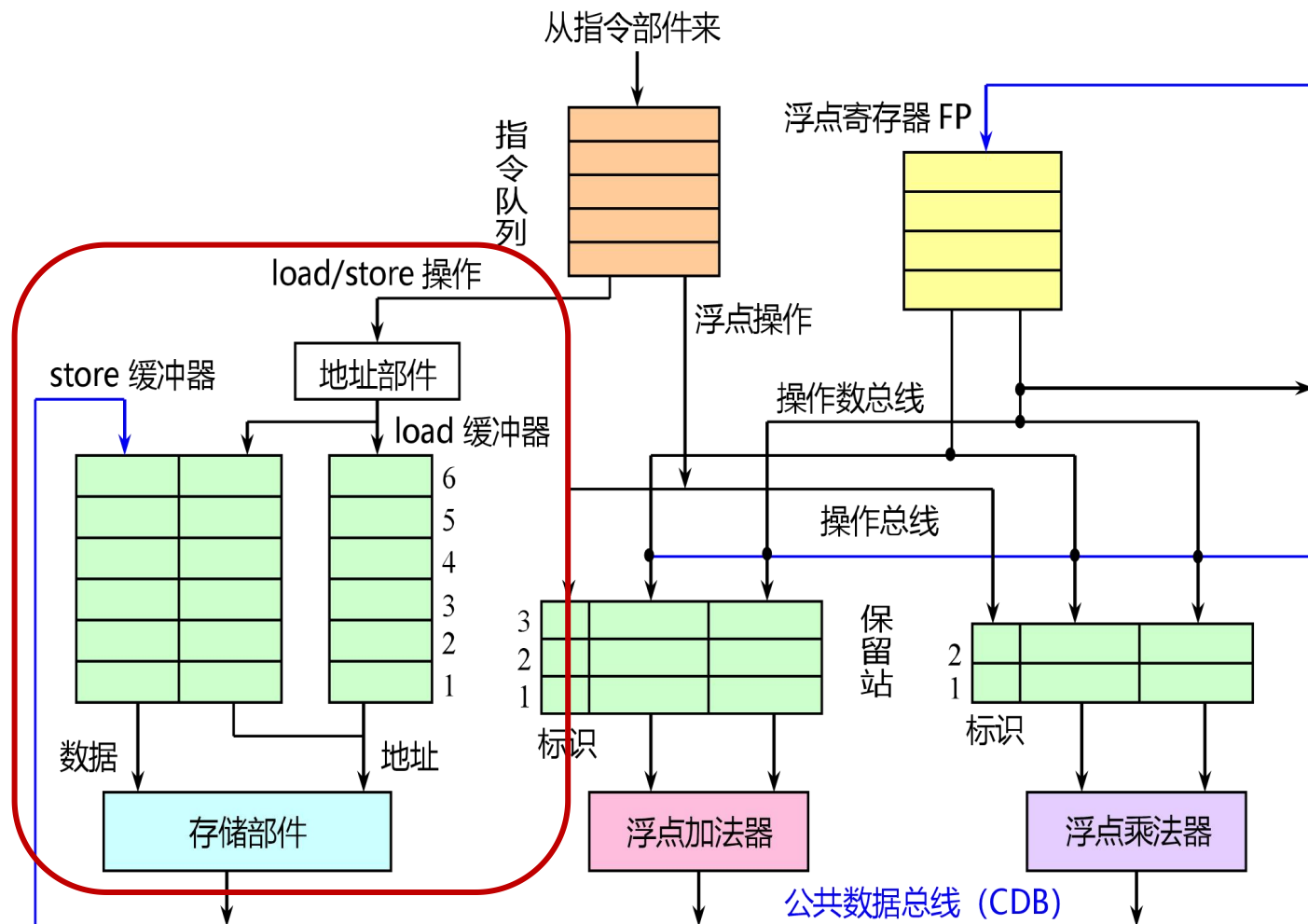
load缓冲器和store缓冲器

- 存放读/写存储器的数据或地址
- load缓冲器的作用有3个：
 - 存放用于计算有效地址的分量；
 - 记录正在进行的load访存，等待存储器的响应；
 - 保存已经完成了的load的结果（即从存储器取来的数据），等待公共数据总线CDB传输。

5.3.4 Tomasulo算法

第 125 页

4. 基于Tomasulo算法的MIPS处理器浮点部件的基本结构



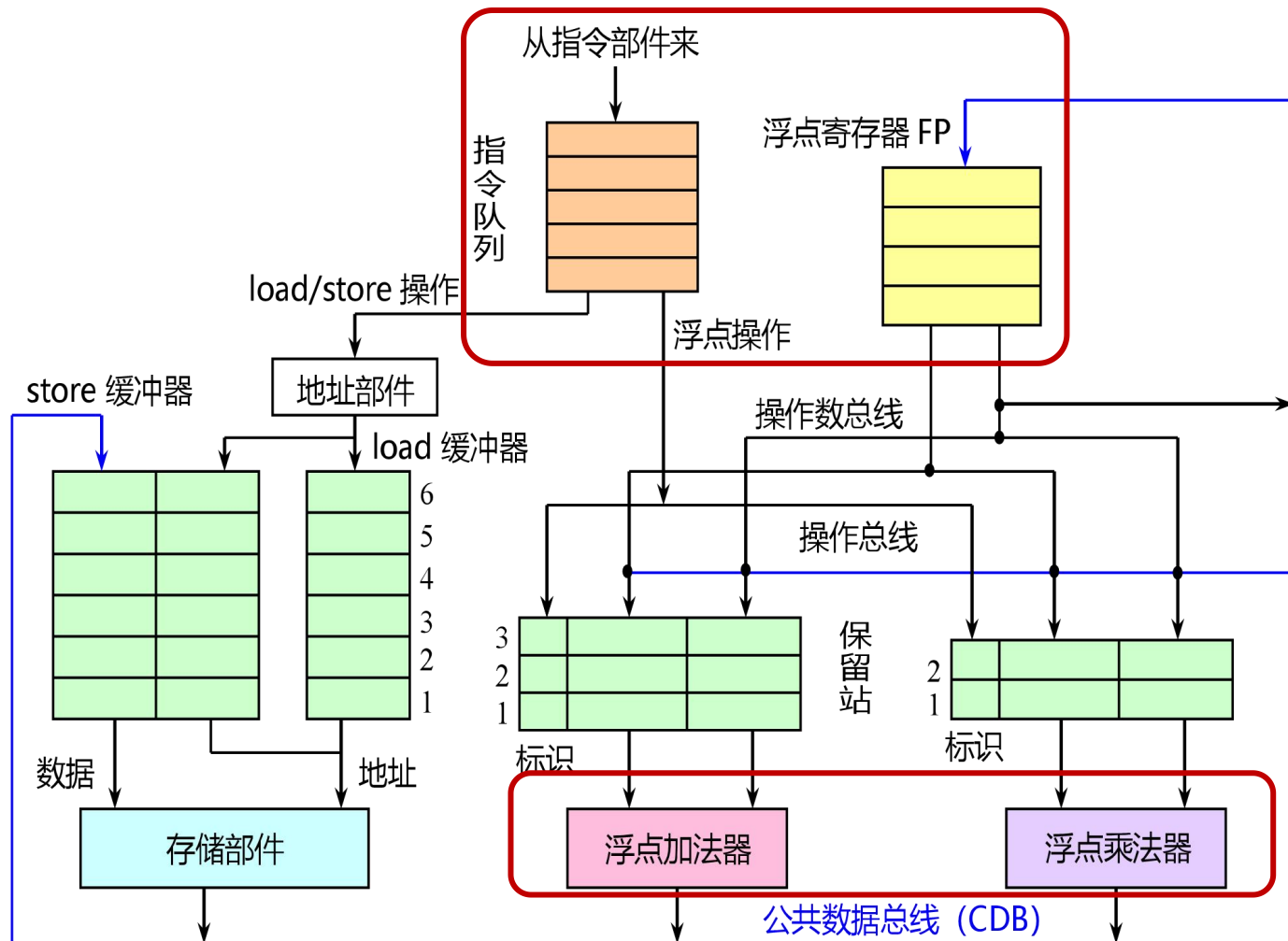
load缓冲器和store缓冲器

- 存放读/写存储器的数据或地址
- store缓冲器的作用有3个：
 - 存放用于计算有效地址的分量；
 - 保存正在进行的store访存的目标地址，该store正在等待存储数据的到达；
 - 保存该store的地址和数据，直到存储部件接收。

5.3.4 Tomasulo算法

第 126 页

4. 基于Tomasulo算法的MIPS处理器浮点部件的基本结构



浮点寄存器FP

- 共有16个浮点寄存器: F0, F2, F4, ..., F30。
- 它们通过一对总线连接到功能部件, 并通过CDB连接到store缓冲器。

指令队列

- 指令部件送来的指令放入指令队列
- 指令队列中的指令按先进先出的顺序流出

运算部件

- 浮点加法器完成加法和减法操作
- 浮点乘法器完成乘法和除法操作

5.3.4 Tomasulo算法

第 127 页

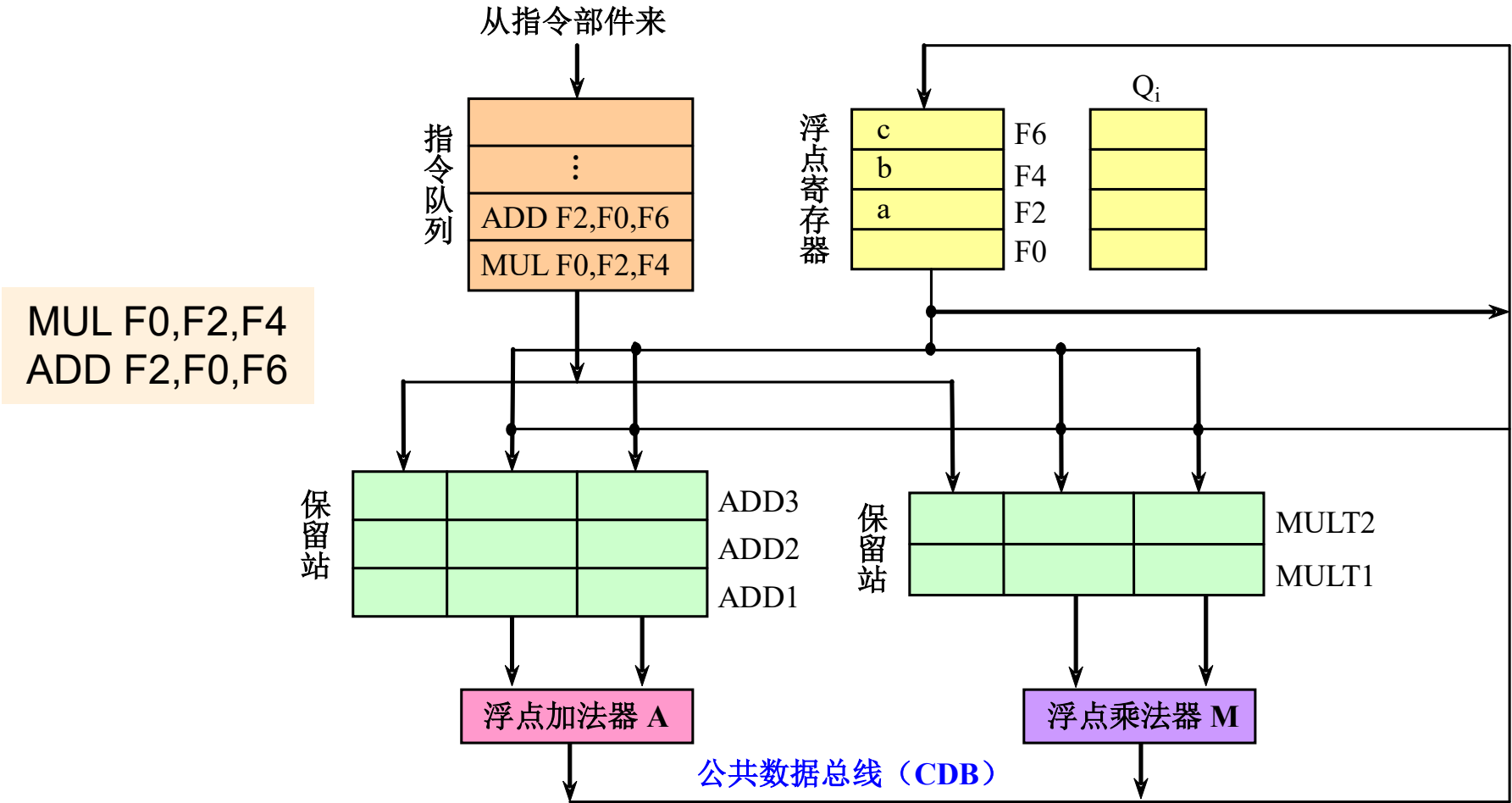
5. 在Tomasulo算法中，寄存器换名是通过保留站和流出逻辑来共同完成的。

- 当指令流出时，如果其操作数还没有计算出来，则将该指令中相应的寄存器号换名为将产生这个操作数的保留站的标识。
- 指令流出到保留站后，其操作数寄存器号或者换成了数据本身（如果该数据已经就绪），或者换成了保留站的标识，不再与寄存器有关系。

隐含寄存器重命名

5.3.4 Tomasulo算法

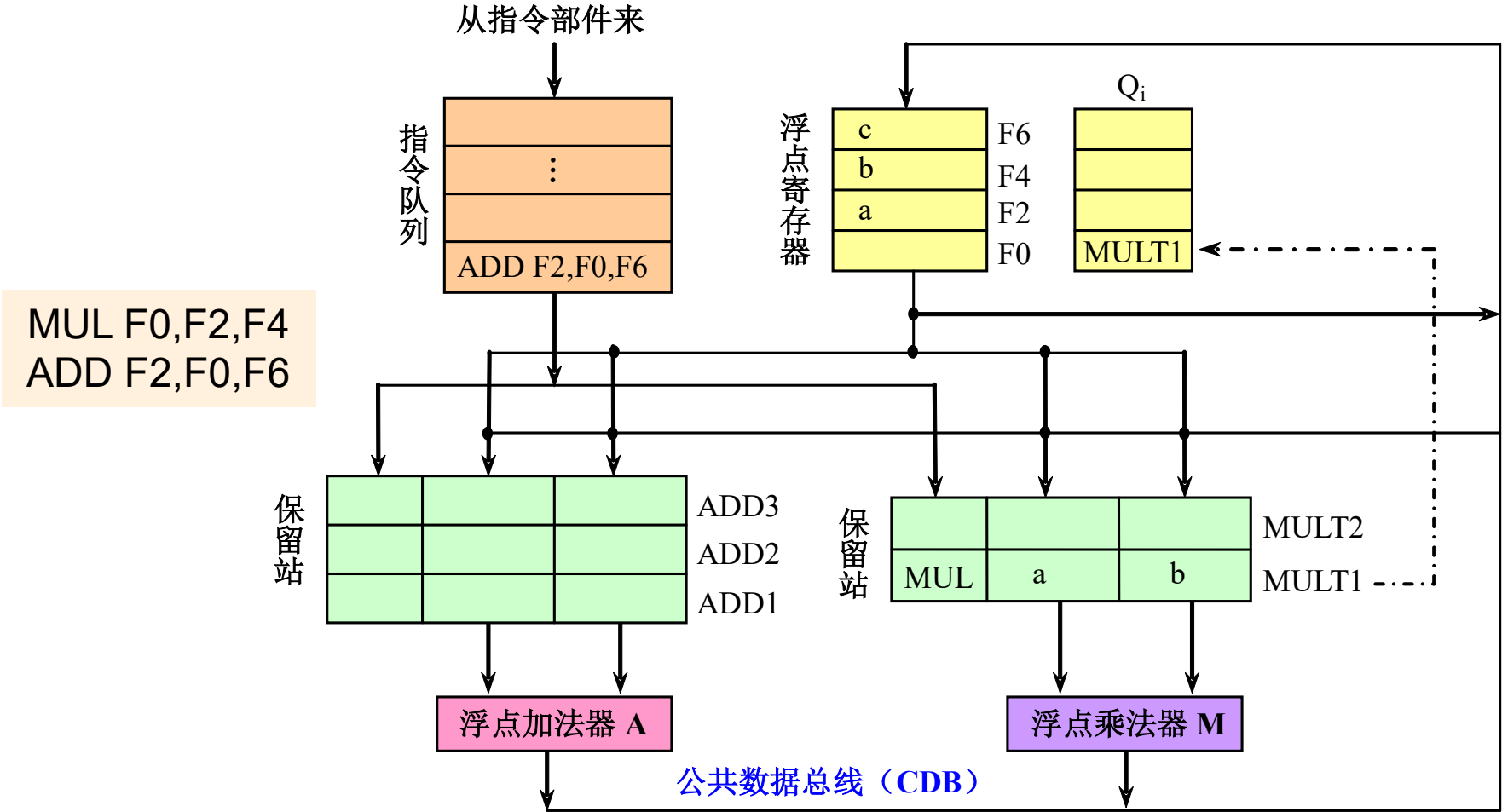
6. 通过一个简单的例子来说明Tomasulo算法的基本思想



将前述的结构图做个简化，并增加一个寄存器状态表Q_j，表示某指令的寄存器内容将由保留站提供。

5.3.4 Tomasulo算法

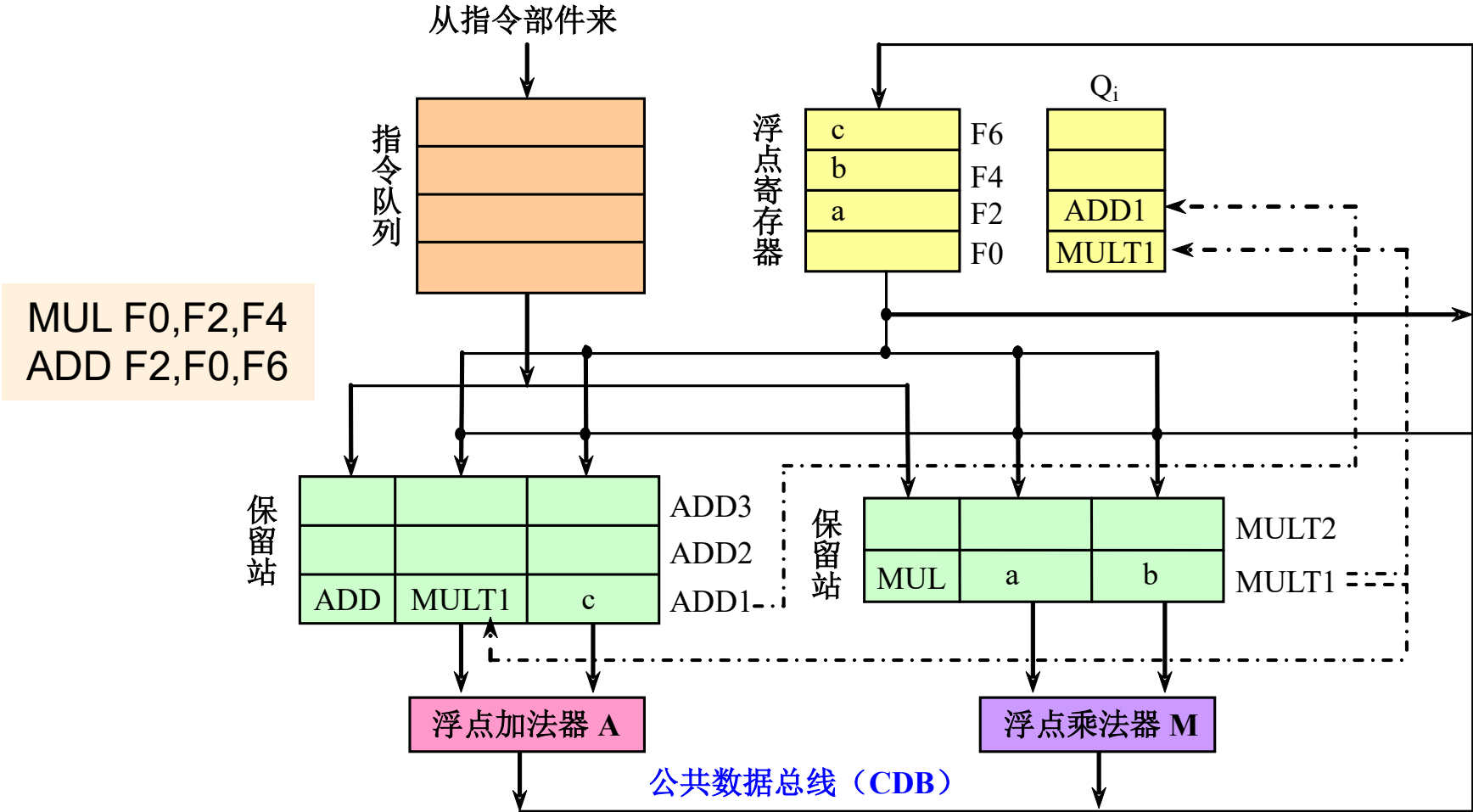
6. 通过一个简单的例子来说明Tomasulo算法的基本思想



当指令 MUL 流出到保留站 MULT1 时，由于其操作数 a 和 b 就绪（在 F2 和 F4 中），就将它们从寄存器取到保留站，因此该指令以后跟 F2 和 F4 没有关系了，执行时直接从保留站中取数据。同时将目的寄存器 F0 对应的 Q_i 标志为 MULT1，表示该寄存器的内容将由保留站 MULT1 提供（如图中的虚线所示）。

5.3.4 Tomasulo算法

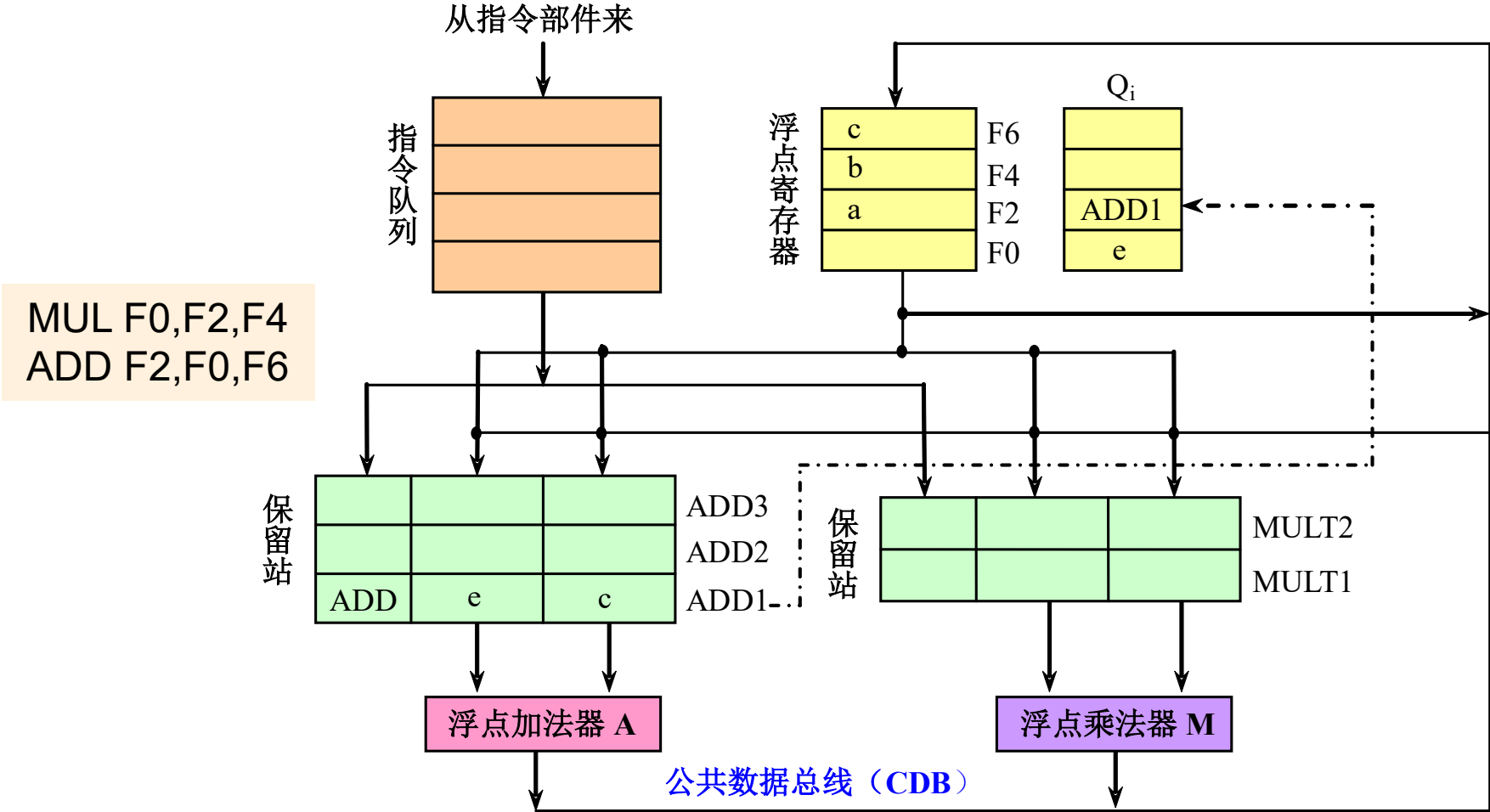
6. 通过一个简单的例子来说明Tomasulo算法的基本思想



当指令ADD流出到保留站ADD1时，也将操作数c（在F0中）取到保留站，但发现F0中的操作数还没有就绪，于是就把其提供者MULT1的标识取到保留站中。这样就有两个地方在等MULT1的结果。同时，它将目的寄存器F2对应的Q_i标志置为ADD1，表示该寄存器的内容将由保留站ADD1提供。

5.3.4 Tomasulo算法

6. 通过一个简单的例子来说明Tomasulo算法的基本思想



当MULT1的运算结果产生后（设为e），就把数据放到总线上（广播），所有等待该数据的地方都会自动把数据取走。ADD1中的ADD指令得到该数据后，马上就可以开始执行。

7. Tomasulo算法具有以下两个特点：

- 冲突检测和指令执行控制是分布的。

每个功能部件的保留站中的信息决定了什么时候指令可以在该功能部件开始执行。

- 计算结果通过CDB直接从产生它的保留站传送到所有需要它的功能部件，而不用经过寄存器。

8. 指令执行的步骤

使用Tomasulo算法的流水线需3段：

■ 流出：从指令队列的头部取一条指令。

➤ 如果该指令的操作所要求的保留站有空闲的，就把该指令送到该保留站（设为r）。

✓ 如果其操作数在寄存器中已经就绪，就将这些操作数送入保留站r。

✓ 如果其操作数还没有就绪，就把将产生该操作数的保留站的标识送入保留站r。

✓ 一旦被记录的保留站完成计算，它将直接把数据送给保留站r。

（寄存器换名和对操作数进行缓冲，消除WAR冲突）

➤ 完成对目标寄存器的预约工作

（消除了WAW冲突）

➤ 如果没有空闲的保留站，指令就不能流出。

（发生了结构冲突）

8. 指令执行的步骤

使用Tomasulo算法的流水线需3段：

■ 执行

- 当两个操作数都就绪后，本保留站就用相应的功能部件开始执行指令规定的操作。
- load和store指令的执行需要两个步骤：
 - ✓ 计算有效地址（要等到基地址寄存器就绪）
 - ✓ 把有效地址放入load或store缓冲器

■ 写结果

- 功能部件计算完毕后，就将计算结果放到CDB上，所有等待该计算结果的寄存器和保留站（包括store缓冲器）都同时从CDB上获得所需要的数据。

5.3.4 Tomasulo算法

第 135 页

9. 每个保留站有以下7个字段：

- Op：要对源操作数进行的操作
- Qj, Qk：将产生源操作数的保留站号，等于0表示操作数已经就绪且在Vj或Vk中，或者不需要操作数。
- Vj, Vk：源操作数的值
 - 对于每一个操作数来说，V或Q字段只有一个有效。
 - 对于load来说，Vk字段用于保存偏移量。
- Busy：为“yes”表示本保留站或缓冲单元“忙”
- A：仅load和store缓冲器有该字段。开始是存放指令中的立即数字段，地址计算后存放有效地址。
- Qi：寄存器状态表
 - 每个寄存器在该表中有对应的一项，用于存放将把结果写入该寄存器的保留站的站号。
 - 为0表示当前没有正在执行的指令要写入该寄存器，也即该寄存器中的内容就绪。

5.3.4 Tomasulo算法

第 136 页

例5.3 对于下述指令序列，给出当第一条指令完成并写入结果时，Tomasulo算法所用的各信息表中的内容。

L.D	F6,34 (R2)
L.D	F2,45 (R3)
MUL.D	F0,F2,F4
SUB.D	F8,F2,F6
DIV.D	F10,F0,F6
ADD.D	F6,F8,F2

5.3.4 Tomasulo算法

当采用Tomasulo算法时，在上述给定的时刻保留站、load缓冲器以及寄存器状态表中的内容。

指令		指令状态表		
		流出	执行	结果
L.D	F6 , 34(R2)	√	√	√
L.D	F2 , 45(R3)	√	√	
MUL.D	F0 , F2 , F4	√		
SUB.D	F8 , F6 , F2	√		
DIV.D	F10 , F0 , F6			
ADD.D	F6 , F8 , F2			

5.3.4 Tomasulo算法

名称	保留站						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	L.D					45+Regs[R3]
Add1	Yes	SUB.D	Reg[F6]	Mem[34+Regs[R2]]	Load2		
Add2	No						
Add3	No						
Mult1	Yes	MUL.D	Reg[F4]	Mem[34+Regs[R2]]	Load2		
Mult2	No						

	寄存器状态表							
	F0	F2	F4	F6	F8	F10		F30
Qi	Mult1	Load2			Add1			

5.3.4 Tomasulo算法

第 139 页

各符号的意义

- r : 分配给当前指令的保留站或者缓冲器单元编号;
- rd : 目标寄存器编号;
- rs 、 rt : 操作数寄存器编号;
- imm : 符号扩展后的立即数;
- RS : 保留站;
- $result$: 浮点部件或load缓冲器返回的结果;
- Qi : 寄存器状态表;
- $Regs[]$: 寄存器组;

- 与 rs 对应的保留站字段: Vj , Qj
- 与 rt 对应的保留站字段: Vk , Qk
- Qi 、 Qj 、 Qk 的内容或者为0, 或者是一个大于0的整数。
 - Qi 为0表示相应寄存器中的数据就绪。
 - Qj 、 Qk 为0表示保留站或缓冲器单元中的 Vj 或 Vk 字段中的数据就绪。
 - 当它们为正整数时, 表示相应的寄存器、保留站或缓冲器单元正在等待结果。

5.3.4 Tomasulo算法

第 140 页

符号说明: (举例)

MUL.D F0, F2, F4
 ↑ ↑ ↑
 rd rs rt
 i j k

L.D F2, 45 (R3)
 ↑ ↑ ↑
 rd imm rs
 k j

S.D F2, 45 (R3)
 ↑ ↑ ↑
 rd imm rs
 k j

5.3.4 Tomasulo算法

第 141 页

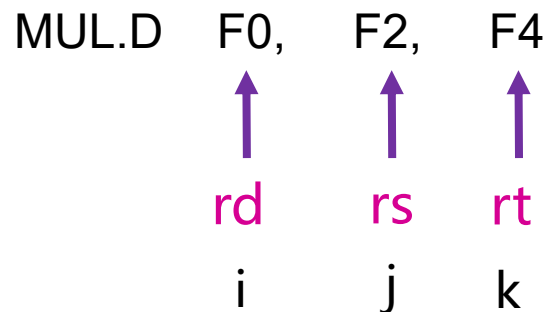
■指令发射

➤ 浮点运算指令

进入条件：有空闲保留站（设为r）

操作和状态表内容修改：

```
if (Qi[rs] ≠ 0)                                // 检测第一操作数是否就绪
    { RS[r].Qj ← Qi[rs] };                      //第一操作数没有就绪，进行寄存器换名，即把将产生该操作数
                                                的保留站的编号放入当前保留站的Qj。该编号是一个大于0的整数。
else { RS[r].Vj ← Regs[rs] ; //第一操作数就绪。把寄存器rs中的操作数取到当前保留站的Vj。
      RS[r].Qj ← 0 }                          //置Qj为0，表示当前保留站的Vj中的操作数就绪。
```



```
if ( $Q_i[rt] \neq 0$ ) //检测第二操作数是否就绪

    {  $RS[r].Q_k \leftarrow Q_i[rt]$  ; //第二操作数没有就绪，进行寄存器换名，即把将产生该操作数的保留站的编  
      号放入当前保留站的 $Q_k$ 。该编号是一个大于0的整数。

    else {  $RS[r].V_k \leftarrow Regs[rt]$  ; //第二操作数就绪。把寄存器 $rt$ 中的操作数取到当前保留站的 $V_k$ 。  
         $RS[r].Q_k \leftarrow 0$  } //置 $Q_k$ 为0，表示当前保留站的 $V_k$ 中的操作数就绪。

 $RS[r].Busy \leftarrow yes$ ; //置当前保留站为“忙”

 $RS[r].Op \leftarrow Op$ ; //设置操作码

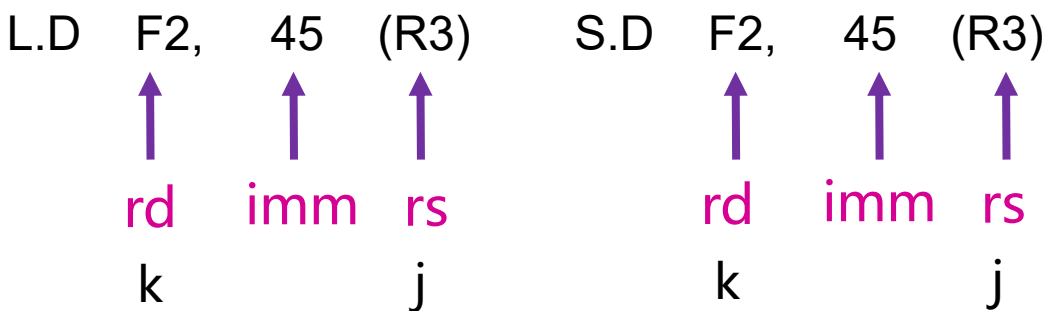
 $Q_i[rd] \leftarrow r$ ; //把当前保留站的编号 $r$ 放入 $rd$ 所对应的寄存器状态表项以便 $rd$ 将来接收结果。
```

5.3.4 Tomasulo算法

➤ load和store指令

进入条件：缓冲器有空闲单元（设为r）

操作和状态表内容修改：



```
if (Qi[rs] ≠ 0) // 检测第一操作数是否就绪
    {RS[r].Qj ← Qi[rs] } //第一操作数没有就绪，进行寄存器换名，即把将产生该操作数的保留站的编号存入当前缓冲器单元的Qj。
else
    {RS[r].Vj ← Regs[rs]; // 第一操作数就绪，把寄存器rs中的操作数取到当前缓冲器单元的Vj
    RS[r].Qj ← 0 }; // 置Qj为0，表示当前缓冲器单元的Vj中的操作数就绪。
```

5.3.4 Tomasulo算法

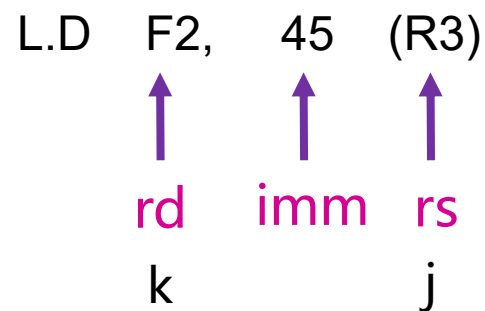
第 144 页

$RS[r].Busy \leftarrow yes;$ // 置当前缓冲器单元为“忙”

$RS[r].A \leftarrow Imm;$ // 把符号位扩展后的偏移量放入当前缓冲器单元的A

对于load指令:

$Qi[rd] \leftarrow r;$ // 把当前缓冲器单元的编号r放入load指令的目标寄存器rd所对应的寄存器状态表项, 以便rd将来接收所取的数据。



5.3.4 Tomasulo算法

第 145 页

对于store指令:

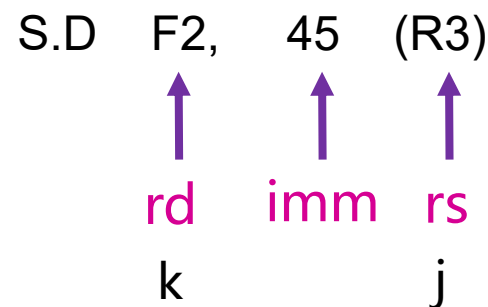
if ($Q_i[rd] \neq 0$) // 检测要存储的数据是否就绪

{ $RS[r].Q_k \leftarrow Q_i[rd]$ } //该数据尚未就绪, 进行寄存器换名, 即把将产生该数据的保留站的编号放入当前缓冲器单元的 Q_k 。

else

{ $RS[r].V_k \leftarrow Regs[rd]$; // 该数据就绪, 把它从寄存器 rd 取到store缓冲器单元的 V_k

$RS[r].Q_k \leftarrow 0$ }; // 置 Q_k 为0, 表示当前缓冲器单元的 V_k 中的数据就绪。



2. 执行

■ 浮点操作指令

- 进入条件: $(RS[r].Qj = 0)$ 且 $(RS[r].Qk = 0)$; // 两个源操作数就绪
- 操作和状态表内容修改: 进行计算, 产生结果。

■ load/store指令

- 进入条件: $(RS[r].Qj = 0)$ 且 r 成为load/store缓冲队列的头部
- 操作和状态表内容修改:

$RS[r].A \leftarrow RS[r].Vj + RS[r].A;$ //计算有效地址

对于load指令, 在完成有效地址计算后, 还要进行:

从 $Mem[RS[r].A]$ 读取数据; //从存储器中读取数据

3. 写结果

■ 浮点运算指令和load指令

进入条件：保留站r执行结束，且CDB就绪。

操作和状态表内容修改：

```

 $\forall x$  (if ( $Q_i[x] = r$ )           // 对于任何一个正在等该结果的浮点寄存器x
    {  $Regs[x] \leftarrow result$ ;   // 向该寄存器写入结果
       $Q_i[x] \leftarrow 0$  };       // 把该寄存器的状态置为数据就绪
 $\forall x$  (if ( $RS[x].Q_j = r$ )         // 对于任何一个正在等该结果作为第一操作数的保留站x
    {  $RS[x].V_j \leftarrow result$ ; // 向该保留站的Vj写入结果
       $RS[x].Q_j \leftarrow 0$  };   // 置Qj为0，表示该保留站的Vj中的操作数就绪
 $\forall x$  (if ( $RS[x].Q_k = r$ )         // 对于任何一个正在等该结果作为第二操作数的保留站x
    {  $RS[x].V_k \leftarrow result$ ; // 向该保留站的Vk写入结果
       $RS[x].Q_k \leftarrow 0$  };   // 置Qk为0，表示该保留站的Vk中的操作数就绪。
 $RS[r].Busy \leftarrow no$ ;        // 释放当前保留站，将之置为空闲状态。
```

5.3.4 Tomasulo算法

第 148 页

■ store指令

进入条件：保留站 r 执行结束，且 $RS[r].Qk = 0$ 要存储的数据已经就绪

操作和状态表内容修改：

$Mem[RS[r].A] \leftarrow RS[r].Vk$ // 数据写入存储器，地址由store缓冲器单元的A字段给出。

$RS[r].Busy \leftarrow no;$ // 释放当前缓冲器单元，将之置为空闲状态。

5.3.4 Tomasulo算法

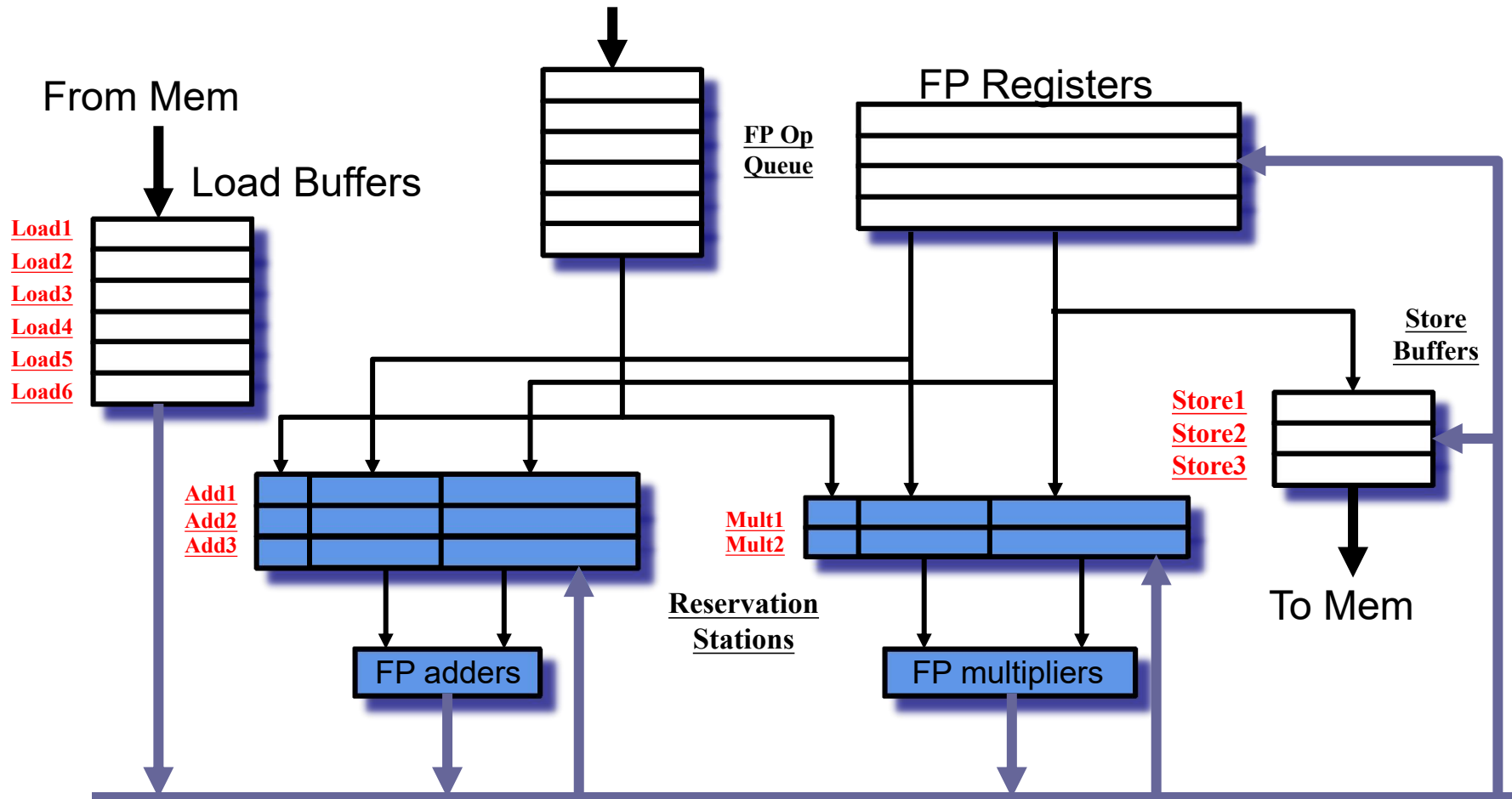
第 149 页

Tomasulo 算法的三个阶段：

- 1. Issue—发射：从浮点操作队列取指令
 - 保留站空闲 (no structural hazard),
控制发射指令 & 发送操作数 (renames registers)--WAR
- 2. Execute—执行(EX)
 - 操作数就绪，开始执行；
若没有就绪，监测 CDB，等待结果
- 3. Write result—执行完成(WB)
 - 写结果到 CDB; 置保留站空闲
- 正常总线: data + destination (“go to” 总线)
- Common data bus (CDB) : data + source (“come from” 总线)
 - 64 位数据 + 4 位功能部件地址
 - 广播，举例: 2 cc for 浮点 +,-; 10 for 浮点 *; 40 cc for 浮点 /

5.3.4 Tomasulo算法

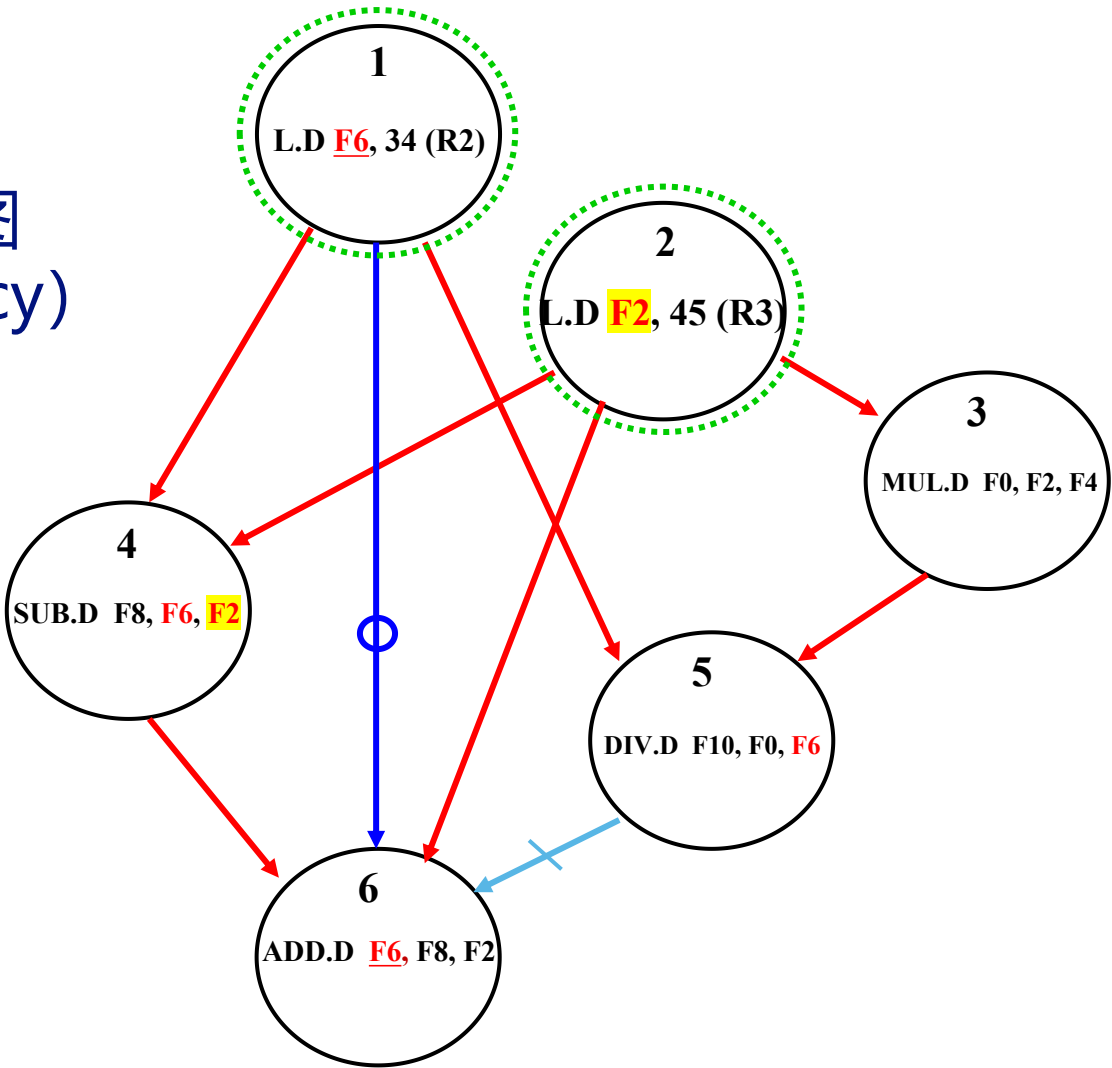
第 150 页



L.D F6, 34(R2)
L.D F2, 45(R3)
MUL.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

5.3.4 Tomasulo算法

指令相关图
(Dependency)



- 1 L.D F6, 34(R2)
- 2 L.D F2, 45(R3)
- 3 MUL.D F0, F2, F4
- 4 SUB.D F8, F6, F2
- 5 DIV.D F10, F0, F6
- 6 ADD.D F6, F8, F2

数据相关:

- (1, 4) (1, 5) (2, 3) (2, 4)
- (2, 6) (3, 5) (4, 6)

输出相关:

- (1, 6)

反相关:

- (5, 6)

真数据相关 (RAW)	→
反相关 (WAR)	+→
输出相关 (WAW)	⊖→

第 152 页

Instruction status:

Instruction		j	k
LD	F6	34+	R2
LD	F2	45+	R3
MULTD	F0	F2	F4
SUBD	F8	F6	F2
DIVD	F10	F0	F6
ADDD	F6	F8	F2

Exec Write

<i>Issue</i>	<i>Comp</i>	<i>Result</i>

Busy Address

Load1	No
Load2	No
Load3	No

3个输入缓冲器

Reservation Stations:

on Stations:				$S1$	$S2$	RS	RS
Time	Name	Busy	Op	V_j	V_k	$\underline{Q_j}$	$\underline{Q_k}$
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
	Mult2	No					

5个功能单元

3个浮点加法保留站
2个浮点乘法保留站

Register result status:

时钟周期计数

Clock

	$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	$\dots$	$F30$
FU									

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 1

Instruction status:

				Exec		Write
Instruction	<i>j</i>	<i>k</i>	Issue	Comp	Result	
LD	F6	34+	R2	1		
LD	F2	45+	R3			
MULTD	F0	F2	F4			
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

	Busy	Address
Load1	Yes	34+R2
Load2	No	
Load3	No	

Reservation Stations:

				<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
	Mult2	No					

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
1				Load1					

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 2

Instruction status:

Instruction	j	k	Exec Write			Busy	Address
			Issue	Comp	Result		
LD	F6	34+	R2	1		Load1	Yes 34+R2
LD	F2	45+	R3	2		Load2	Yes 45+R3
MULTD	F0	F2	F4			Load3	No
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

Reservation Stations:

Time	Name	Busy	Op	S1		S2		RS	
				Vj	Vk	Vj	Vk	Qj	Qk
	Add1	No							
	Add2	No							
	Add3	No							
	Mult1	No							
	Mult2	No							

Register result status:

Clock	F0 F2 F4 F6 F8 F10 F12 ... F30											
	FU											
2		Load2										
				Load1								

假设：
浮点乘法需要10个时钟周期
浮点除法需要40个时钟周期
其他操作需要2个时钟周期
(LD、ADD、SUB)

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 3

Instruction status:

Instruction	<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result
LD	F6	34+	R2	1	3
LD	F2	45+	R3	2	
MULTD	F0	F2	F4	3	
SUBD	F8	F6	F2		
DIVD	F10	F0	F6		
ADDD	F6	F8	F2		

Busy

Address

Load1

Load2

Load3

Yes

Yes

No

34+R2

45+R3

Reservation Stations:

Time	Name	Busy	Op	S1 <i>Vj</i>	S2 <i>Vk</i>	RS <i>Qj</i>	RS <i>Qk</i>
Add1		No					
Add2		No					
Add3		No					
Mult1		Yes	MULTD		R(F4)	Load2	
Mult2		No					

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
3	Mult1	Load2		Load1					

注意：保留站已经把寄存器名去掉

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 4

Instruction status:

				Exec Write				
Instruction	j	k		Issue	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1	No
LD	F2	45+	R3	2	4		Load2	Yes 45+R3
MULTD	F0	F2	F4	3			Load3	No
SUBD	F8	F6	F2	4				
DIVD	F10	F0	F6					
ADDD	F6	F8	F2					

Reservation Stations:

				S1	S2	RS	RS
Time	Name	Busy	Op	Vj	Vk	Qj	Qk
Add1	Yes	SUBD	M(A1)				Load2
Add2	No						
Add3	No						
Mult1	Yes	MULTD			R(F4)	Load2	
Mult2	No						

Load2 完成

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
4	Mult1	Load2		M(A1)	Add1				

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 5

Instruction status:

Instruction	<i>j</i>	<i>k</i>	Issue	Exec	Write	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	4		Load1	No
LD	F2	45+	R3	2	4	5		Load2	No
MULTD	F0	F2	F4	3				Load3	No
SUBD	F8	F6	F2	4					
DIVD	F10	F0	F6	5					
ADDD	F6	F8	F2						

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> <i>Vj</i>	<i>S2</i> <i>Vk</i>	<i>RS</i> <i>Qj</i>	<i>RS</i> <i>Qk</i>
2	Add1	Yes	SUBD	M(A1)	M(A2)		
	Add2	No					
	Add3	No					
10	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
5	Mult1	M(A2)		M(A1)	Add1	Mult2			

注意：增加了ADD和MULT的时钟周期计数

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 6

Instruction status:

Instruction		<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	4	5
MULTD	F0	F2	F4	3		
SUBD	F8	F6	F2	4		
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6		

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> <i>Vj</i>	<i>S2</i> <i>Vk</i>	<i>RS</i> <i>Qj</i>	<i>RS</i> <i>Qk</i>
1	Add1	Yes	SUBD	M(A1)	M(A2)		
	Add2	Yes	ADDD		M(A2)	Add1	
	Add3	No					
9	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
6	FU	Mult1	M(A2)		Add2	Add1	Mult2			

注意：尽管在F6上有名相关，依然发射ADDD指令

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 7

Instruction status:

Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Exec</i>		<i>Write</i>	<i>Busy</i>	<i>Address</i>
				<i>Comp</i>	<i>Result</i>			
LD	F6	34+	R2	1	3	4	Load1	No
LD	F2	45+	R3	2	4	5	Load2	No
MULTD	F0	F2	F4	3			Load3	No
SUBD	F8	F6	F2	4	7			
DIVD	F10	F0	F6	5				
ADDD	F6	F8	F2	6				

Reservation Stations:

<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
				<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
0	Add1	Yes	SUBD	M(A1)	M(A2)		
	Add2	Yes	ADDD		M(A2)	Add1	
	Add3	No					
8	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Add1 (SUBD) 完成

Register result status:

Clock										
	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>	
7	Mult1	M(A2)		Add2	Add1	Mult2				

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 8

Instruction status:

Instruction	<i>j</i>	<i>k</i>	Issue	Exec	Write
LD	F6	34+	R2	1	3
LD	F2	45+	R3	2	4
MULTD	F0	F2	F4	3	5
SUBD	F8	F6	F2	4	7
DIVD	F10	F0	F6	5	8
ADDD	F6	F8	F2	6	

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
				<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
2	Add2	Yes	ADDD	(M-M)	M(A2)		
	Add3	No					
7	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
8	Mult1	M(A2)		Add2	(M-M)	Mult2			

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 9

Instruction status:

				Exec		Write		
Instruction	<i>j</i>	<i>k</i>	Issue	Comp	Result		Busy	Address
LD	F6	34+	R2	1	3	4	Load1	No
LD	F2	45+	R3	2	4	5	Load2	No
MULTD	F0	F2	F4	3			Load3	No
SUBD	F8	F6	F2	4	7	8		
DIVD	F10	F0	F6	5				
ADDD	F6	F8	F2	6				

Reservation Stations:

				<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
1	Add2	Yes	ADDD	(M-M)	M(A2)		
	Add3	No					
6	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
9	FU	Mult1	M(A2)		Add2	(M-M)	Mult2		

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 10

Instruction status:

Instruction	<i>j</i>	<i>k</i>	<i>Exec Write</i>			Busy	Address
			<i>Issue</i>	<i>Comp</i>	<i>Result</i>		
LD	F6	34+	R2	1	3	4	Load1
LD	F2	45+	R3	2	4	5	Load2
MULTD	F0	F2	F4	3			Load3
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5			
ADDD	F6	F8	F2	6	10		

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
				<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
0	Add2	Yes	ADDD	(M-M)	M(A2)		
	Add3	No					
5	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Add2 (ADDD) 完成

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
10									
FU	Mult1	M(A2)		Add2	(M-M)	Mult2			

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 11

Instruction status:

				Exec		Write
Instruction		<i>j</i>	<i>k</i>	Issue	Comp	Result
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	4	5
MULTD	F0	F2	F4	3		
SUBD	F8	F6	F2	4	7	8
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	10	11

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

ADDD 写结果?

Reservation Stations:

				<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
4	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
11	FU	Mult1	M(A2)		(M-M+M	(M-M)	Mult2			

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 12

Instruction status:

Instruction	<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result	
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	4	5
MULTD	F0	F2	F4	3		
SUBD	F8	F6	F2	4	7	8
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	10	11

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations:

Time	Name	Busy	Op	S1 <i>Vj</i>	S2 <i>Vk</i>	RS <i>Qj</i>	RS <i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
3	Mult1	Yes	MULTD	M(A2)		R(F4)	
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
12	Mult1	M(A2)		(M-M+M	(M-M)	Mult2			

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 13

Instruction status:

Instruction	j	k	Exec Write			Busy	Address
			Issue	Comp	Result		
LD	F6	34+	R2	1	3	4	Load1 Load2 Load3
LD	F2	45+	R3	2	4	5	
MULTD	F0	F2	F4	3			
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5			
ADDD	F6	F8	F2	6	10	11	

Reservation Stations:

Time	Name	Busy	Op	S1		S2		RS	
				Vj	Vk	Vk	Vk	Qj	Qk
	Add1	No							
	Add2	No							
	Add3	No							
2	Mult1	Yes	MULTD	M(A2)	R(F4)				
	Mult2	Yes	DIVD		M(A1)	Mult1			

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
13	FU	Mult1	M(A2)		(M-M+M	(M-M)	Mult2		

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 14

Instruction status:

Instruction	<i>j</i>	<i>k</i>	<i>Exec Write</i>				Busy	Address
			<i>Issue</i>	<i>Comp</i>	<i>Result</i>			
LD	F6	34+	R2	1	3	4	Load1	No
LD	F2	45+	R3	2	4	5	Load2	No
MULTD	F0	F2	F4	3			Load3	No
SUBD	F8	F6	F2	4	7	8		
DIVD	F10	F0	F6	5				
ADDD	F6	F8	F2	6	10	11		

Reservation Stations:

<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
				<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
1	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
14	FU	Mult1	M(A2)		(M-M+V	(M-M)	Mult2		

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 15

Mult1 (MULTD) 完成

Instruction status:

Instruction	<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result	
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	4	5
MULTD	F0	F2	F4	3	15	
SUBD	F8	F6	F2	4	7	8
DIVD	F10	F0	F6	5		
ADDD	F6	F8	F2	6	10	11

Busy

Address

Load1	No
Load2	No
Load3	No

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> <i>Vj</i>	<i>S2</i> <i>Vk</i>	<i>RS</i> <i>Qj</i>	<i>RS</i> <i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
0	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)		Mult1

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
15	Mult1	M(A2)		(M-M+M	(M-M)	Mult2			

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 16

Instruction status:

Instruction		<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result		Busy	Address
LD	F6	34+	R2	1	3	4	Load1	No	
LD	F2	45+	R3	2	4	5	Load2	No	
MULTD	F0	F2	F4	3	15	16	Load3	No	
SUBD	F8	F6	F2	4	7	8			
DIVD	F10	F0	F6	5					
ADDD	F6	F8	F2	6	10	11			

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> <i>Vj</i>	<i>S2</i> <i>Vk</i>	<i>RS</i> <i>Qj</i>	<i>RS</i> <i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
40	Mult2	Yes	DIVD	M*F4	M(A1)		

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
16	FU	M*F4	M(A2)		(M-M+M	(M-M)	Mult2		

只等待Mult2 (DIVD)指令完成

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 55

Instruction status:

				Exec		Write		
Instruction		j	k	Issue	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1	No
LD	F2	45+	R3	2	4	5	Load2	No
MULTD	F0	F2	F4	3	15	16	Load3	No
SUBD	F8	F6	F2	4	7	8		
DIVD	F10	F0	F6	5				
ADDD	F6	F8	F2	6	10	11		

Reservation Stations:

				S1	S2	RS	RS
Time	Name	Busy	Op	Vj	Vk	Qj	Qk
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
1	Mult2	Yes	DIVD	M*F4	M(A1)		

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
55	M*F4	M(A2)		(M-M+M	(M-M)	Mult2			

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 56

Instruction status:

Instruction		<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Exec Comp</i>	<i>Write Result</i>		Busy	Address
LD	F6	34+	R2	1	3	4	Load1	No	
LD	F2	45+	R3	2	4	5	Load2	No	
MULTD	F0	F2	F4	3	15	16	Load3	No	
SUBD	F8	F6	F2	4	7	8			
DIVD	F10	F0	F6	5	56				
ADDD	F6	F8	F2	6	10	11			

Reservation Stations:

				<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
0	Mult2	Yes	DIVD	M*F4	M(A1)		

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
56	M*F4	M(A2)		(M-M+N	(M-M)	Mult2			

FU

5.3.4 Tomasulo算法

Tomasulo 示例
Cycle 57

Instruction status:

Instruction	<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result	
LD	F6	34+	R2	1	3	4
LD	F2	45+	R3	2	4	5
MULTD	F0	F2	F4	3	15	16
SUBD	F8	F6	F2	4	7	8
DIVD	F10	F0	F6	5	56	57
ADDD	F6	F8	F2	6	10	11

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> <i>Vj</i>	<i>S2</i> <i>Vk</i>	<i>RS</i> <i>Qj</i>	<i>RS</i> <i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
	Mult2	Yes	DIVD	M*F4	M(A1)		

按序发射,
乱序执行,
乱序完成

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
56	M*F4	M(A2)		(M-M+M	(M-M)	Result			

5.3.4 Tomasulo算法

LD	F6	34+	R2
LD	F2	45+	R3
MULTD	F0	F2	F4
SUBD	F8	F6	F2
DIVD	F10	F0	F6
ADDD	F6	F8	F2

2 cc for 浮点 +,-;
10 cc for 浮点 *;
40 cc for 浮点 /

	1	2	3	4	5	6	7	8	9	10	11	12	13
LD F6, 34(R2)	Iss	Ex	M	Wb									
LD F2, 45(R3)		Iss	Ex	M	Wb								
MULTD F0, F2, F4			Iss	Iss	Iss	M1	M2	M3	M4	M5	M6	M7	M8
SUBD F8, F6, F2				Iss	Iss	A1	A2	Wb					
DIVD F10, F0, F6					Iss	Iss	Iss	Iss	Iss	Iss	Iss	Iss	Iss
ADDD F6, F8, F2						Iss	Iss	Iss	A1	A2	Wb		

第 173 页

2 cc for 浮点 +, -;
10 cc for 浮点 *;
40 cc for 浮点 /

[illegible]

为什么Tomasulo可以实现循环间并行？

■寄存器重命名

- 动态循环展开 loop unrolling

■保留站

- 循序浮点指令发射先于整数控制流
- 避免WAR引起的停顿（如记分板）

■Tomasulo 算法在运行中维护数据相关性

5.3.4 Tomasulo算法

第 175 页

Tomasulo Loop示例:

Loop: LD	F0	0(R1)	
MULTD	F4	F0	F2
SD	F4	0(R1)	
SUBI	R1	R1	#8
BNEZ	R1	Loop	

- 假设乘法需要4个时钟周期
- 第一次数据加载需要8个时钟周期 (L1 cache miss) , 第二次数据加载需要一个时钟周期 (hit)

第 176 页

Exec Write

循环计数

$$S1 \quad S2 \quad RS$$

增加Store缓冲器

指令循环

R1 的值

5.3.4 Tomasulo算法

Loop 示例
Cycle 1

Instruction status:

ITER	Instruction	j	k	Issue	Comp	Result
1	LD	F0	0	R1	1	

Exec Write

	Busy	Addr	Fu
Load1	Yes	80	
Load2	No		
Load3	No		
Store1	No		
Store2	No		
Store3	No		

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:
	Add1	No						LD
	Add2	No						F0
	Add3	No						0
	Mult1	No						R1
	Mult2	No						

MULTD	F4	F0	F2
SD	F4	0	R1
SUBI	R1	R1	#8
BNEZ	R1	Loop	



Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
1	80	Fu	Load1							

5.3.4 Tomasulo算法

Loop 示例
Cycle 2

Instruction status:

ITER	Instruction	j	k	Issue	Comp	Result
1	LD	F0	0	R1	1	
1	MULTD	F4	F0	F2	2	

Exec Write

	Busy	Addr	Fu
Load1	Yes	80	
Load2	No		
Load3	No		
Store1	No		
Store2	No		
Store3	No		

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:
	Add1	No						LD F0 0 R1
	Add2	No						MULTD F4 F0 F2
	Add3	No						SD F4 0 R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI R1 R1 #8
	Mult2	No						BNEZ R1 Loop

←

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
2	80	Load1		Mult1						

5.3.4 Tomasulo算法

Instruction status:

					Exec Write			
ITER	Instruction		j	k	Issue	Comp	Result	
1	LD	F0	0	R1	1			Load1
1	MULTD	F4	F0	F2	2			Load2
1	SD	F4	0	R1	3			Load3
								Store1
								Store2
								Store3

Busy	Addr	Fu
Yes	80	
No		
No		
Yes	80	Mult1
No		
No		

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:
	Add1	No						LD
	Add2	No						MULTD
	Add3	No						SD
	Mult1	Yes	Multd					SUBI
	Mult2	No						BNEZ

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
3	80	Fu	Load1	Mult1						

Implicit renaming sets up data flow graph

5.3.4 Tomasulo算法

Instruction status:

					Exec Write		
ITER	Instruction		j	k	Issue CompResult	Busy	Addr Fu
1	LD	F0	0	R1	1	Load1	Yes 80
1	MULTD	F4	F0	F2	2	Load2	No
1	SD	F4	0	R1	3	Load3	No
						Store1	Yes 80 Mult1
						Store2	No
						Store3	No

Reservation Stations:

				S1	S2	RS					
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI	R1	R1	#8
	Mult2	No						BNEZ	R1	Loop	

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
4	80	Fu	Load1		Mult1						

5.3.4 Tomasulo算法

Instruction status:

					Exec Write		
ITER	Instruction		j	k	Issue	CompResult	Busy Addr Fu
1	LD	F0	0	R1	1		Load1 Yes 80
1	MULTD	F4	F0	F2	2		Load2 No
1	SD	F4	0	R1	3		Load3 No
							Store1 Yes 80 Mult1
							Store2 No
							Store3 No

Reservation Stations:

					S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI	R1	R1	#8
	Mult2	No						BNEZ	R1	Loop	

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
5	72	Load1		Mult1						

5.3.4 Tomasulo算法

Instruction status:

					Exec Write				
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1		Load1	Yes 80	
1	MULTD	F4	F0	F2	2		Load2	Yes 72	
1	SD	F4	0	R1	3		Load3	No	
2	LD	F0	0	R1	6		Store1	Yes 80	Mult1
							Store2	No	
							Store3	No	

Reservation Stations:

					S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI	R1	R1	#8
	Mult2	No						BNEZ	R1	Loop	

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
6	72	Fu	Load2		Mult1					

5.3.4 Tomasulo算法

Instruction status:

					Exec Write				
ITER	Instruction		j	k	Issue	CompResult	Busy	Addr	Fu
1	LD	F0	0	R1	1		Load1	Yes 80	
1	MULTD	F4	F0	F2	2		Load2	Yes 72	
1	SD	F4	0	R1	3		Load3	No	
2	LD	F0	0	R1	6		Store1	Yes 80	Mult1
2	MULTD	F4	F0	F2	7		Store2	No	
							Store3	No	

Reservation Stations:

					S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI	R1	R1	#8
	Mult2	Yes	Multd		R(F2)	Load2		BNEZ	R1	Loop	

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
7	72	Fu	Load2		Mult2						

5.3.4 Tomasulo算法

Instruction status:

					Exec Write				
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1		Load1	Yes	80
1	MULTD	F4	F0	F2	2		Load2	Yes	72
1	SD	F4	0	R1	3		Load3	No	
2	LD	F0	0	R1	6		Store1	Yes	80
2	MULTD	F4	F0	F2	7		Store2	Yes	72
2	SD	F4	0	R1	8		Store3	No	

Reservation Stations:

					S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI	R1	R1	#8
	Mult2	Yes	Multd		R(F2)	Load2		BNEZ	R1	Loop	

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
8	72	Fu	Load2	Mult2						

5.3.4 Tomasulo算法

Instruction status:

					Exec Write				
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	Load1	Yes	80
1	MULTD	F4	F0	F2	2		Load2	Yes	72
1	SD	F4	0	R1	3		Load3	No	
2	LD	F0	0	R1	6		Store1	Yes	80
2	MULTD	F4	F0	F2	7		Store2	Yes	72
2	SD	F4	0	R1	8		Store3	No	
									Mult1
									Mult2

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	S1	S2	RS	Qj	Qk	Code:
	Add1	No									LD
	Add2	No									F0
	Add3	No									0
	Mult1	Yes	Multd			R(F2)	Load1				R1
	Mult2	Yes	Multd			R(F2)	Load2				#8
											SubI
											R1
											R1
											Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
9	72	Fu	Load2	Mult2						

5.3.4 Tomasulo算法

Instruction status:

					Exec Write				
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	No
1	MULTD	F4	F0	F2	2		Load2	Yes	72
1	SD	F4	0	R1	3		Load3	No	
2	LD	F0	0	R1	6	10	Store1	Yes	80
2	MULTD	F4	F0	F2	7		Store2	Yes	72
2	SD	F4	0	R1	8		Store3	No	
									Mult1
									Mult2

Reservation Stations:

					S1		S2	RS			
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
4	Mult1	Yes	Multd	M[80]	R(F2)			SUBI	R1	R1	#8
	Mult2	Yes	Multd		R(F2)	Load2		BNEZ	R1	Loop	

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
10	64	Load2	Mult2							

5.3.4 Tomasulo算法

Instruction status:

					Exec Write					
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu	
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2			Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80
2	MULTD	F4	F0	F2	7			Store2	Yes	72
2	SD	F4	0	R1	8			Store3	No	
									Mult1	
									Mult2	

Reservation Stations:

					S1			S2			RS					
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:								
	Add1	No						LD	F0	0	R1					
	Add2	No						MULTD	F4	F0	F2					
	Add3	No						SD	F4	0	R1					
3	Mult1	Yes	Multd	M[80]	R(F2)			SUBI	R1	R1	#8					
4	Mult2	Yes	Multd	M[72]	R(F2)			BNEZ	R1	Loop						

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
11	64	Fu	Load3		Mult2					

5.3.4 Tomasulo算法

Instruction status:

					Exec Write					
ITER	Instruction		j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2			Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80
2	MULTD	F4	F0	F2	7			Store2	Yes	72
2	SD	F4	0	R1	8			Store3	No	
										Mult1
										Mult2

Reservation Stations:

				S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:		
	Add1	No						LD	F0	0
	Add2	No						MULTD	F4	F0
	Add3	No						SD	F4	0
2	Mult1	Yes	Multd	M[80]	R(F2)			SUBI	R1	R1
3	Mult2	Yes	Multd	M[72]	R(F2)			BNEZ	R1	Loop

←

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
12	64	Fu	Load3		Mult2						

5.3.4 Tomasulo算法

Instruction status:

						Exec Write					
ITER	Instruction		j	k	Issue	Comp	Result		Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	No		
1	MULTD	F4	F0	F2	2			Load2	No		
1	SD	F4	0	R1	3			Load3	Yes	64	
2	LD	F0	0	R1	6	10	11	Store1	Yes	80	Mult1
2	MULTD	F4	F0	F2	7			Store2	Yes	72	Mult2
2	SD	F4	0	R1	8			Store3	No		

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
1	Mult1	Yes	Multd	M[80]	R(F2)			SUBI	R1	R1	#8
2	Mult2	Yes	Multd	M[72]	R(F2)			BNEZ	R1	Loop	

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
13	64	Fu	Load3	Mult2							

5.3.4 Tomasulo算法

Instruction status:

					Exec Write					
ITER	Instruction		j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2	14		Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80
2	MULTD	F4	F0	F2	7			Store2	Yes	72
2	SD	F4	0	R1	8			Store3	No	
										Mult1
										Mult2

Reservation Stations:

				S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:		
	Add1	No						LD	F0	0
	Add2	No						MULTD	F4	F0
	Add3	No						SD	F4	0
0	Mult1	Yes	Multd	M[80]	R(F2)			SUBI	R1	R1
1	Mult2	Yes	Multd	M[72]	R(F2)			BNEZ	R1	Loop

←

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
14	64	Fu	Load3	Mult2							

5.3.4 Tomasulo算法

Instruction status:

					Exec Write					
ITER	Instruction	j	k		Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2	14	15	Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80
2	MULTD	F4	F0	F2	7	15		Store2	Yes	72
2	SD	F4	0	R1	8			Store3	No	
										[80]*R2
										Mult2

Reservation Stations:

					S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
	Mult1	No						SUBI	R1	R1	#8
0	Mult2	Yes	Multd	M[72]	R(F2)			BNEZ	R1	Loop	

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
15	64	Fu	Load3	Mult2							

5.3.4 Tomasulo算法

Instruction status:

					Exec Write					
ITER	Instruction		j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2	14	15	Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80 [80]*R2
2	MULTD	F4	F0	F2	7	15	16	Store2	Yes	72 [72]*R2
2	SD	F4	0	R1	8			Store3	No	

Reservation Stations:

				S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:		
	Add1	No						LD	F0	0 R1
	Add2	No						MULTD	F4	F0 F2
	Add3	No						SD	F4	0 R1
4	Mult1	Yes	Multd		R(F2)	Load3		SUBI	R1	R1 #8
	Mult2	No						BNEZ	R1	Loop

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
16	64	Fu	Load3		Mult1						

5.3.4 Tomasulo算法

Instruction status:

					Exec Write					
ITER	Instruction		j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2	14	15	Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80 [80]*R2
2	MULTD	F4	F0	F2	7	15	16	Store2	Yes	72 [72]*R2
2	SD	F4	0	R1	8			Store3	Yes	64 Mult1

Reservation Stations:

				S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:		
	Add1	No						LD	F0	0 R1
	Add2	No						MULTD	F4	F0 F2
	Add3	No						SD	F4	0 R1
	Mult1	Yes	Multd		R(F2)	Load3		SUBI	R1	R1 #8
	Mult2	No						BNEZ	R1	Loop

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
17	64	Fu	Load3		Mult1						

5.3.4 Tomasulo算法

Instruction status:

ITER	Instruction		j	k	Issue	Comp	Result	Exec	Write
1	LD	F0	0	R1	1	9	10	Load1	
1	MULTD	F4	F0	F2	2	14	15	Load2	
1	SD	F4	0	R1	3	18		Load3	
2	LD	F0	0	R1	6	10	11	Store1	
2	MULTD	F4	F0	F2	7	15	16	Store2	
2	SD	F4	0	R1	8			Store3	

Busy	Addr	Fu
No		
No		
Yes	64	
Yes	80	[80]*R2
Yes	72	[72]*R2
Yes	64	Mult1

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	S1	S2	RS	Code:
	Add1	No							LD
	Add2	No							F0
	Add3	No							0
	Mult1	Yes	Multd		R(F2)	Load3			R1
	Mult2	No							F2

SD	F4	0	R1
SUBI	R1	R1	#8
BNEZ	R1	Loop	

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
18	64	Fu	Load3		Mult1						

5.3.4 Tomasulo算法

Instruction status:

					Exec Write					
ITER	Instruction		j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2	14	15	Load2	No	
1	SD	F4	0	R1	3	18	19	Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	No	
2	MULTD	F4	F0	F2	7	15	16	Store2	Yes	72 [72]*R2
2	SD	F4	0	R1	8	19		Store3	Yes	64 Mult1

Reservation Stations:

					S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
	Mult1	Yes	Multd		R(F2)	Load3		SUBI	R1	R1	#8
	Mult2	No						BNEZ	R1	Loop	

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
19	56	Fu	Load3		Mult1						

5.3.4 Tomasulo算法

Instruction status:

					Exec Write					
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu	
1	LD	F0	0	R1	1	9	10	Load1	Yes	56
1	MULTD	F4	F0	F2	2	14	15	Load2	No	
1	SD	F4	0	R1	3	18	19	Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	No	
2	MULTD	F4	F0	F2	7	15	16	Store2	No	
2	SD	F4	0	R1	8	19	20	Store3	Yes	64
										Mult1

Reservation Stations:

					S1			S2			RS					
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:								
	Add1	No						LD	F0	0	R1					
	Add2	No						MULTD	F4	F0	F2					
	Add3	No						SD	F4	0	R1					
	Mult1	Yes	Multd		R(F2)	Load3		SUBI	R1	R1	#8					
	Mult2	No						BNEZ	R1	Loop						

按序发射,
乱序执行,
乱序完成

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
20	56	Fu	Load1	Mult1						

Tomasulo算法具有两个主要的优点：

- 冲突检测逻辑是分布的

(通过保留站和CDB实现)

- 如果有多条指令已经获得了一个操作数，并同时在等待同一运算结果，那么这个结果一产生，就可以通过CDB同时播送给所有这些指令，使它们可以同时执行。

- 消除了WAW冲突和WAR冲突导致的停顿

使用保留站进行寄存器换名，并且在操作数一旦就绪就将之放入保留站。

5.3.4 Tomasulo算法

第 198 页

例5.4 对于例5.4中的代码，假设各种操作的延迟为：

load：1个时钟周期

加法：2个时钟周期

乘法：10个时钟周期

除法：40个时钟周期

给出MUL.D指令准备写结果时各状态表的内容。

解 MUL.D指令准备写结果时各状态表的内容如下图所示。

5.3.4 Tomasulo算法

当采用Tomasulo算法时，在上述给定的时刻保留站、load缓冲器以及寄存器状态表中的内容。

指令		指令状态表		
		流出	执行	结果
L.D	F6 , 34(R2)	√	√	√
L.D	F2 , 45(R3)	√	√	√
MUL.D	F0 , F2 , F4	√	√	
SUB.D	F8 , F6 , F2	√	√	√
DIV.D	F10 , F0 , F6	√		
ADD.D	F6 , F8 , F2	√	√	√

5.3.4 Tomasulo算法

名称	保留站						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	No						
Add1	No						
Add2	No						
Add3	No						
Mult1	Yes	MUL.D	Mem[45+Regs[R3]]	Reg[F4]			
Mult2	Yes	DIV.D		Mem[34+Regs[R2]]	Mult1		

	寄存器状态表							
	F0	F2	F4	F6	F8	F10		F30
Qi	Mult1					Mult2		

5.3.4 Tomasulo算法

第 201 页

Tomasulo算法存在的主要问题：

- Tomasulo算法中每一个执行单元对应一个保留站，保留站中缓冲多条指令，所以有可能在同一周期有多条指令准备好数据，但是执行单元同时只能执行一条指令，所以就需要从中选择一条指令，简单的解决办法是为保留站的每一行增加一个年龄位，每次出现冲突，就选择最老的指令送到执行单元。
- Tomasulo算法没办法实现精确中断。精确中断是指在指令和指令之间如果出现了中断/异常，那么处理器要确保中断/异常之前的所有指令都执行完毕，而中断/异常之后的所有指令都没有执行，然后处理器把中断发生时的处理器状态给保存下来或是呈现给程序员看。精确中断保证了程序的正确执行，也提供程序员调试程序的手段，是现代处理器必不可少的能力，而Tomasulo并不支持。

要支持精确中断，就要确保指令按序提交

5.3.5 重排序缓存ROB

第 202 页

Instruction status:

Instruction	<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1
LD	F2	45+	R3	2	4	5	Load2
MULTD	F0	F2	F4	3	15	16	Load3
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5	56	57	
ADDD	F6	F8	F2	6	10	11	

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> <i>Vj</i>	<i>S2</i> <i>Vk</i>	<i>RS</i> <i>Qj</i>	<i>RS</i> <i>Qk</i>
Add1		No					
Add2		No					
Add3		No					
Mult1		No					
Mult2		Yes	DIVD	M*F4	M(A1)		

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
56	FU	M*F4	M(A2)		(M-M+N	(M-M)	Result		

希望：
按序发射，
乱序执行，
按序完成

5.3.5 重排序缓存ROB

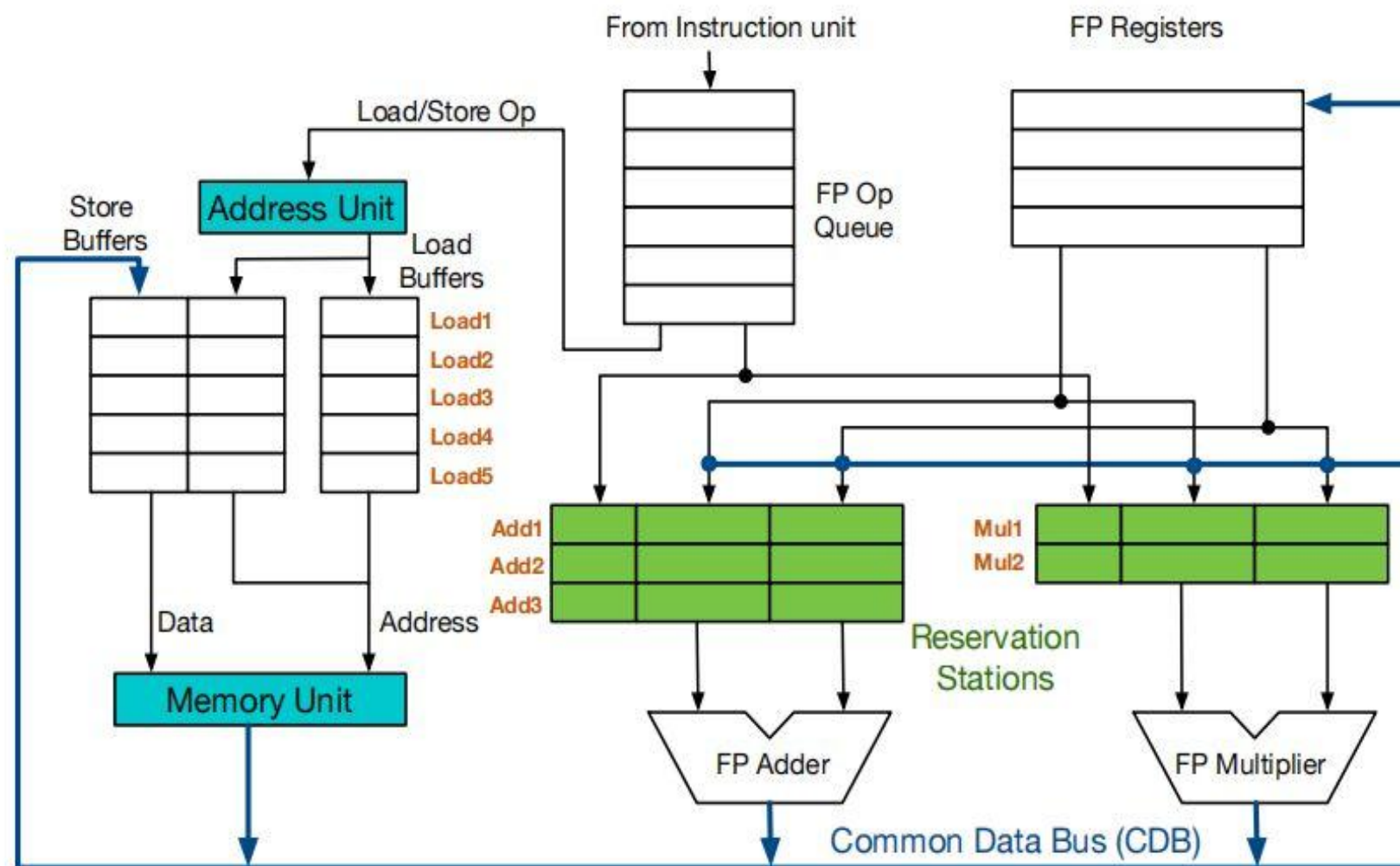
第 203 页

- 由于控制指令、程序异常和外部中断也会截断指令流，所以通过顺序提交指令来实现精确中断至关重要
- 重排序缓存（Reorder Buffer, ROB）的目的是让乱序执行的指令被顺序地提交，现在已经成为乱序处理器的核心基础
- ROB的核心思想是记录下指令在程序中的顺序，一条指令在执行完毕之后不会立马提交（例如修改逻辑寄存器堆RF中的寄存器值），而是先在Buffer中等待直等到前面的所有指令都提交完毕，才可以提交结果到逻辑寄存器堆。

5.3.5 重排序缓存ROB

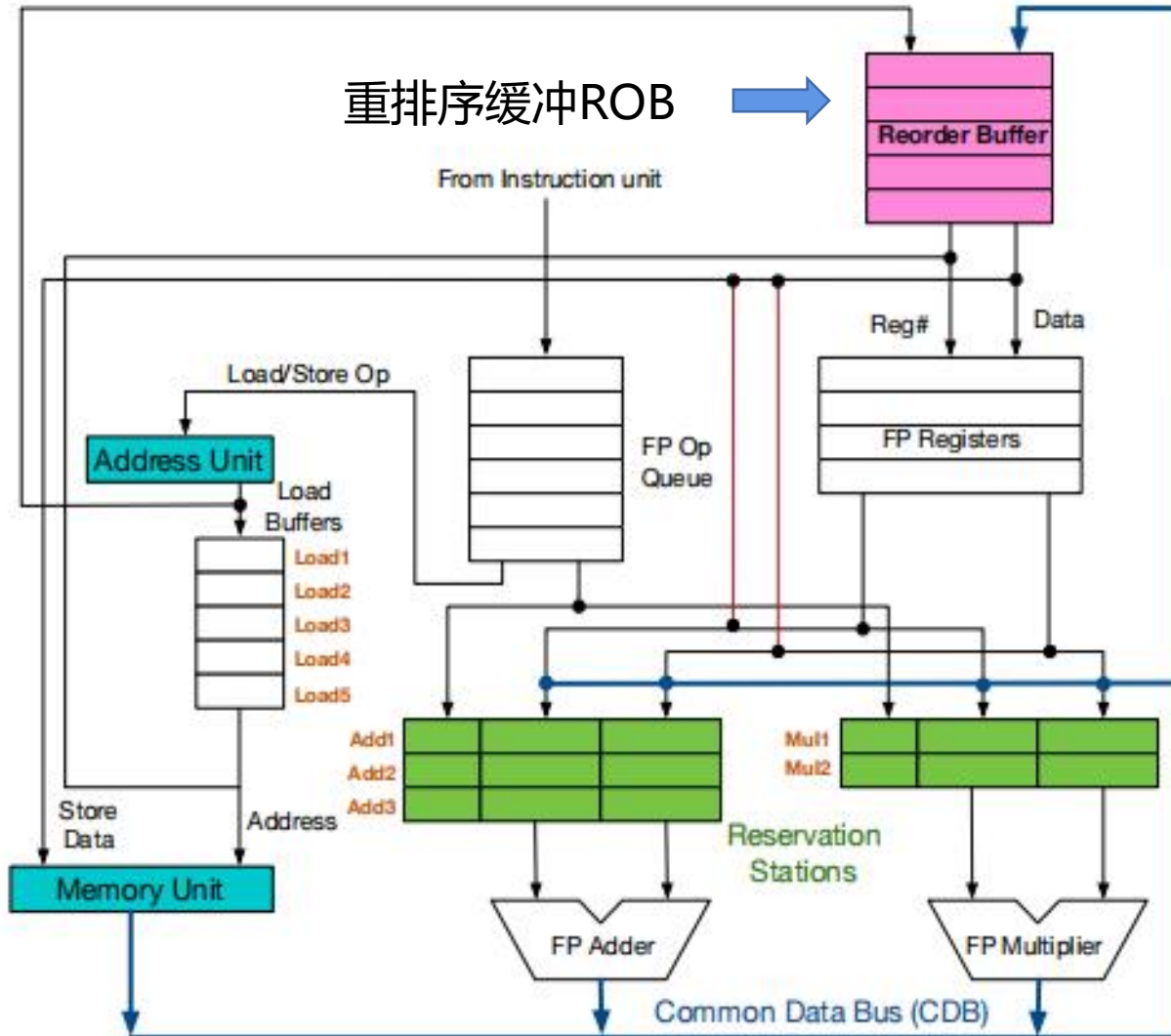
第 204 页

原始Tomasulo算法
结构图



5.3.5 重排序缓存ROB

第 205 页

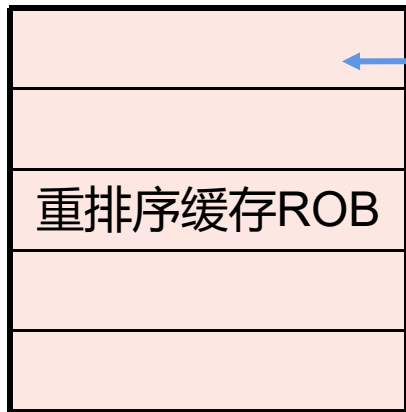


改进Tomasulo算法结构图：

1. 增加了Reorder Buffer（即ROB）；
2. CDB总线不再直通逻辑寄存器堆，而是直通ROB；
3. 指令需要从ROB读取数据。ROB在这里就像是一个FIFO队列，指令在发射时入队，在提交时出队

5.3.5 重排序缓存ROB

第 206 页



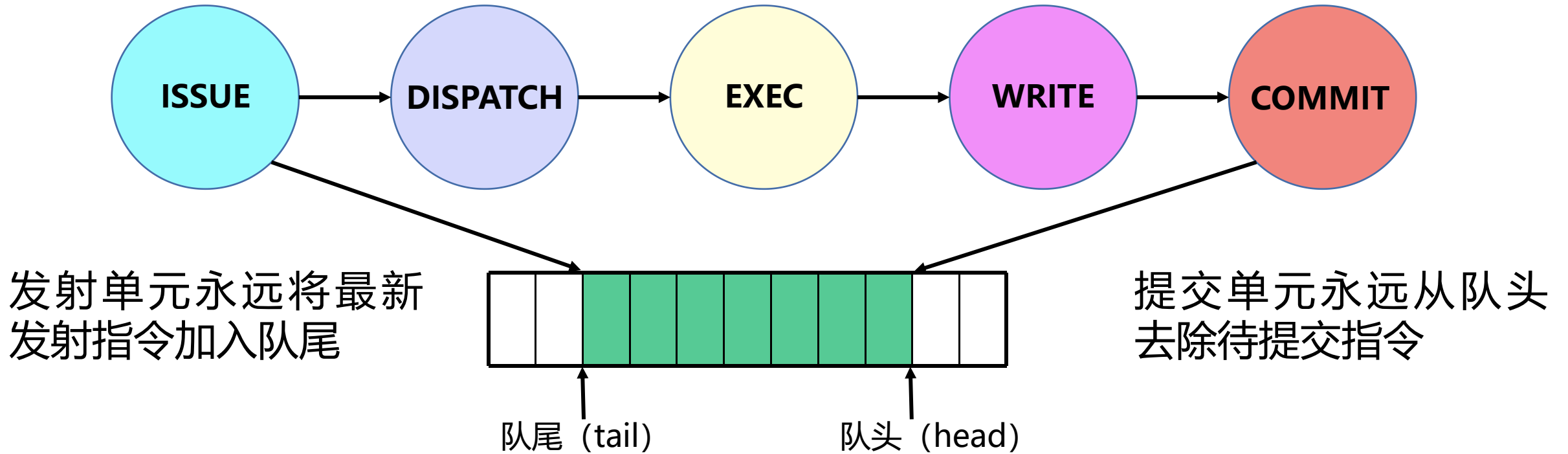
Entry	Busy	Instruction	State	Destination	Value
1	yes	LD F6, 34(R2)	issue	F6	

- Entry位：ROB在每条指令发射的时候会为其分配一个编号，这个编号可以取代保留站编号；
- Busy位：指示某一行是否正保存有指令；
- State位：用来指示保存的指令当前的运行情况，ROB就是通过State的信息来判断某条指令是否可以提交；当最老的指令还没到提交阶段时，ROB中的所有指令都要等待，不能提交；
- Destination位：指示指令的目的寄存器；
- Value位：保存指令的结果，当指令可以提交，就直接提交Value到逻辑寄存器。

5.3.5 重排序缓存ROB

第 207 页

■ ROB的基本执行思路



- 注意1：存储到缓冲区的写入（write）直到等到准备提交（commit）
- 注意2：按fifo顺序写入和释放循环队列
- 注意3：每个条目为1条指令分配空间，以存储其结果和标识信息

5.3.5 重排序缓存ROB

第 208 页

例5.4 对于下述指令序列，给出当第一条指令完成并写入结果时，Tomasulo算法+ROB所用的各信息表中的内容。

L.D	F6,34 (R2)
L.D	F2,45 (R3)
MUL.D	F0,F2,F4
SUB.D	F8,F2,F6
DIV.D	F10,F0,F6
ADD.D	F6,F8,F2

5.3.5 重排序缓存ROB

时钟周期1:
发射 L.D F6,34 (R2)

重排序缓冲器

Entry	Busy	Instruction	State	Destination	Value
1(head)	yes	LD F6, 34(R2)	issue	F6	
2	no				
3	no				
4	no				
5	no				
6	no				

保留站

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	yes	LD	[R2]=200				34	#1
	Load2	no							
	Add1	no							
	Add2	no							
	Mult1	no							
	Mult2	no							

寄存器结果状态

	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #				#1					
Busy	no	no	no	yes	no	no	no		no

5.3.5 重排序缓存ROB

时钟周期2:
执行 L.D F6,34 (R2)
发射 L.D F2,45 (R3)

重排序缓冲器



Entry	Busy	Instruction	State	Destination	Value
1(head)	yes	LD F6, 34(R2)	exec	F6	
2(tail)	yes	L.D F2,45(R3)	issue	F2	
3	no				
4	no				
5	no				
6	no				

保留站



Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	yes	LD					234	#1
	Load2	yes	LD	[R3]=300				45	#2
	Add1	no							
	Add2	no							
	Mult1	no							
	Mult2	no							

寄存器结果状态



	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #		#2		#1					
Busy	no	yes	no	yes	no	no	no		no

5.3.5 重排序缓存ROB

第 211 页

时钟周期3:

写回 L.D F6,34 (R2)

执行 L.D F2,45 (R3)

发射 MULTD F0,F2,F4

重排序缓冲器



Entry	Busy	Instruction	State	Destination	Value
1(head)	yes	LD F6, 34(R2)	write	F6	Mem[234]
2	yes	L.D F2,45(R3)	exec	F2	
3(tail)	yes	MULTD F0, F2, F4	issue	F0	
4	no				
5	no				
6	no				

保留站



Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	yes	LD	[R3]=300				345	#2
	Add1	no							
	Add2	no							
	Mult1	yes	MULTD		[F4]=4.0	#2			#3
	Mult2	no							

寄存器结果状态



	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #	#3	#2		#1					
Busy	yes	yes	no	yes	no	no	no		no

5.3.5 重排序缓存ROB

时钟周期4:
写回 L.D F2,45 (R3)
(广播)
发射 SUBB F8,F2,F6

重排序缓冲器

Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2(head)	yes	L.D F2,45(R3)	write	F2	Mem[345]
3	yes	MULTD F0, F2, F4	exec1	F0	
4(tail)	yes	SUBD F8, F2, F6	issue	F8	
5	no				
6	no				

保留站

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	yes	SUBD	345.0	[F6]=234.0				#4
	Add2	no							
	Mult1	yes	MULTD	345.0	[F4]=4.0				#3
	Mult2	no							

寄存器结果状态

	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #	#3	#2			#4				
Busy	yes	yes	no	no	yes	no	no		no

5.3.5 重排序缓存ROB

第 213 页

时钟周期5:
发射DIV.D F10,F0,F6

重排序缓冲器



Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2	no	L.D F2,45(R3)	commit	F2	Mem[345]
3(head)	yes	MULTD F0, F2, F4	exec2	F0	
4	yes	SUBD F8, F2, F6	exec1	F8	
5(tail)	yes	DIVD F10, F0, F6	issue	F10	
6	no				

保留站



Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	yes	SUBD	345.0	234.0				#4
	Add2	no							
	Mult1	yes	MULTD	345.0	4.0				#3
	Mult2	yes	DIVD		[F6]=234.0	#3			#5

寄存器结果状态



	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #	#3				#4	#5			
Busy	yes	no	no	no	yes	yes	no		no

5.3.5 重排序缓存ROB

时钟周期6:
发射ADD.D F6,F8,F2

重排序缓冲器

Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2	no	L.D F2,45(R3)	commit	F2	Mem[345]
3(head)	yes	MULTD F0, F2, F4	exec3	F0	
4	yes	SUBD F8, F2, F6	exec2	F8	
5	yes	DIVD F10, F0, F6	issue	F10	
6(tail)	yes	ADDD F6, F8, F2	issue	F6	

保留站

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	yes	SUBD	345.0	234.0				#4
	Add2	yes	ADDD		[F2]=345.0	#4			#6
	Mult1	yes	MULTD	345.0	4.0				#3
	Mult2	yes	DIVD		234.0	#3			#5

寄存器结果状态

	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #	#3			#6	#4	#5			
Busy	yes	no	no	yes	yes	yes	no		no

5.3.5 重排序缓存ROB

时钟周期7:
SUBD指令执行写回

重排序缓冲器

Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2	no	L.D F2,45(R3)	commit	F2	Mem[345]
3(head)	yes	MULTD F0, F2, F4	exec4	F0	
4	yes	SUBD F8, F2, F6	write	F8	111.0
5	yes	DIVD F10, F0, F6	issue	F10	
6(tail)	yes	ADDD F6, F8, F2	exec1	F6	

保留站

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	no							
	Add2	yes	ADDD	111.0	345.0				#6
	Mult1	yes	MULTD	345.0	4.0				#3
	Mult2	yes	DIVD		234.0	#3			#5

寄存器结果状态

	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #	#3			#6	#4	#5			
Busy	yes	no	no	yes	yes	yes	no		no

5.3.5 重排序缓存ROB

第 216 页

时钟周期8:
SUBD等待结束
其他指令执行

重排序缓冲器



Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2	no	L.D F2,45(R3)	commit	F2	Mem[345]
3(head)	yes	MULTD F0, F2, F4	exec5	F0	
4	yes	SUBD F8, F2, F6	write	F8	111.0
5	yes	DIVD F10, F0, F6	issue	F10	
6(tail)	yes	ADDD F6, F8, F2	exec2	F6	

保留站



Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	no							
	Add2	yes	ADDD	111.0	345.0				#6
	Mult1	yes	MULTD	345.0	4.0				#3
	Mult2	yes	DIVD		234.0	#3			#5

寄存器结果状态



	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #	#3			#6	#4	#5			
Busy	yes	no	no	yes	yes	yes	no		no

5.3.5 重排序缓存ROB

时钟周期9:
ADDD指令执行写回

重排序缓冲器

Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2	no	L.D F2,45(R3)	commit	F2	Mem[345]
3(head)	yes	MULTD F0, F2, F4	exec6	F0	
4	yes	SUBD F8, F2, F6	write	F8	111.0
5	yes	DIVD F10, F0, F6	issue	F10	
6(tail)	yes	ADDD F6, F8, F2	write	F6	456.0

保留站

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	no							
	Add2	no							
	Mult1	yes	MULTD	345.0	4.0				#3
	Mult2	yes	DIVD		234.0	#3			#5

寄存器结果状态

	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #	#3			#6	#4	#5			
Busy	yes	no	no	yes	yes	yes	no		no

5.3.5 重排序缓存ROB

时钟周期13:
MULTD指令执行写回

重排序缓冲器

Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2	no	L.D F2,45(R3)	commit	F2	Mem[345]
3(head)	yes	MULTD F0, F2, F4	write	F0	1380.0
4	yes	SUBD F8, F2, F6	write	F8	111.0
5	yes	DIVD F10, F0, F6	exec1	F10	
6(tail)	yes	ADDD F6, F8, F2	write	F6	456.0

保留站

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	no							
	Add2	no							
	Mult1	no							
	Mult2	yes	DIVD		234.0	1380.0			#5

寄存器结果状态

	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #	#3			#6	#4	#5			
Busy	yes	no	no	yes	yes	yes	no		no

5.3.5 重排序缓存ROB

时钟周期14:
MULTD指令完成

重排序缓冲器

Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2	no	L.D F2,45(R3)	commit	F2	Mem[345]
3	no	MULTD F0, F2, F4	commit	F0	1380.0
4(head)	yes	SUBD F8, F2, F6	write	F8	111.0
5	yes	DIVD F10, F0, F6	exec2	F10	
6(tail)	yes	ADDD F6, F8, F2	write	F6	456.0

保留站

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	no							
	Add2	no							
	Mult1	no							
	Mult2	yes	DIVD		234.0	1380.0			#5

寄存器结果状态

	F0	F2	F4	F6	F8	F10	F12		F30
Recoder #				#6	#4	#5			
Busy	no	no	no	yes	yes	yes	no		no

5.3.5 重排序缓存ROB

第 220 页

时钟周期15-51:
SUBD指令完成
DIVD指令执行
ADDD指令等待

重排序缓冲器

Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2	no	L.D F2,45(R3)	commit	F2	Mem[345]
3	no	MULTD F0, F2, F4	commit	F0	1380.0
4	no	SUBD F8, F2, F6	commit	F8	111.0
5(head)	yes	DIVD F10, F0, F6	exec39	F10	5.9
6(tail)	yes	ADDD F6, F8, F2	write	F6	456.0

保留站

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	no							
	Add2	no							
	Mult1	no							
	Mult2	yes	DIVD		234.0	1380.0			#5

寄存器结果状态

[illegible]

5.3.5 重排序缓存ROB

第 221 页

时钟周期52:
ADDD指令完成

重排序缓冲器

Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2	no	L.D F2,45(R3)	commit	F2	Mem[345]
3	no	MULTD F0, F2, F4	commit	F0	1380.0
4	no	SUBD F8, F2, F6	commit	F8	111.0
5	no	DIVD F10, F0, F6	commit	F10	5.9
6(head)	yes	ADDD F6, F8, F2	write	F6	456.0

保留站

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	no							
	Add2	no							
	Mult1	no							
	Mult2	no							

寄存器结果状态

[illegible]

5.3.5 重排序缓存ROB

第 222 页

时钟周期53:

重排序缓冲器

Entry	Busy	Instruction	State	Destination	Value
1	no	LD F6, 34(R2)	commit	F6	Mem[234]
2	no	L.D F2,45(R3)	commit	F2	Mem[345]
3	no	MULTD F0, F2, F4	commit	F0	1380.0
4	no	SUBD F8, F2, F6	commit	F8	111.0
5	no	DIVD F10, F0, F6	commit	F10	5.9
6	no	ADDD F6, F8, F2	commit	F6	456.0

保留站

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	A	Dest
	Load1	no							
	Load2	no							
	Add1	no							
	Add2	no							
	Mult1	no							
	Mult2	no							

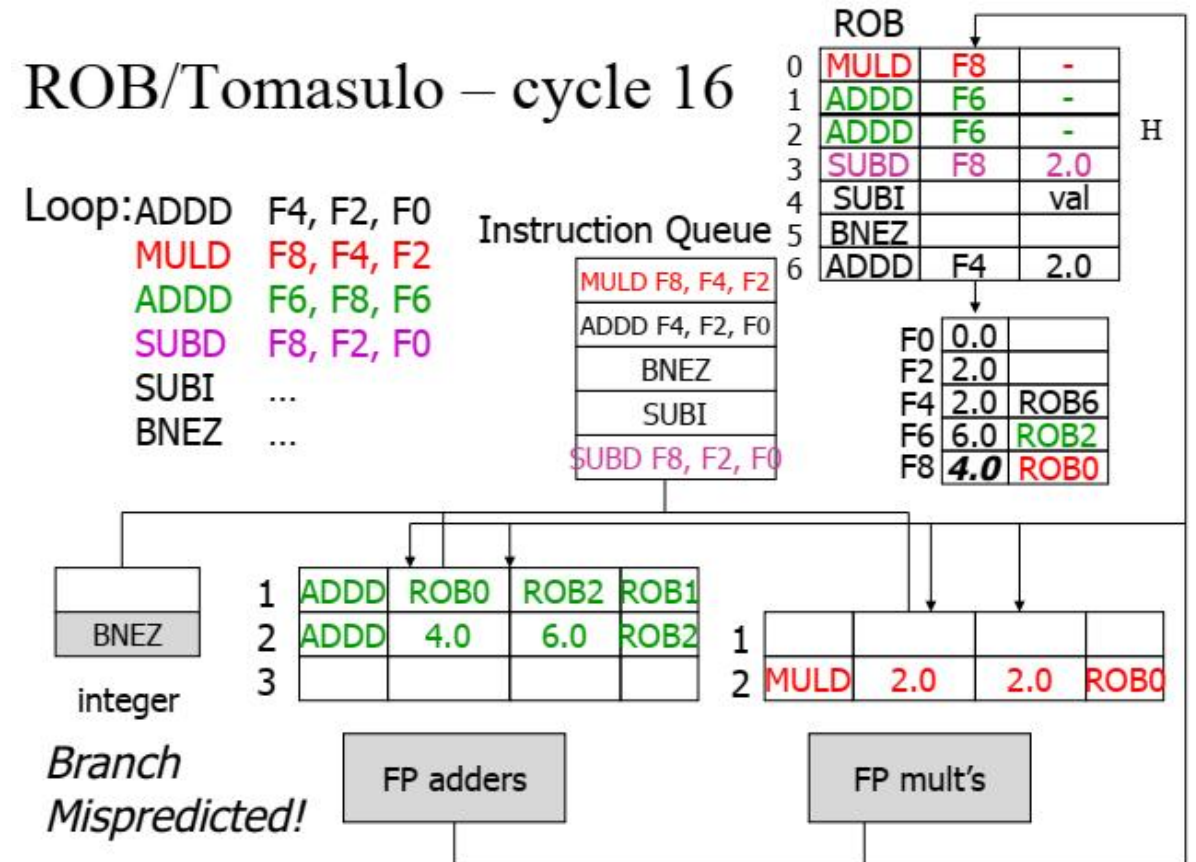
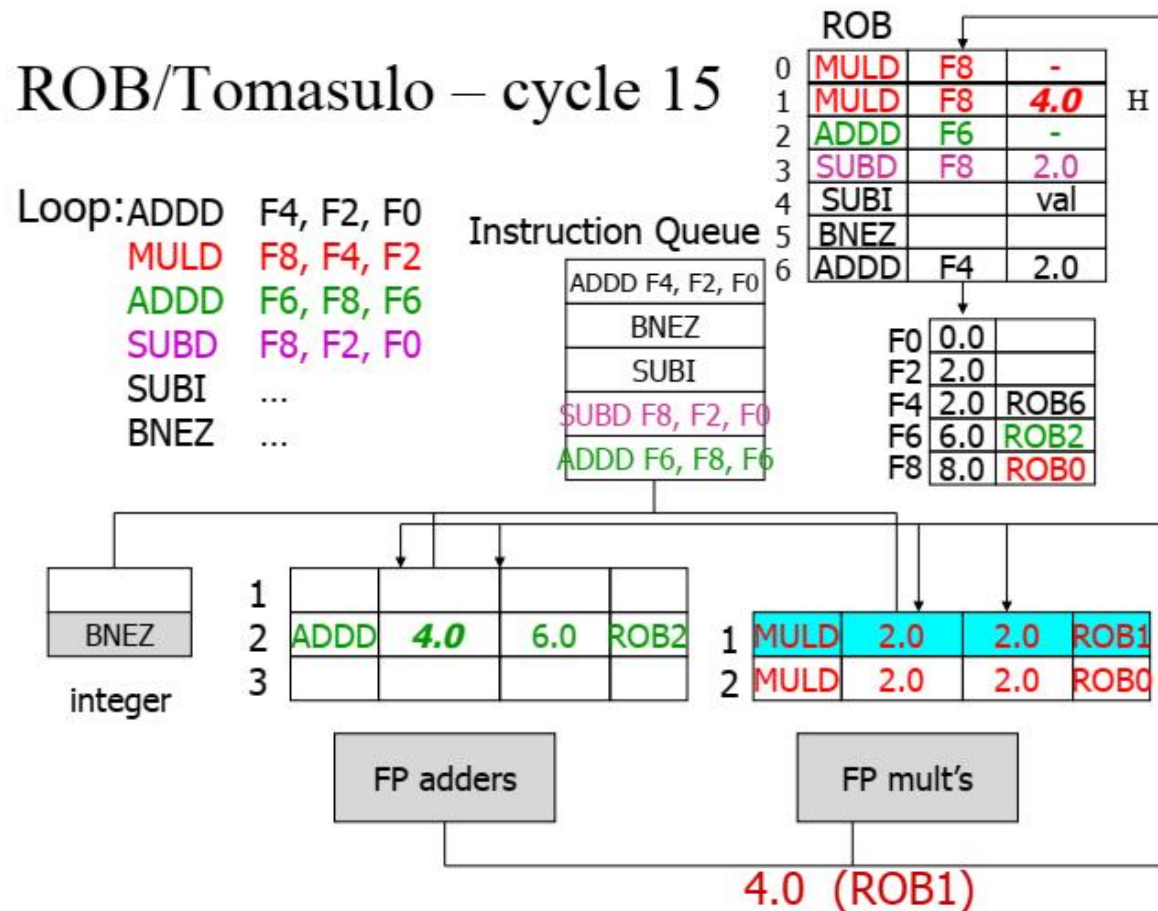
寄存器结果状态

[illegible]

5.3.5 重排序缓存ROB

第 223 页

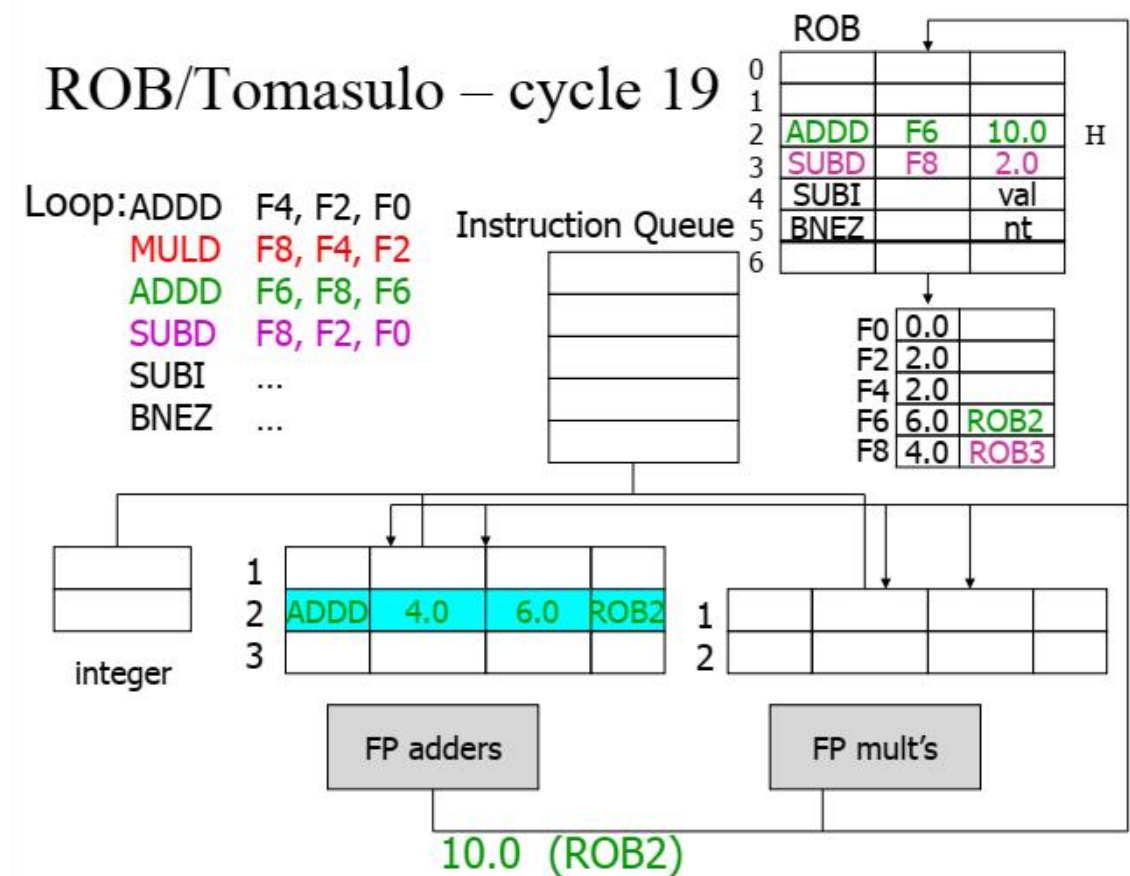
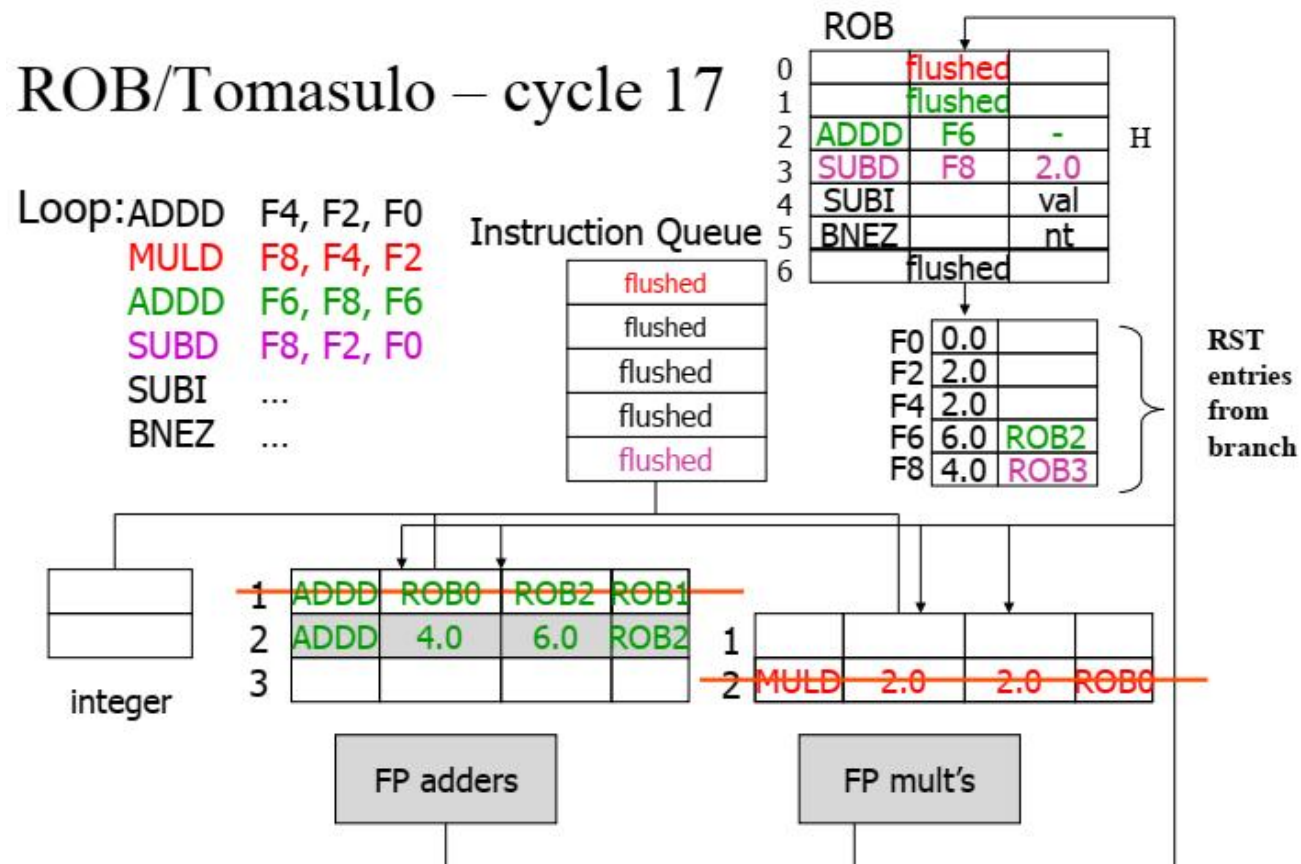
■ Tomasulo+ROB实现精确中断



5.3.5 重排序缓存ROB

第 224 页

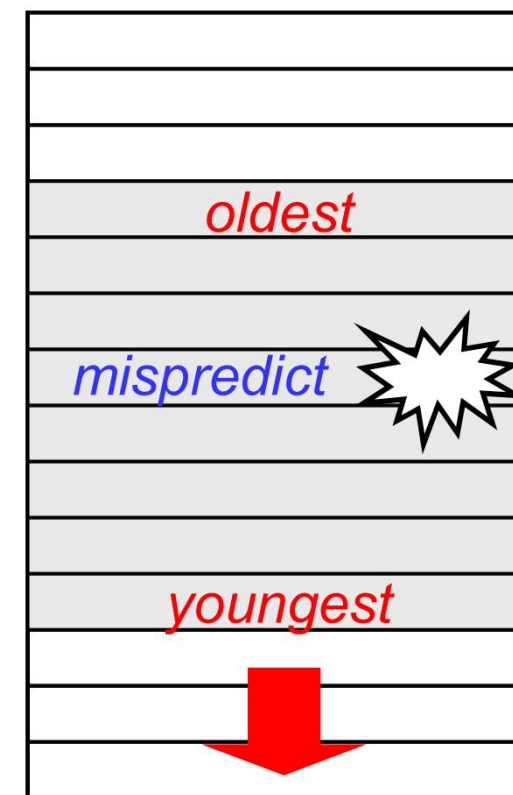
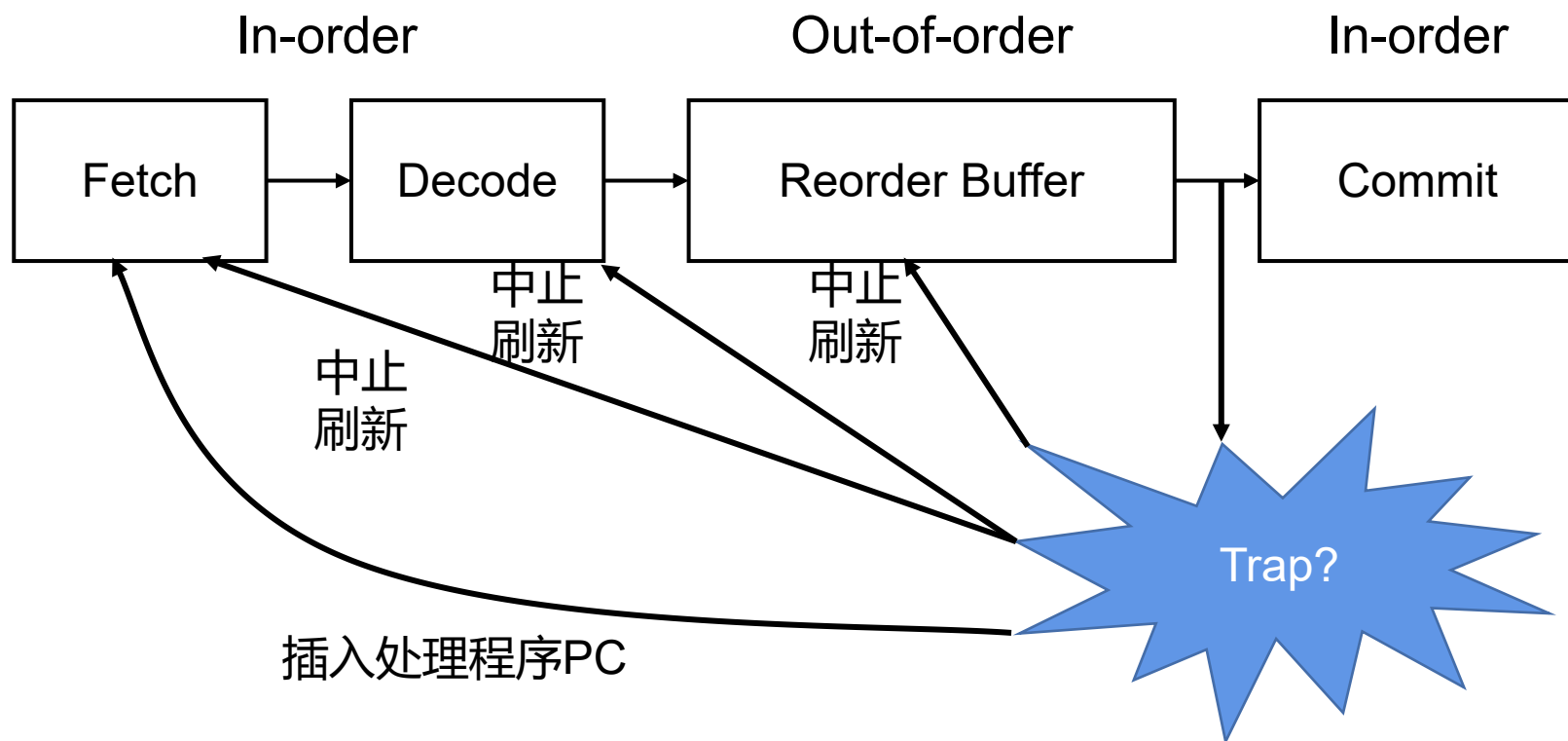
■ Tomasulo+ROB实现精确中断



5.3.5 重排序缓存ROB

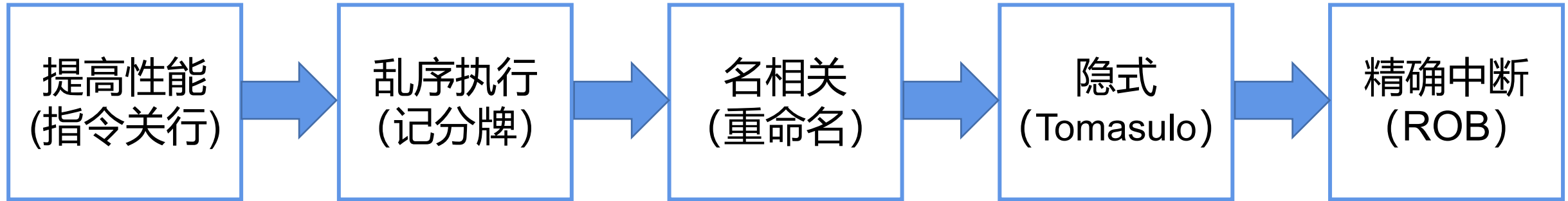
第 225 页

■ Tomasulo+ROB实现精确中断



5.3.5 重排序缓存ROB

第 226 页



- 数据相关——真相关
- 名相关——WAR、WAW
- 控制相关——分支指令

	有条件	无条件
直接	if-then-else for loops (bez, bnez, etc)	procedure calls(jal) goto(j)
间接		return(jr) virtual function lookup function pointer(jalr)

5.4 动态分支预测技术

5.4.1 分支指令的影响

第 228 页

五级流水线时空图

指令	时钟								
	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
i+1		IF	ID	EX	MEM	WB			
i+2			IF	ID	EX	MEM	WB		
i+3		k=5		IF	ID	EX	MEM	WB	
i+4					IF	ID	EX	MEM	WB

5.4.1 分支指令的影响

第 229 页

■ 最大吞吐率

$$TP_{\max} = \lim_{n \rightarrow \infty} \frac{n}{(n+k-1)\Delta t} = \frac{1}{\Delta t}$$

各段时间均相等的流水线

■ 最大吞吐率与实际吞吐率的关系

$$TP = \frac{n}{n+k-1} TP_{\max}$$

问题：如何保障流水线不断流？

只有当 $n \gg k$ 时，才有 $TP \approx TP_{\max}$

5.4.1 分支指令的影响

第 230 页

指令	时钟								
	1	2	3	4	5	6	7	8	9
分支指令 i	IF	ID	EX	MEM	WB				
i+1		IF	ID	EX	MEM	WB			
i+2	成功		IF	ID	EX	MEM	WB		
j				IF	ID	EX	MEM	WB	
j+1					IF	ID	EX	MEM	WB

分支指令

无效指令

对于一条有 k 个流水段的流水线，在最坏情况下每一次条件转移（条件分支）指令将造成 m 个时钟的“停顿”；假设条件转移指令在一般程序中所占的比例为 p ，转移成功的概率为 q 。则 n 条指令的总执行时间是：

$$T_{K-IF} = (n + k - 1) \cdot \Delta t + p \cdot q \cdot n \cdot m \cdot \Delta t$$

5.4.1 分支指令的影响

第 231 页

■有条件转移影响的流水线吞吐率

$$TP_{IF} = \frac{n}{(n + k - 1) \Delta t + pqnm \Delta t}$$

■有条件转移影响的最大流水线吞吐率

$$TP_{MAX - IF} = \frac{1}{(1 + pqm) \Delta t}$$

降低流水线吞吐率

5.4.1 分支指令的影响

第 232 页

■流水线吞吐率下降的比例

$$D = \frac{TP_{MAX} - TP_{MAX-IF}}{TP_{MAX}} = 1 - \frac{\frac{1}{(1 + pqm)\Delta t}}{\frac{1}{\Delta t}} = \frac{pqm}{1 + pqm}$$

如果 $m = k - 1$, 则

$$= \frac{pq(k-1)}{1 + pq(k-1)}$$

5.4.1 分支指令的影响

第 233 页

■ 流水线吞吐率下降的比例

$$D = \frac{TP_{MAX} - TP_{MAX-IF}}{TP_{MAX}} = \frac{pq(k-1)}{1 + pq(k-1)}$$

■ 例子

据统计：在一些典型程序中 $p=20\%$ 、 $q=60\%$ ，对于有8个流水段的流水线，最大吞吐率会下降46%；对于有10个流水段的流水线，最大吞吐率会下降52%。

条件转移指令对流水线的影响很大，必须采取措施降低影响

5.4.1 分支指令的影响

第 234 页

■ 有条件转移影响的最大流水线吞吐率

$$TP_{MAX-IF} = \frac{1}{(1 + pqm) \Delta t}$$

条件转移指令比例为p
转移成功的概率为q
转移成功的开销m

■ 软件解决方案

- 减少分支：循环展开 (Loop Unrolling) —— 优化 p
- 提前形成条件码 (Reduce resolution time) —— 优化 m
- 延迟转移 (Delay Slot, 编译技术) —— 优化 m

■ 硬件解决方案

- 延迟转移 (Delay Slot, FLUSH技术) —— 优化 m
- 前瞻执行或分支预测 (Speculation - branch prediction) —— 优化 q

5.4.1 分支指令的影响

第 235 页

■ 循环展开 (Loop Unrolling)

```
void test(void)
{
    int x[100];
    int i;
    for (i = 1; i <= 100; i++) {
        x[i] = x[i] * 2 + 10;
    }
}
```

原程序

```
void test(void)
{
    int x[100];
    int i;
    for (i = 1; i <= 100; i+=2) {
        x[i+0] = x[i+0] * 2 + 10;
        x[i+1] = x[i+1] * 2 + 10;
    }
}
```

改进程序

优点:

- 1、减少了分支预测失败的可能性;
- 2、增加了循环体内语句并发执行的可能性 (需要循环体内各语句不存在数据相关性)。

缺点:

- 1、造成代码膨胀;
- 2、代码可读性显著降低。

5.4.1 分支指令的影响

■提前形成条件码

可在运算开始或中间产生条件码，在绝大多数情况下，只要在运算部件的入口处设置一个比较器，通过比较两个操作数的符号或者阶码就能够提前形成结果的正负号、是否为0、是否溢出等条件码。如果在一个时钟周期之内产生条件码，则正好可以提供给下一条条件转移指令使用，这样流水线就不会“断流”。

指令	时钟								
	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
i+1		IF	ID	EX	MEM	WB			
i+2			IF	ID	EX	MEM	WB		
j				IF	ID	EX	MEM	WB	
j+1					IF	ID	EX	MEM	WB



指令	时钟								
	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
j		IF	ID	EX	MEM	WB			
j+1			IF	ID	EX	MEM	WB		
j+2				IF	ID	EX	MEM	WB	
j+3					IF	ID	EX	MEM	WB

5.4.1 分支指令的影响

第 237 页

■提前形成条件码

编译前:

```
        LOAD R1,NUM    ;循环次数初值装入R1
LOOP:    .....        ;循环体开始

        .....
        DEC  R1         ;循环次数减 "1"
        BNE LOOP       ;测试循环是否则结束
        HALT           ;程序结束
NUM: n                ;循环次数
```

5.4.1 分支指令的影响

第 238 页

■提前形成条件码

编译后：

LOAD R1,NUM	;循环次数装入R1中
LOOP: LDEC R1	;一条专用的循环次数减1指令
.....	;循环体开始
.....	
LBNE LOOP	;一条专用的测试循环是否结束的指令
HALT	;程序结束
NUM: n	;循环次数

【提示】 指令LDEC和LBNE使用专用的条件码寄存器。

5.4.1 分支指令的影响

第 239 页

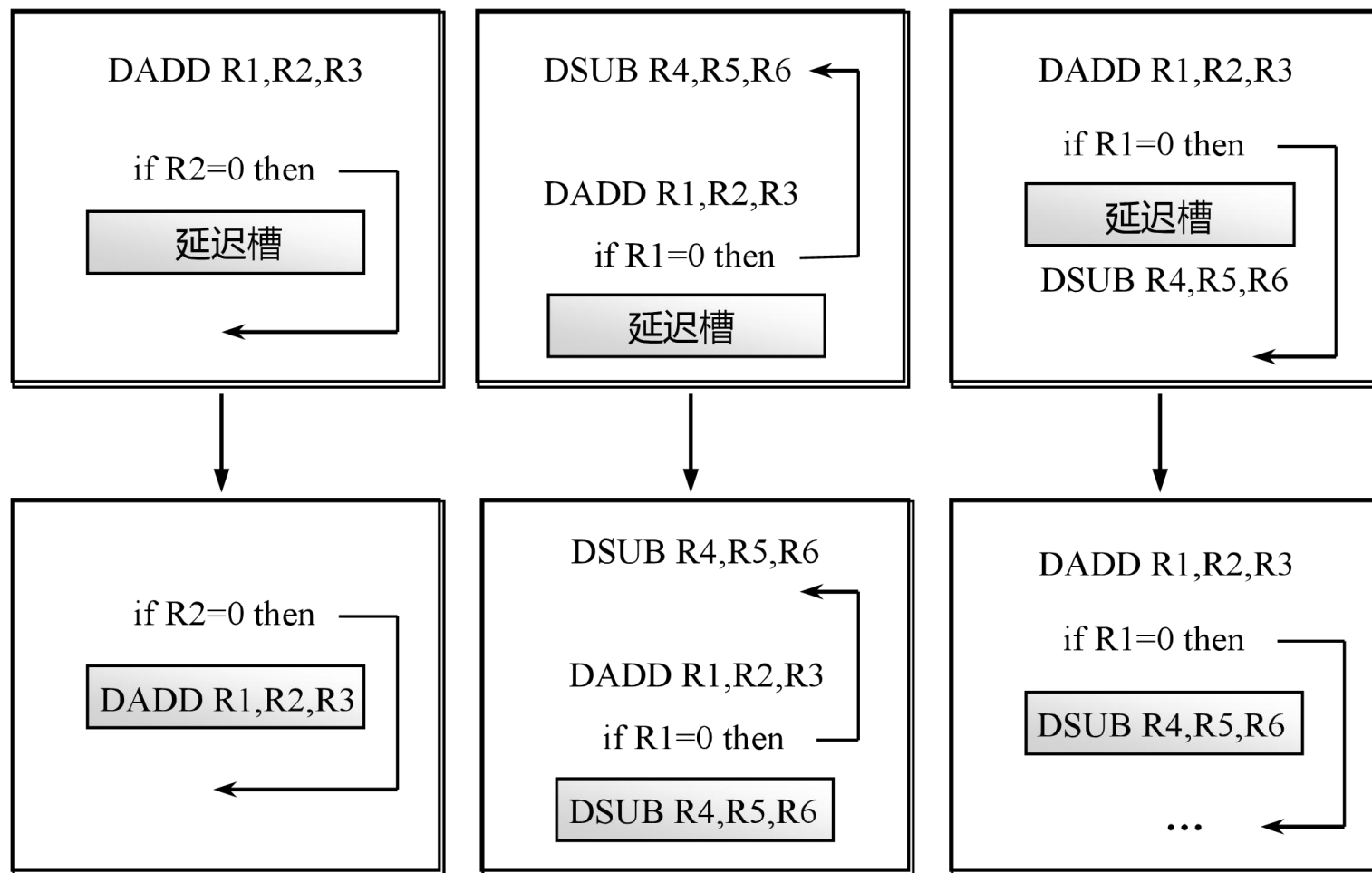
■ 延迟转移 Delay Slot

➤ 任务：在延迟槽中放入有用的指令，由编译器完成。

➤ 三种调度方法：

- ✓ 从前调度
- ✓ 从目标处调度
- ✓ 从失败处调度

如何硬件确定在延迟槽中放入有用的指令？



(a) 从前调度

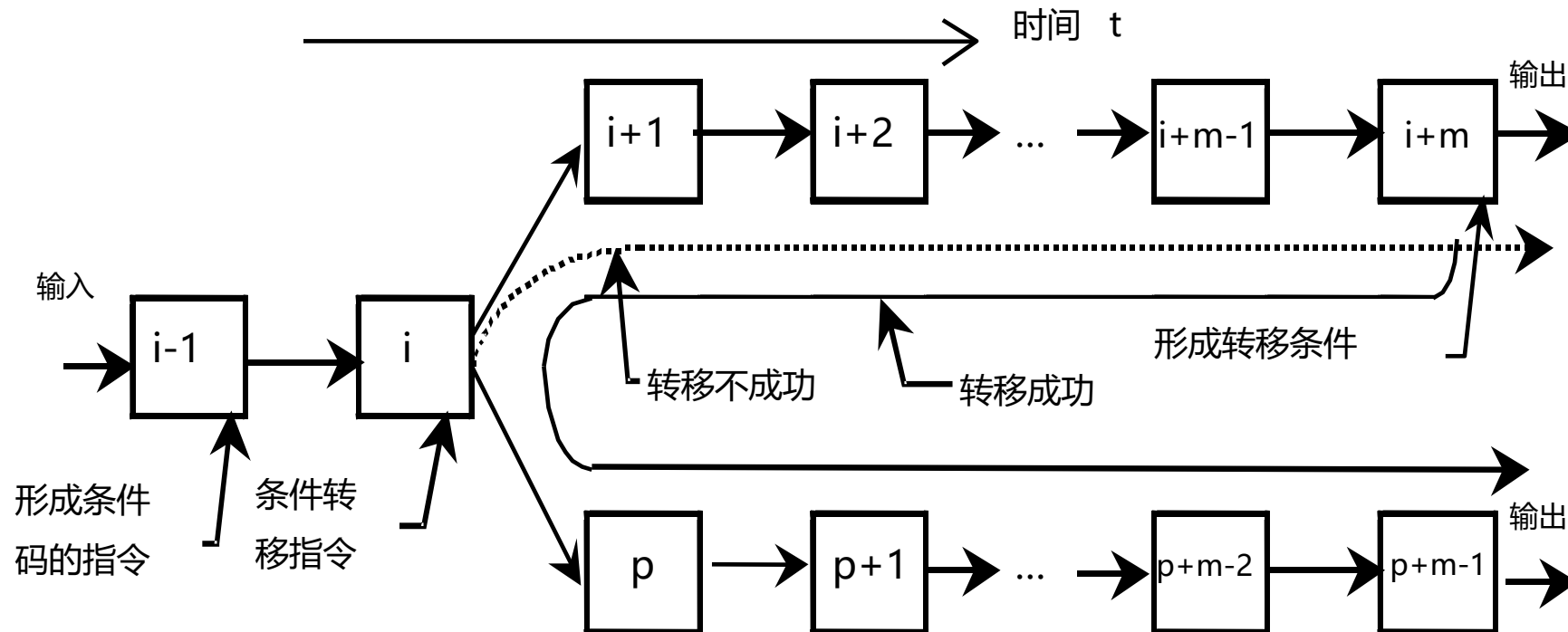
(b) 从目标处调度

(c) 从失败处调度

- 基本原理
- 预测的后续处理
 - ◆ 分支现场的保护与恢复
 - ◆ 预防预测不中时的加速处理
- 提高预测命中率
 - ◆ 静态预测
 - ◆ 动态预测

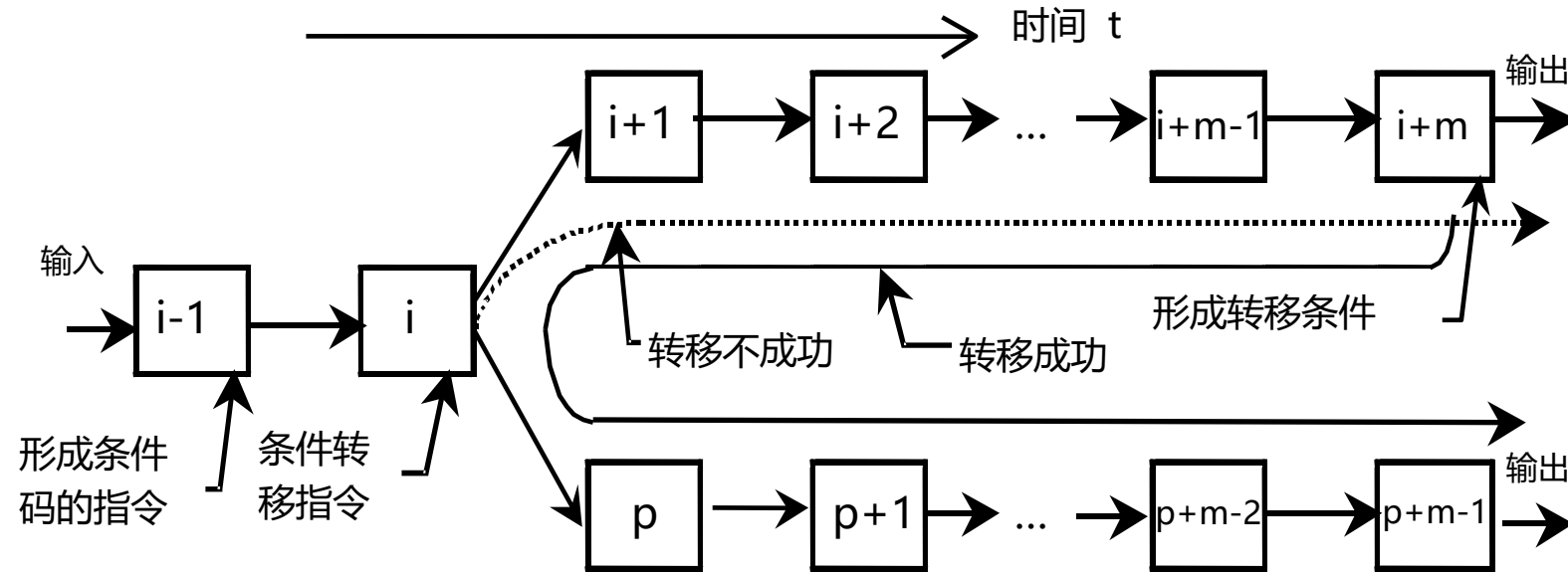
1. 分支预测的基本原理

选取发生概率较高的分支为预测方向，运行但不写回结果。若猜对，继续执行；否则，作废预测方向的执行，返回实际转移处。



5.4.2 分支预测

第 242 页



对于一条有 k 个流水段的流水线，在最坏情况下每一次条件转移（条件分支）指令将造成 m 个时钟的“停顿”；假设条件转移指令在一般程序中所占的比例为 p ，预测转移成功的概率为 q ，则 n 条指令的总执行时间是：

$$T_{K-IF} = (n + k - 1) \cdot \Delta t + p \cdot (1 - q) \cdot n \cdot m \cdot \Delta t$$

2. 分支现场的保护与恢复

■问题

- 当分支方向预测错误，不仅要浪费时间，更严重的还可能造成程序执行结果发生错误，因此要能恢复分支点处的原有现场。

指令	时钟								
	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
i+1		IF	ID	EX	MEM	WB			
i+2			IF	ID	EX	MEM	WB		
j				IF	ID	EX	MEM	WB	
j+1					IF	ID	EX	MEM	WB

指令	时钟								
	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
j		IF	ID	EX	MEM	WB			
j+1			IF	ID	EX	MEM	WB		
i+1				IF	ID	EX	MEM	WB	
i+2					IF	ID	EX	MEM	WB

5.4.2 分支预测

第 244 页

指令	时钟								
	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
i+1		IF	ID	EX	MEM	WB			
i+2			IF	ID	EX	MEM	WB		
j				IF	ID	EX	MEM	WB	
j+1					IF	ID	EX	MEM	WB

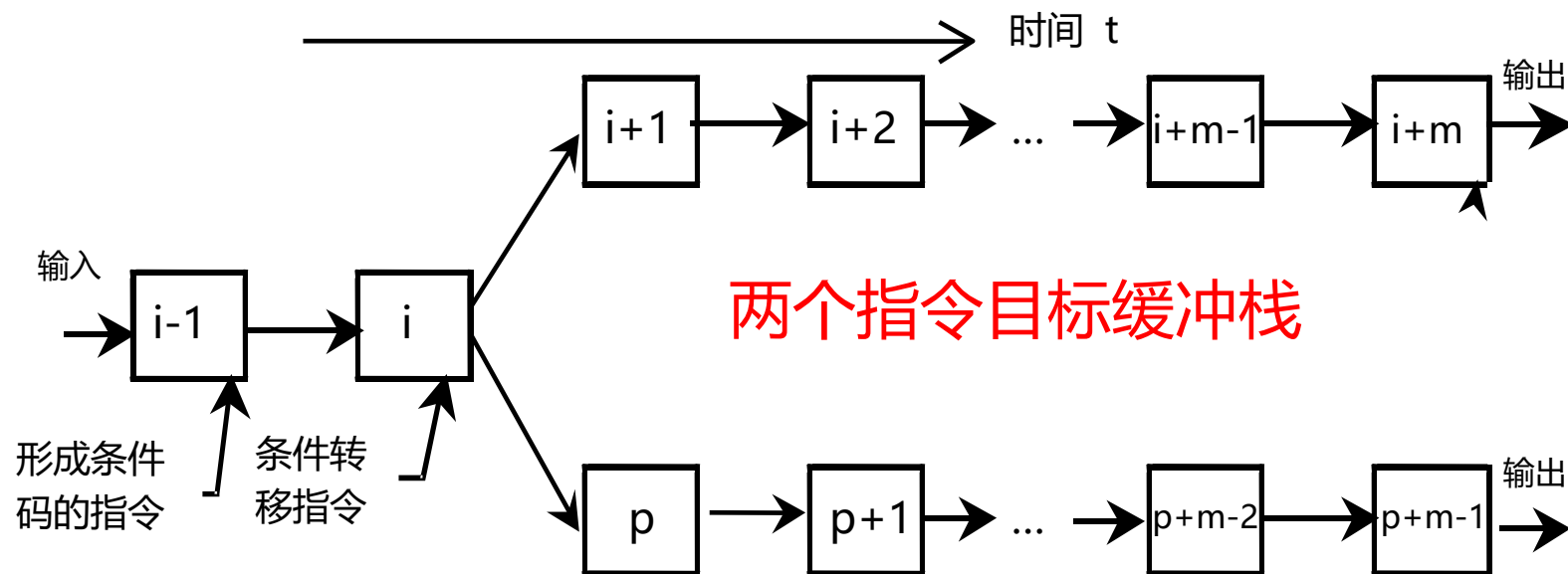
指令	时钟								
	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
j		IF	ID	EX	MEM	WB			
j+1			IF	ID	EX	MEM	WB		
i+1				IF	ID	EX	MEM	WB	
i+2					IF	ID	EX	MEM	WB

■ 解决

- 层次一：只进行指令译码和准备好运算所需的操作数，在转移条件没有形成之前不执行运算；
- 层次二：一直执行到运算完成，但不送回运算结果；
- 层次三：把可能被破坏的原始状态都用后援寄存器保存起来（此方法实现效率较高）。

3. 预防预测不中时的加速处理

可以设置两个**指令目标缓冲栈**。当指令分析器分析到条件转移指令时，一个指令目标缓冲栈按照转移成功方向预取指令，另一个指令目标缓冲栈按照转移不成功方向预取指令。不管转移是否成功都可以继续分析指令目标缓冲栈中的指令。

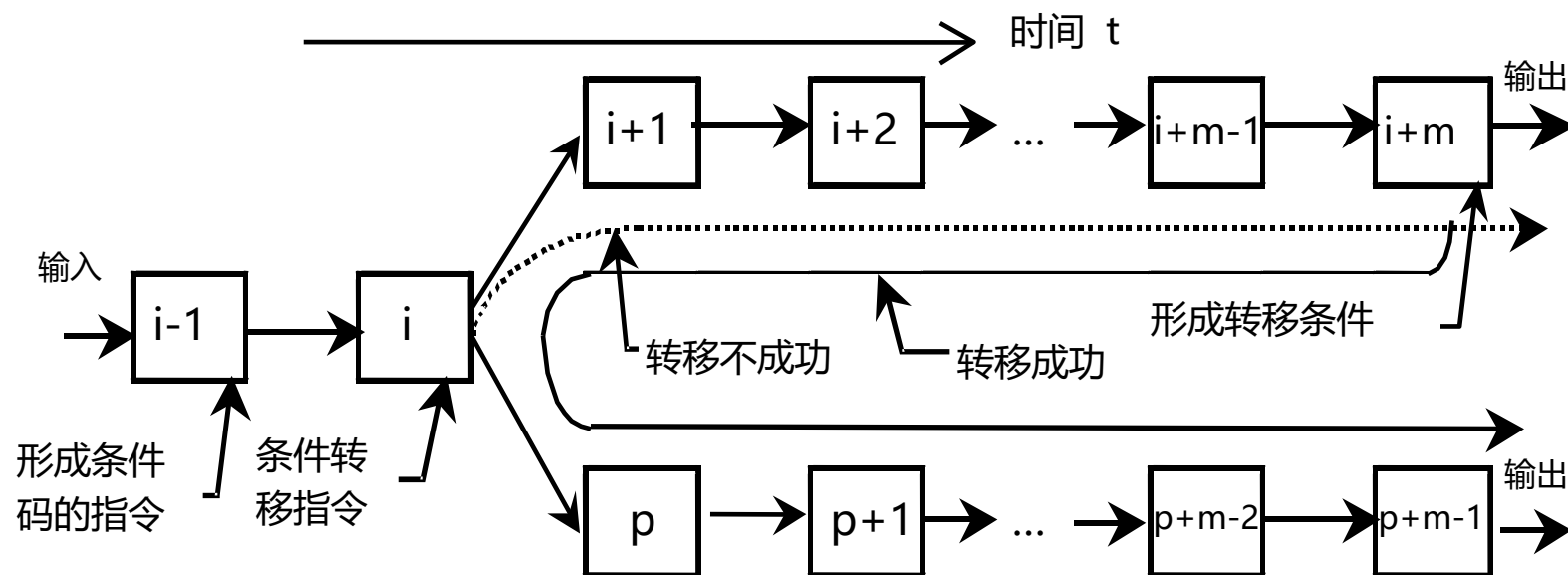


4. 提高预测命中率

◆静态预测

◆动态预测

$$T_{K-IF} = (n + k - 1) \cdot \Delta t + p \cdot (1 - q) \cdot n \cdot m \cdot \Delta t$$

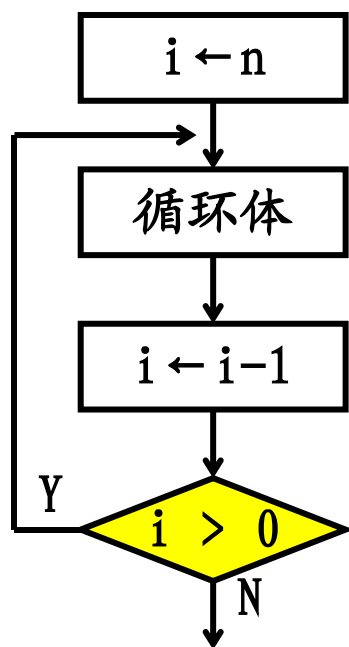


5.4.2 分支预测

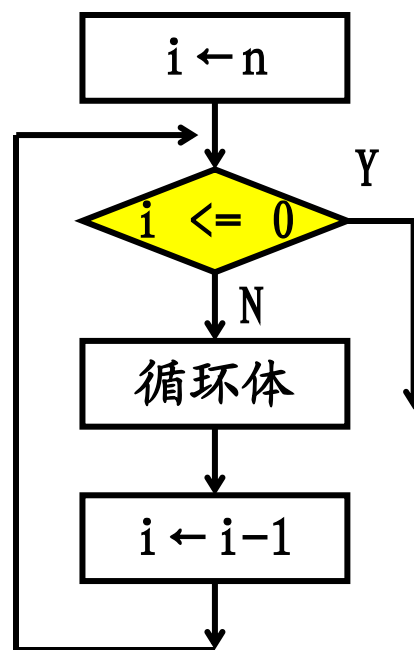
第 247 页

■ 静态预测

在程序实际执行过程中，转移猜测的方向不能改变。猜测不成功方向，由程序员和编译程序把发生概率高的分支安排在猜测方向。



①
编译前



②
编译后

■ 动态预测

➤ 转移预测缓冲器



➤ 相关转移预测器

➤ 转移目标缓冲器

■ 基本原理

■ 实现

➤ 1-bit转移预测缓冲器

➤ 2-bit转移预测缓冲器

➤ n-bit转移预测缓冲器

■ 性能

➤ 转移预测缓冲器的基本原理

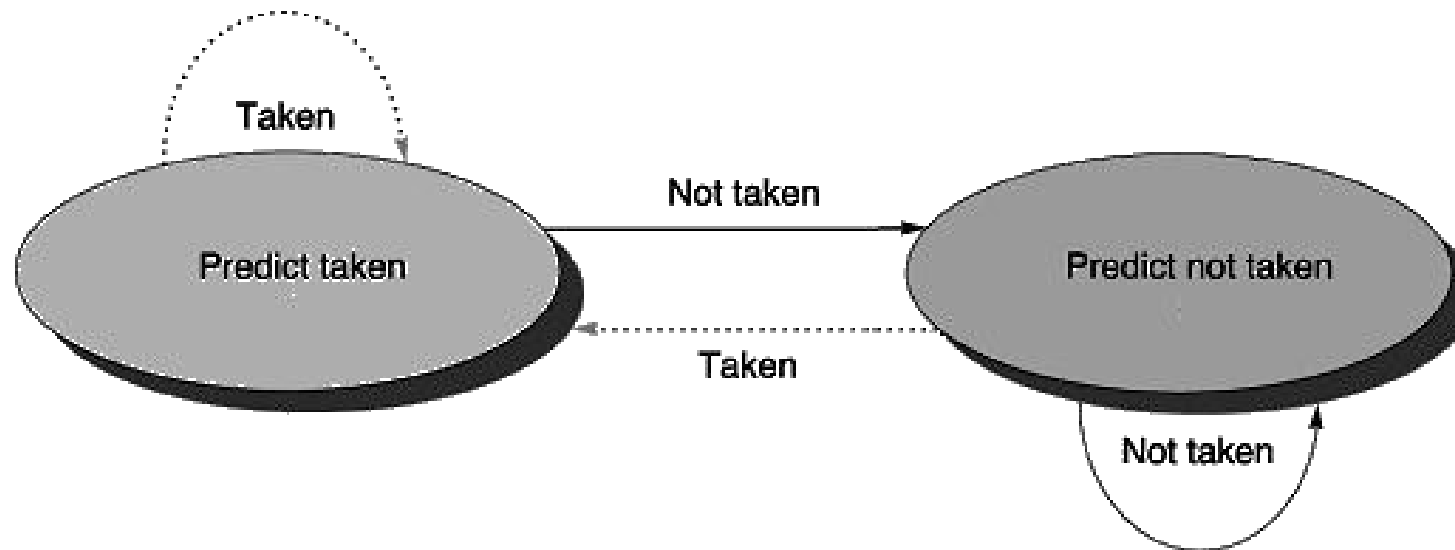
转移预测缓冲器也称为转移历史表（BHT: Branch History Table），预测只依赖本条件转移指令的历史信息（**局部信息**）。

具体实现为：设置一个较小的、按照条件转移指令的**低位地址部分**进行访问的存储器（每个表项对应一条条件转移指令）；在执行条件转移指令时，将转移成功或不成功的信息记录在该存储器中相应表项中；当下次再执行到该条件转移指令时，转移预测逻辑就根据记录的信息来预测是否转移。

➤ 1-bit转移预测缓冲器

在转移预测缓冲器中每个表项中只设置1bit，用以说明该条件转移指令是否曾经成功转移过。

该方法的有限状态机（状态转换图）为：



5.4.2 分支预测

第 251 页

➤ 1-bit转移预测缓冲器

	LOAD R1,NUM	;循环次数初值装入R1
LOOP:		;循环体开始
		
	DEC R1	;循环次数减 “1”
	BNE LOOP	;测试循环是否则结束
	HALT	;程序结束
NUM:	n	

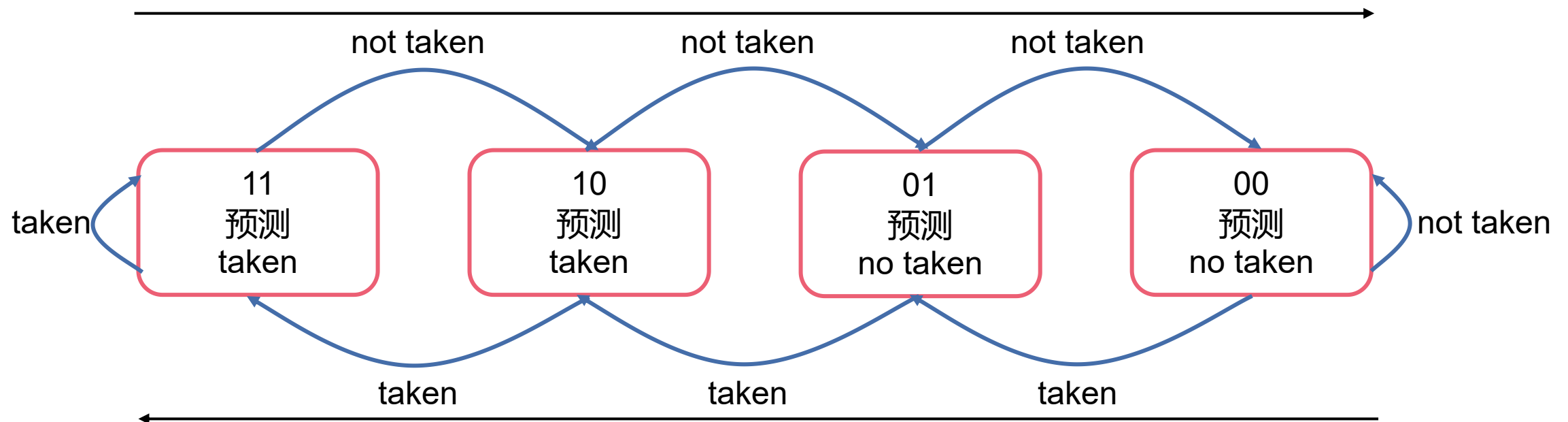
缺点：在1bit转移预测缓冲器中，即使条件转移指令大多数情况下总是转移成功（例如：循环），但也有可能错误地预测两次。例如，考察一个9次成功转移、1次不成功转移的循环程序。

5.4.2 分支预测

第 252 页

➤ 2-bit转移预测缓冲器

在转移预测缓冲器中每个表项中设置2bit，用以记录条件转移指令最近2次转移是否成功的信息。当预测不成功达到2次时才改变其预测方向。可以通过一个2bit饱和计数器来实现：当转移成功计数器加1，当转移不成功计数器减1。当计数器的值大于等于2（ 2^{2-1} ），则预测转移成功，否则预测转移不成功。



➤ 2-bit转移预测缓冲器

✓ 两位分支预测中的操作有**两个步骤**:

□ 分支预测;

- ◆ 当分支指令到达译码段 (**ID**) 时, 根据从**BHT**读出的信息进行分支预测。
- ◆ 若预测正确, 就继续处理后续的指令, 流水线没有断流。否则, 就要作废已经预取和分析的指令, 恢复现场, 并从另一条分支路径重新取指令。

□ 状态修改。

5.4.2 分支预测

第 254 页

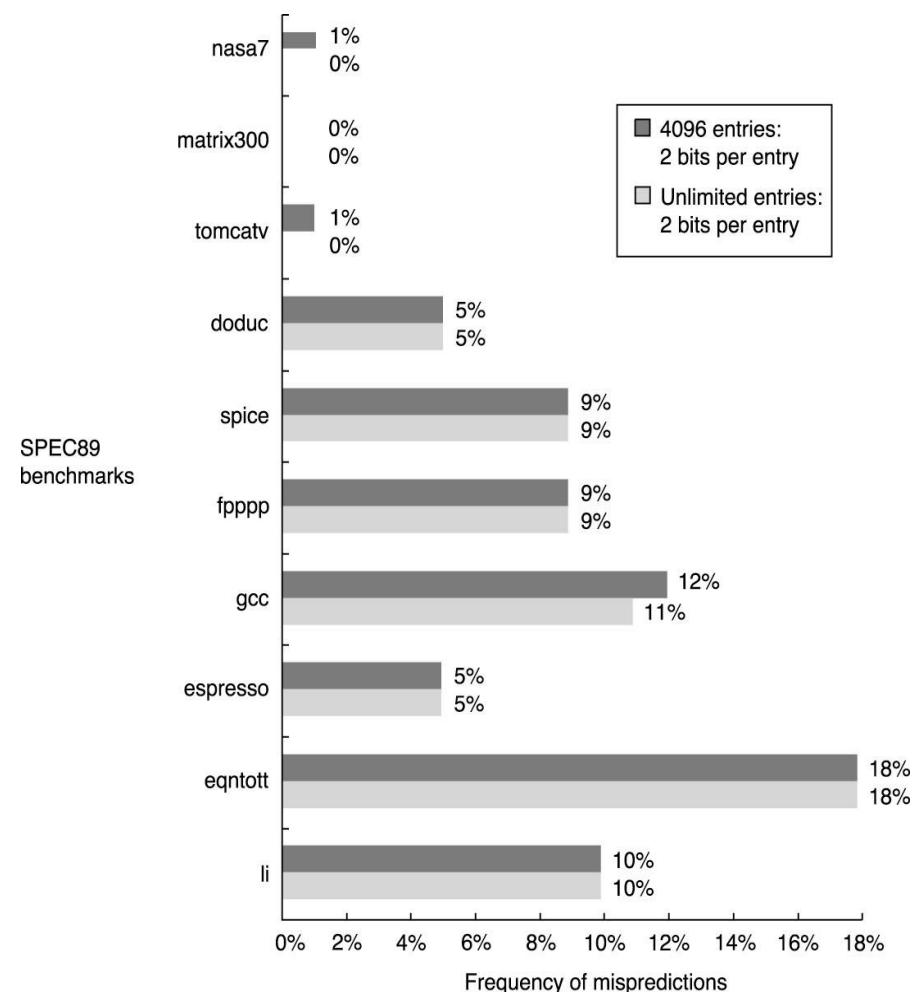
➤ n-bit转移预测缓冲器

在转移预测缓冲器中每个表项中设置n-bit预测位，用以记录最近多次转移是否成功的信息。

通过设置一个n-bit饱和计数器来实现：当转移成功计数器加1，当转移不成功计数器减1。当计数器的值大于等于 2^{n-1} ，则预测转移成功，否则预测转移不成功。

缺点：硬件开销大，控制复杂

在绝大多数系统中，往往采用2-bit转移预测缓冲器，而不采用n-bit转移预测缓冲器。



5.4.2 分支预测

第 255 页

4. BHT方法只在以下情况下才有用：

- 判定分支是否成功所需的时间大于确定分支目标地址所需的时间。
- 前述5段经典流水线：由于判定分支是否成功和计算分支目标地址都是在ID段完成，所以BHT方法不会给该流水线带来好处。

5. 研究结果表明：对于SPEC89测试程序来说，具有大小为4KB的BHT的预测准确率为82% ~ 99%。

- 一般来说，采用4KB的BHT就可以了。

6. BHT可以跟分支指令一起存放在指令Cache中，也可以用一块专门的硬件来实现。

■ 动态预测

➤ 转移预测缓冲器

➤ **相关转移预测器**

➤ 转移目标缓冲器

依靠已经发生过的其他条件转移指令的历史信息
(**全局信息**) 来进行预测，可以提高预测的精确度。

◆ C语言程序

```
if (d == 0)
    d = 1;
if (d == 1)
    {...}
```

◆ MIPS代码

```
BNEZ R1,L1
DADDIU R1,R0,#1
L1:DADDIU R3,R1,#-1
    BNEZ R3,L2
    ...
L2:
```

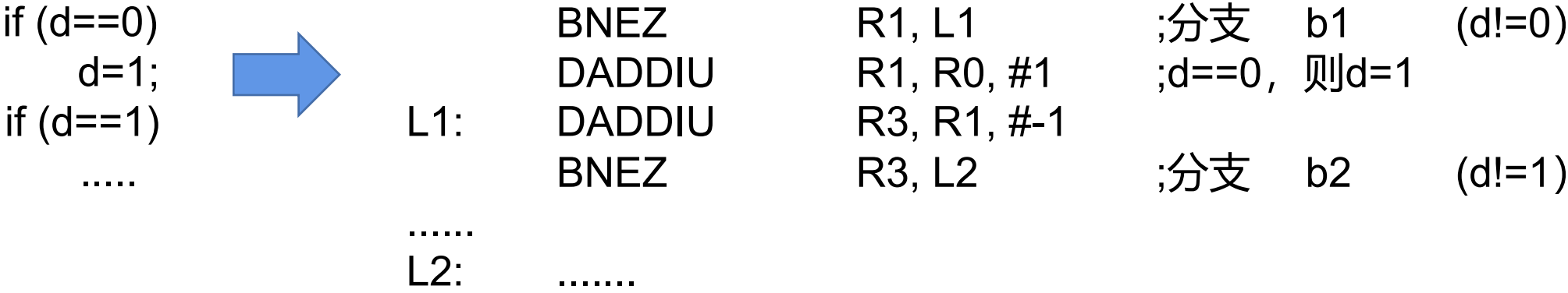
;分支b1(d!=0)
;d==0,so d=1

;分支b2(d!=1)

若b1不被执行, 则b2也不会被执行

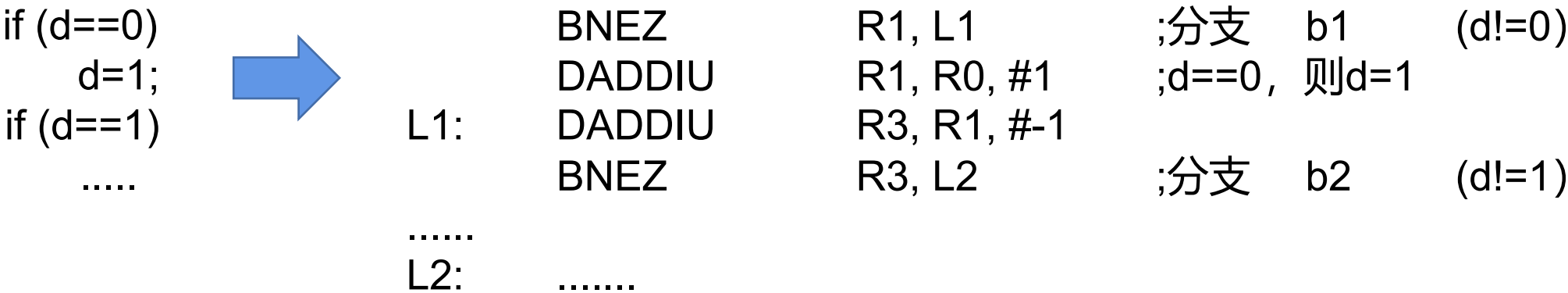
5.4.2 分支预测

■ 相关转移预指令：有两条转移指令，后一条转移指令的转移行为与前面那条转移指令的转移行为相关，那么这两条转移指令叫相关转移指令



5.4.2 分支预测

■ 相关转移预指令：有两条转移指令，后一条转移指令的转移行为与前面那条转移指令的转移行为相关，那么这两条转移指令叫相关转移指令



假设d的取值可为0、1和2，R1寄存器中的值为d，则这段代码的执行顺序如下：

d的初始值	d==0?	b1	执行b2前d的值	d==1?	b2
0	是	执行	1	是	执行
1	否	未执行	1	是	执行
2	否	未执行	2	否	未执行

相关转移组合（3种）：1、b1不被执行，b2也不会被执行；2、b1被执行，但b2不会被执行；3、b1被执行，b2也被执行

5.4.2 分支预测

第 259 页

➤ (1, 1) 预测器

根据前一次其它条件转移指令的行为去从一对1bit预测器中进行选择。即：每个条件转移指令需设置一对1bit预测器，其中1bit用于前一次其它条件转移指令未成功转移时的预测，另1bit用于前一次其它条件转移指令成功转移时的预测。

预测位1bit/1bit	未成功转移分支	成功转移分支
NT/NT	NT	NT
NT/T	NT	T
T/NT	T	NT
T/T	T	T

5.4.2 分支预测

➤ (1, 1) 预测器

	d=?	对b1的预测	b1的实际动作	对b1的新预测	对b2的预测	b2的实际动作	对b2的新预测
1bit转移预测器 (全部失败)	0	NT	T	T	NT	T	T
	2	T	NT	NT	T	NT	NT
	0	NT	T	T	NT	T	T
	2	T	NT	NT	T	NT	NT
(1,1) 预测器 (仅1次失败)	0	NT/NT	T	T/NT	NT/NT	T	NT/T
	2	T/NT	NT	T/NT	NT/T	NT	NT/T
	0	T/NT	T	T/NT	NT/T	T	NT/T
	2	T/NT	NT	T/NT	NT/T	NT	NT/T

➤ (m, n) 预测器

根据前 m 次其它条件转移指令的行为去从 2^m 个 n -bit预测器中进行选择。即：每个条件转移指令需设置 2^m 个 n -bit预测器，分别用于前 m 次其它条件转移指令的 2^m 个行为的预测。

具体硬件实现为：最近 m 个条件转移指令的全局历史记录可以记录在一个 m -bit移位寄存器中，每1-bit记录着该条件转移指令是否转移。对相关转移预测缓冲器的访问可由条件转移指令的低位拼接上 m -bit全局历史记录而得到。

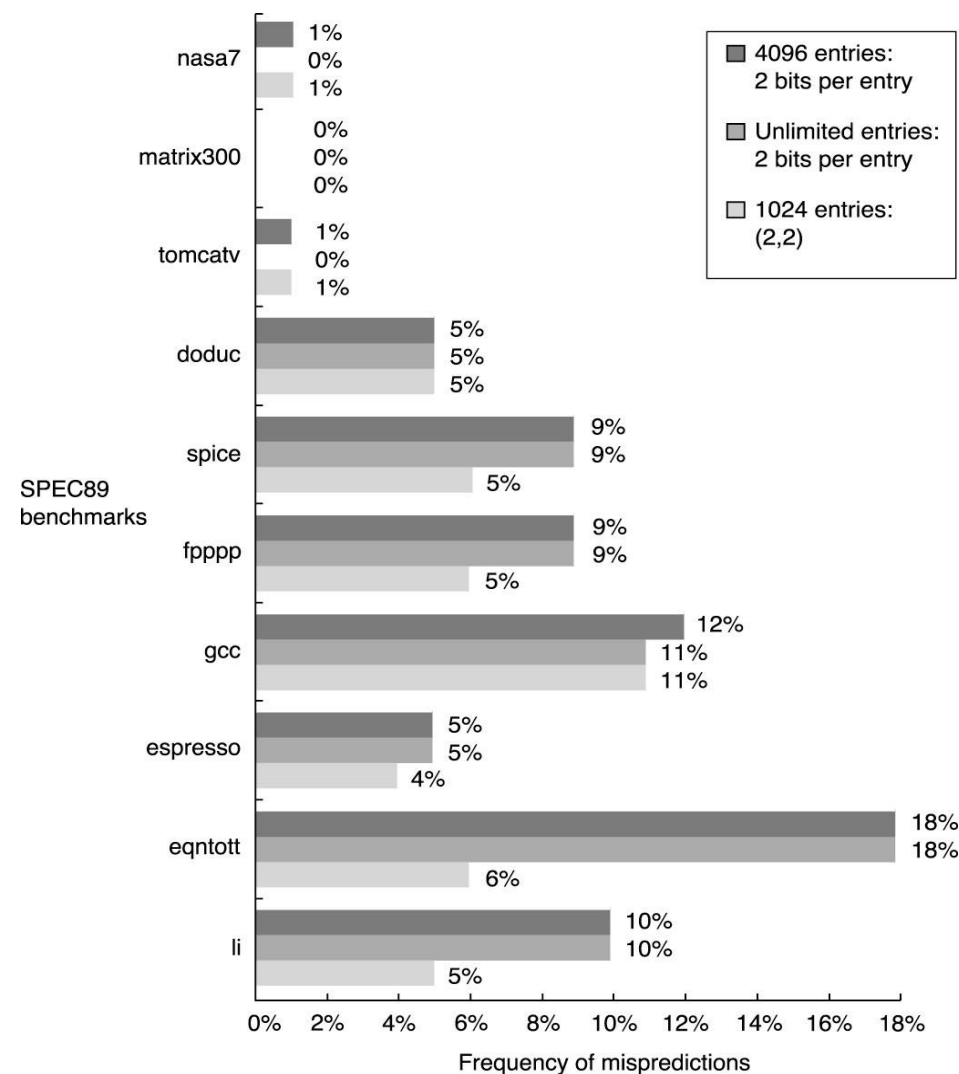
5.4.2 分支预测

第 262 页

➤ (m, n) 预测器

提示： 2bit转移预测缓冲器也可称为 (0, 2) 预测器。 (0, 2) 预测器和 (2, 2) 预测器最受欢迎。

相关转移预测
缓冲器的性能



5.4.3 采用分支目标缓冲器BTB

第 263 页

目标：将分支的开销降为0（取指令阶段是否能够判断是否转移？）

方法：分支目标缓冲

- 将分支成功的分支指令的地址和它的分支目标地址都放到一个缓冲区中保存起来，缓冲区以分支指令的地址作为标识。
- 这个缓冲区就是分支目标缓冲器（Branch-Target Buffer，简记为BTB），或者分支目标Cache（Branch-Target Cache，简记为BTC）。

指令	时钟								
	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
i+1		IF	ID	EX	MEM	WB			
i+2			IF	ID	EX	MEM	WB		



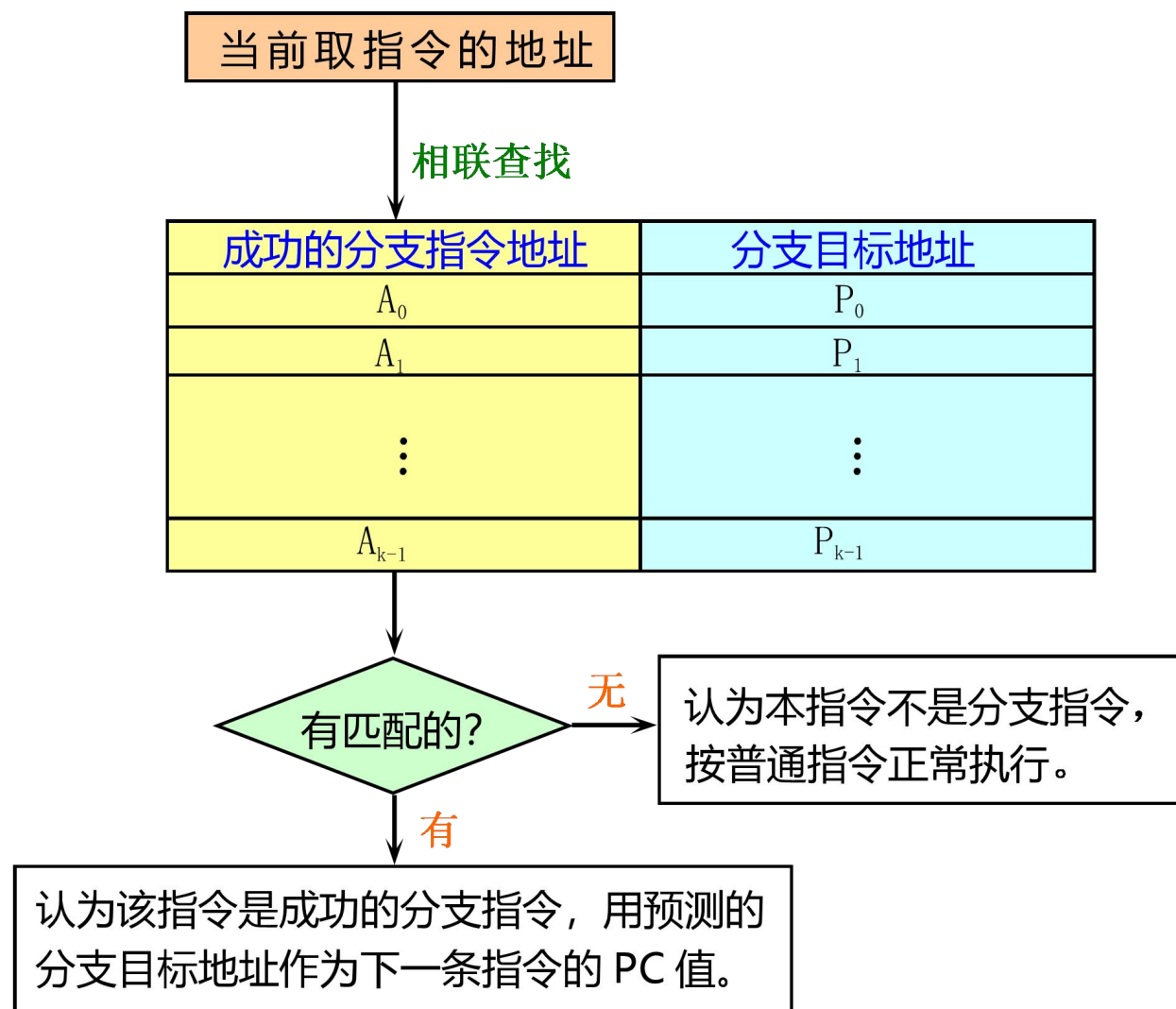
优化掉

5.4.3 采用分支目标缓冲器BTB

第 264 页

1. BTB的结构

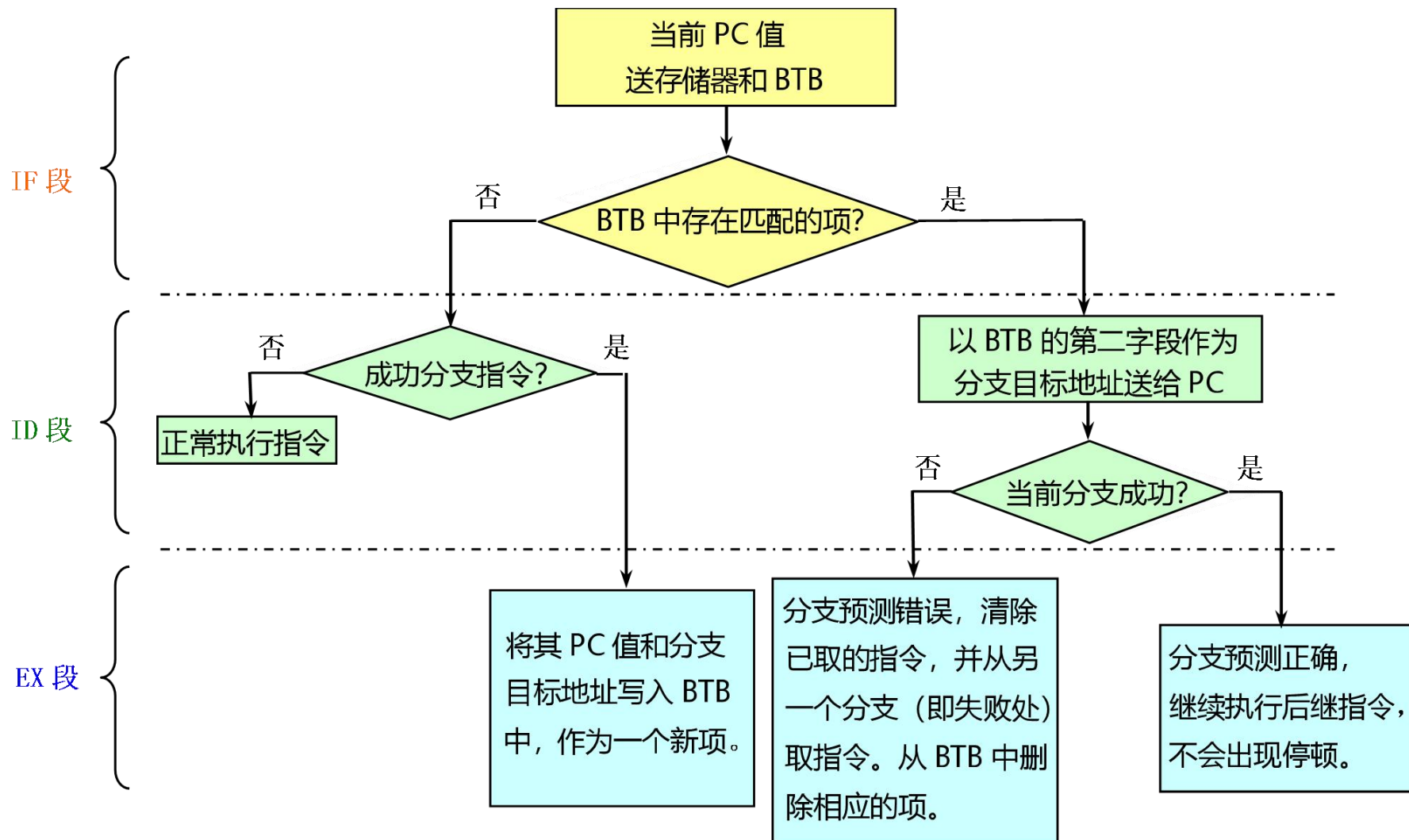
- 看成是用专门的硬件实现的一张表格。
- 表格中的每一项至少有两个字段：
 - 执行过的成功分支指令的地址；
(作为该表的匹配标识)
 - 预测的分支目标地址。



5.4.3 采用分支目标缓冲器BTB

第 265 页

2. 采用BTB后，在流水线各个阶段所进行的相关操作：



5.4.3 采用分支目标缓冲器BTB

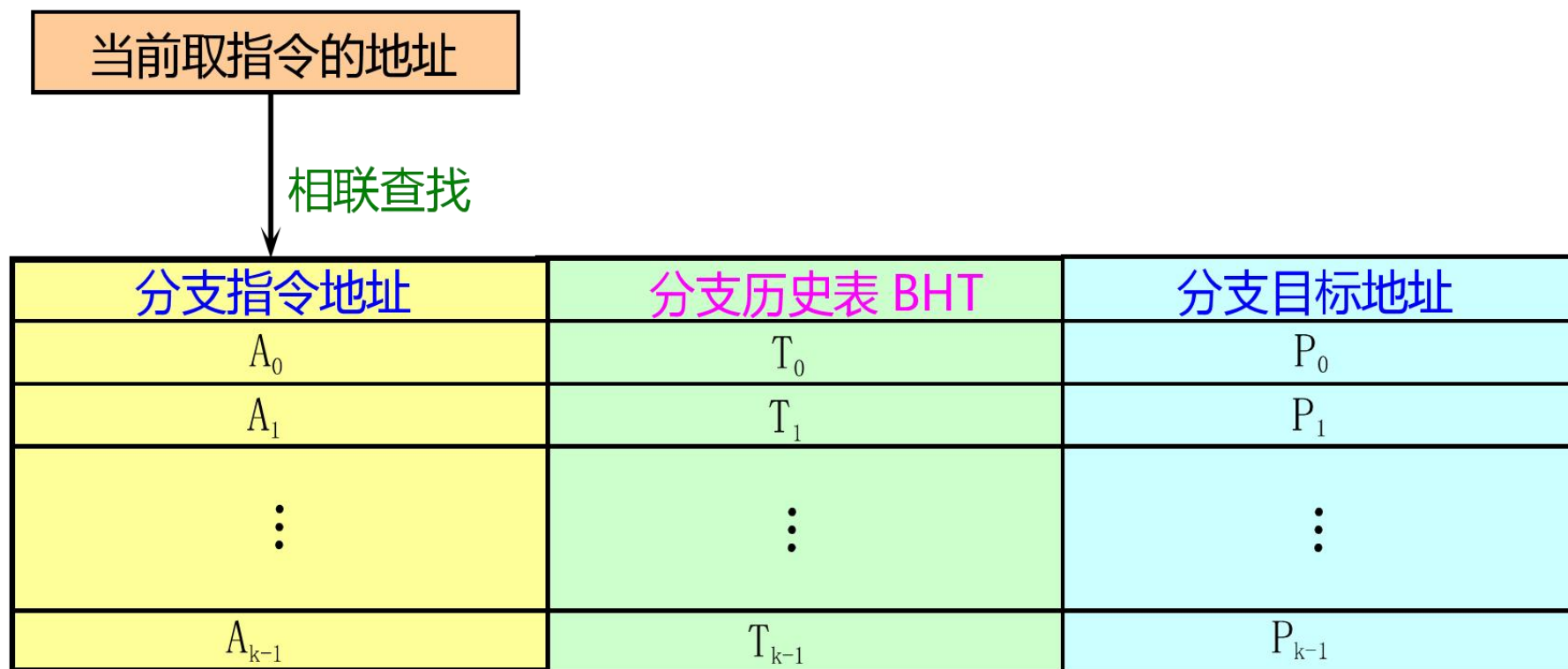
3. 采用BTB后，各种可能情况下的延迟：

指令在BTB中?	预测	实际情况	延迟周期
是	成功	成功	0
是	成功	不成功	2
不是		成功	2
不是		不成功	0

5.4.3 采用分支目标缓冲器BTB

第 267 页

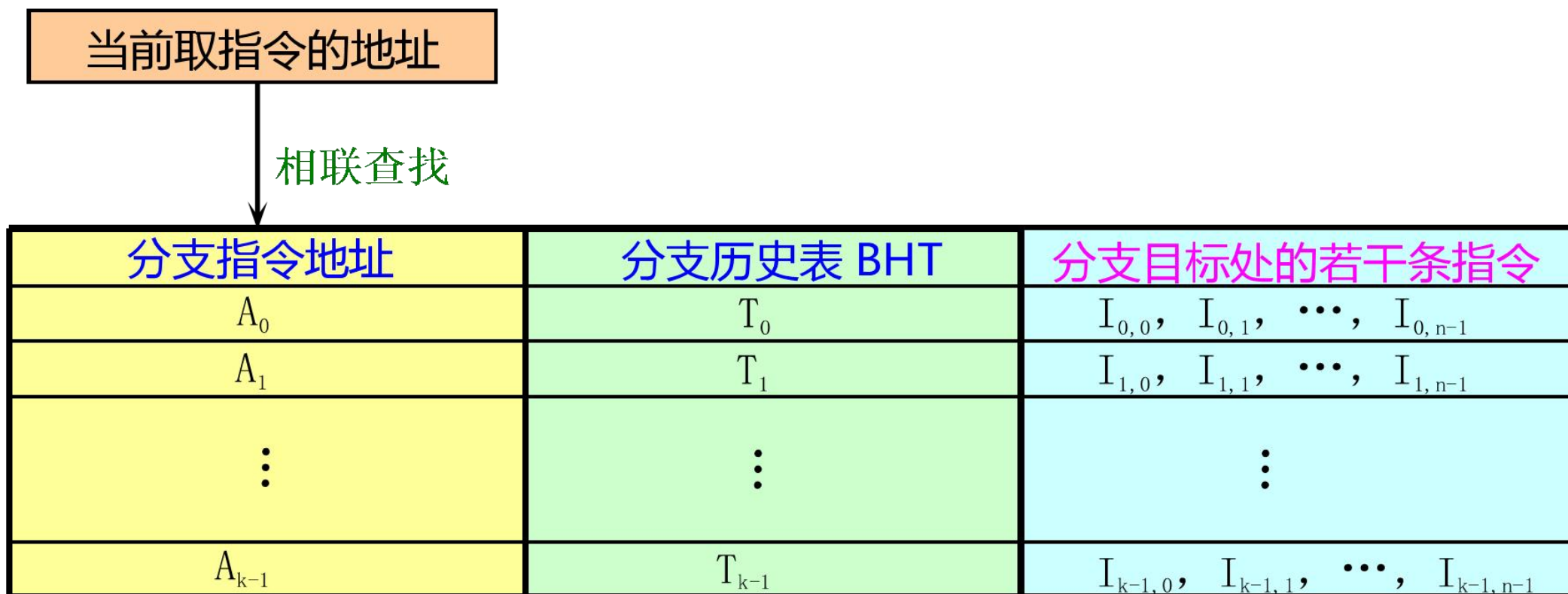
4. BTB的另一种形式：在分支目标缓冲器中增设一个至少是两位的“分支历史表”字段



5.4.3 采用分支目标缓冲器BTB

第 268 页

5. 更进一步，在表中对于每条分支指令都存放若干条分支目标处的指令，就形成了分支目标指令缓冲器。



前瞻执行（speculation）的基本思想：

对分支指令的结果进行猜测，并假设这个猜测总是对的，然后按这个猜测结果继续取、流出和执行后续的指令。只是执行指令的结果不是写回到寄存器或存储器，而是写入一个称为再定序缓冲器ROB（ReOrder Buffer）中。等到相应的指令得到“确认”（commit）（即确实是应该执行的）之后，才将结果写入寄存器或存储器。

5.4.4 基于硬件的前瞻执行

第 270 页

1. 基于硬件的前瞻执行结合了3种思想：

- 动态分支预测。用来选择后续执行的指令。
- 在控制相关的结果尚未出来之前，前瞻地执行后续指令。
- 用动态调度对基本块的各种组合进行跨基本块的调度。

2. 对Tomasulo算法加以扩充，就可以支持前瞻执行。

- 区分Tomasulo算法的写结果和指令完成，分成两个不同的段：写结果，指令确认

■ 写结果段

- 把前瞻执行的结果写到ROB中；
- 通过CDB在指令之间传送结果，供需要用到这些结果的指令使用。

■ 指令确认段：在分支指令的结果出来后，对相应指令的前瞻执行给予确认。

- 如果前面所做的猜测是对的，把在ROB中的结果写到寄存器或存储器。
- 如果发现前面对分支结果的猜测是错误的，那就不予以确认，并从那条分支指令的另一条路径开始重新执行。

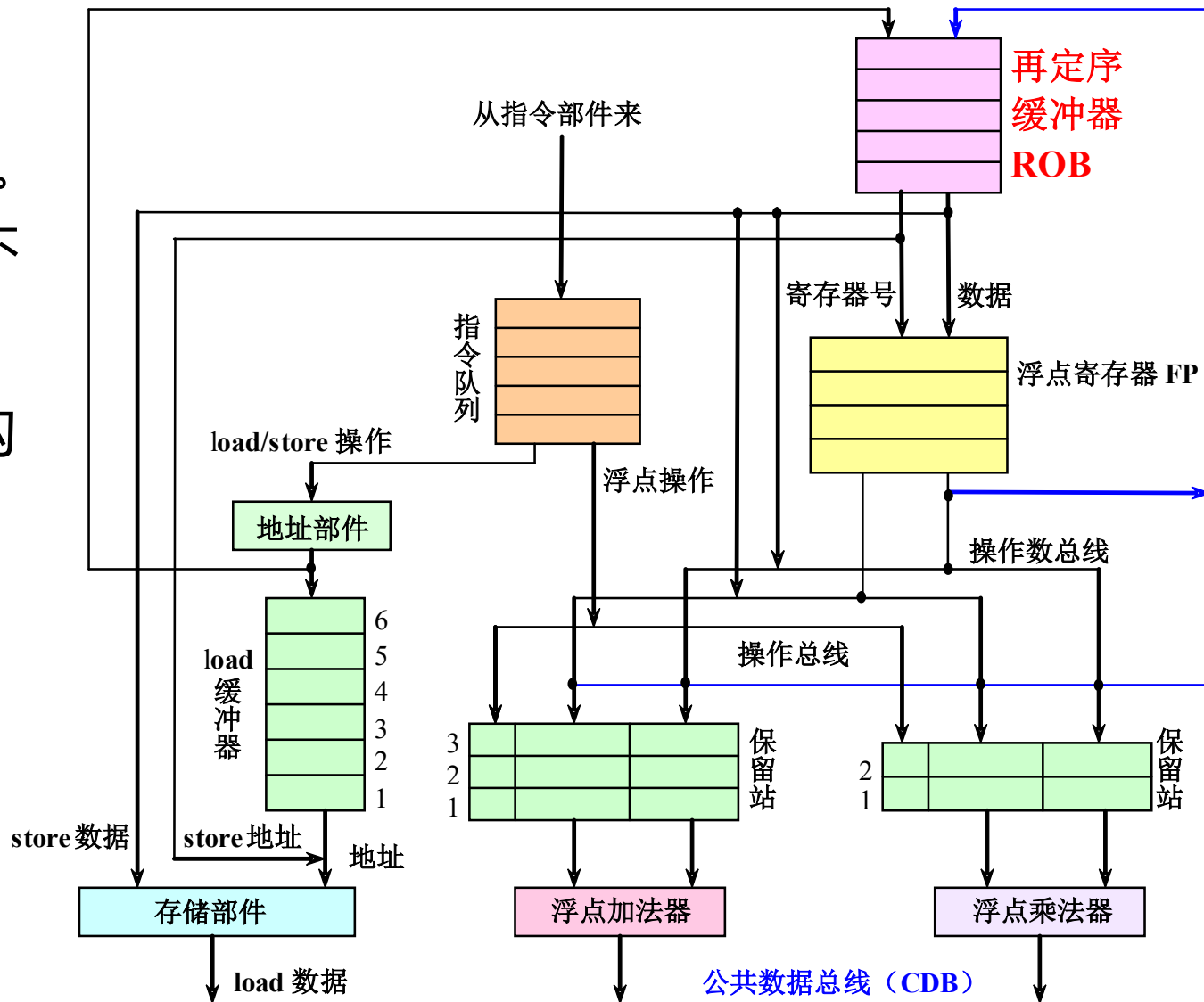
5.4.4 基于硬件的前瞻执行

第 271 页

3. 实现前瞻的关键思想：

允许指令乱序执行，但必须顺序确认。
在指令被确认之前，不允许它进行不可恢复的操作。

4. 支持前瞻执行的浮点部件的结构



5.4.4 基于硬件的前瞻执行

第 272 页

■ ROB中的每一项由以下4个字段组成：

➤指令类型

指出该指令是分支指令、store指令或寄存器操作指令。

➤目标地址

给出指令执行结果应写入的目标寄存器号（如果是load和ALU指令）或存储器单元的地址（如果是store指令）。

➤数据值字段

用来保存指令前瞻执行的结果，直到指令得到确认。

➤就绪字段

指出指令是否已经完成执行并且数据已就绪。

■ Tomasulo算法中保留站的换名功能是由ROB来完成的。

5.4.4 基于硬件的前瞻执行

第 273 页

5. 采用前瞻执行机制后，指令的执行步骤：（在Tomasulo算法的基础上改造的）

➤ 流出

- ✓从浮点指令队列的头部取一条指令。
- ✓如果有空闲的保留站（设为 r ）且有空闲的ROB项（设为 b ），就流出该指令，并把相应的信息放入保留站 r 和ROB项 b 。
- ✓如果保留站或ROB全满，便停止流出指令，直到它们都有空闲的项。

5.4.4 基于硬件的前瞻执行

第 274 页

5. 采用前瞻执行机制后，指令的执行步骤：（在Tomasulo算法的基础上改造的）

➤ 执行

✓如果有操作数尚未就绪，就等待，并不断地监测CDB。

（检测RAW冲突）

✓当两个操作数都已在保留站中就绪后，就可以执行该指令的操作。

5.4.4 基于硬件的前瞻执行

第 275 页

5. 采用前瞻执行机制后，指令的执行步骤：（在Tomasulo算法的基础上改造的）

➤ 写结果

- ✓当结果产生后，将该结果连同本指令在流出段所分配到的ROB项的编号放到CDB上，经CDB写到ROB以及所有等待该结果的保留站。
- ✓释放产生该结果的保留站。
- ✓store指令在本阶段完成，其操作为：
 - 如果要写入存储器的数据已经就绪，就把该数据写入分配给该store指令的ROB项。
 - 否则，就监测CDB，直到那个数据在CDB上播送出来，才将之写入分配给该store指令的ROB项。

5.4.4 基于硬件的前瞻执行

第 276 页

5. 采用前瞻执行机制后，指令的执行步骤：（在Tomasulo算法的基础上改造的）

- 确认：对分支指令、store指令以及其它指令的处理不同：
 - ✓其它指令（除分支指令和store指令）：当该指令到达ROB队列的头部而且其结果已经就绪时，就把该结果写入该指令的目的寄存器，并从ROB中删除该指令。
 - ✓store指令：处理与上面的类似，只是它把结果写入存储器。
 - ✓分支指令：
 - 当预测错误的分支指令到达ROB队列的头部时，清空ROB，并从分支指令的另一个分支重新开始执行。（错误的前瞻执行）
 - 当预测正确的分支指令到达ROB队列的头部时，该指令执行完毕。

5.4.4 基于硬件的前瞻执行

第 277 页

例5.5 假设浮点功能部件的延迟时间为：加法2个时钟周期，乘法10个时钟周期，除法40个时钟周期。对于下面的代码段，给出当指令MUL.D即将确认时的状态表内容。

L.D	F6,34 (R2)
-----	------------

L.D	F2,45 (R3)
-----	------------

MUL.D	F0,F2,F4
-------	----------

SUB.D	F8,F6,F2
-------	----------

DIV.D	F10,F0,F6
-------	-----------

ADD.D	F6,F8,F2
-------	----------

5.4.4 基于硬件的前瞻执行

前瞻执行中MUL.D确认前，保留站和ROB的状态

名称	保留站							
	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	no							
Load2	no							
Add1	no							
Add2	no							
Add3	no							
Mult1	no	MUL.D	Mem[45+ Regs[R3]]	Regs[F4]			#3	
Mult2	yes	DIV.D		Mem[34+Regs[R2]]	#3		#5	

5.4.4 基于硬件的前瞻执行

第 279 页

项号	ROB				
	Busy	指令	状态	目的	Value
1	no	L.D F6, 34 (R2)	确认	F6	Mem[34+Regs[R2]]
2	no	L.D F2, 45 (R3)	确认	F2	Mem[45+Regs[R3]]
3	yes	MUL.D F0, F2, F4	写结果	F0	#2×Regs[F4]
4	yes	SUB.D F8, F6, F2	写结果	F8	#1 - #2
5	yes	DIV.D F10, F0, F6	执行	F10	
6	yes	ADD.D F6,F8,F2	写结果	F6	#4 + #2

字段	浮点寄存器状态							
	F0	F2	F4	F6	F8	F10	...	F30
ROB项编号	3			6	4	5		
Busy	yes	no	no	yes	yes	yes	...	no

6. 前瞻执行

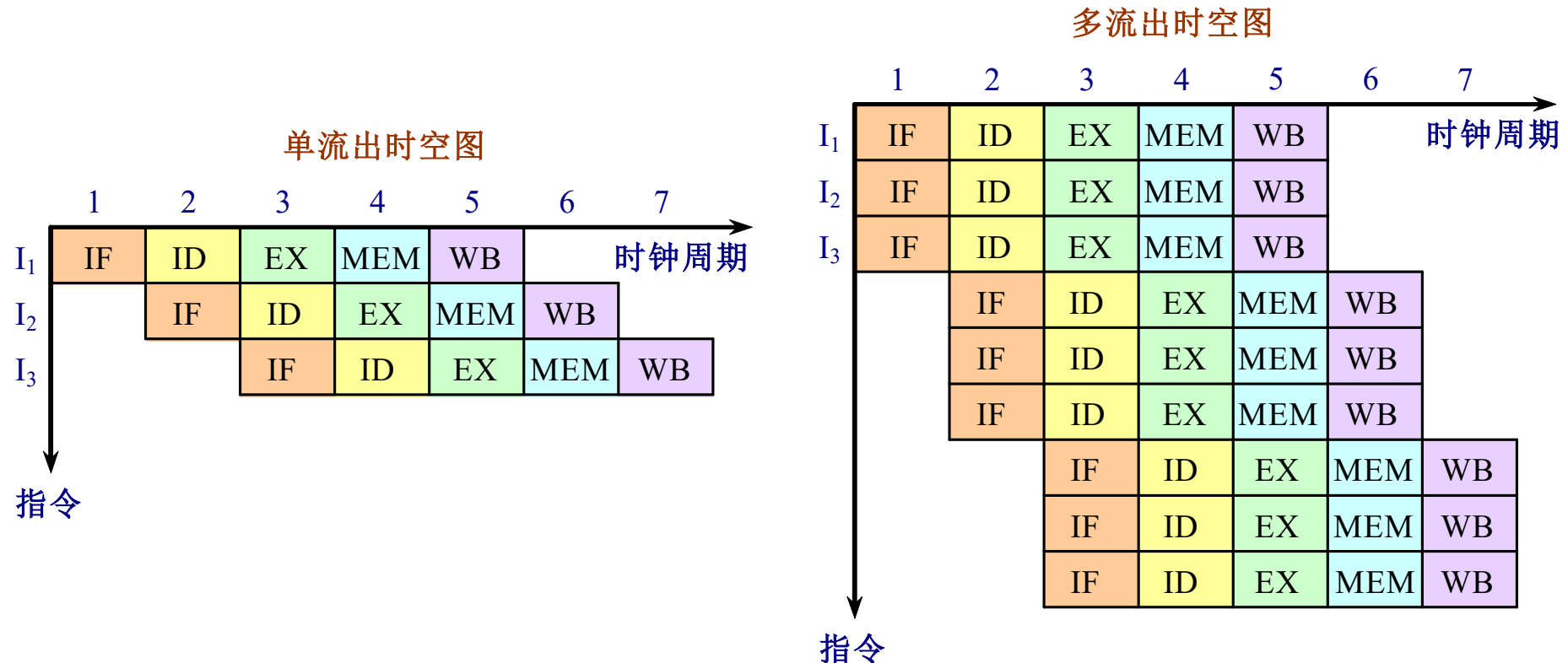
- 通过ROB实现了指令的顺序完成。
- 能够实现精确异常。
- 很容易地推广到整数寄存器和整数功能单元上。
- 主要缺点：所需的硬件太复杂。

5.5 多指令流出技术

5.5 多指令流出技术

第 282 页

- 在每个时钟周期内流出多条指令， $CPI < 1$ 。
- 单流出和多流出处理机执行指令的时空图对比



1. 多流出处理机有两种基本风格：

■ 超标量 (Superscalar)

- 在每个时钟周期流出的指令条数不固定，依代码的具体情况而定。（有个上限）
- 设这个上限为 n ，就称该处理机为 n -流出。
- 可以通过编译器进行静态调度，也可以基于Tomasulo算法进行动态调度。

■ 超长指令字VLIW (Very Long Instruction Word)

- 在每个时钟周期流出的指令条数是固定的，这些指令构成一条长指令或者一个指令包。
- 指令包中，指令之间的并行性是通过指令显式地表示出来的。
- 指令调度是由编译器静态完成的。

2. 超标量处理机与VLIW处理机相比有两个优点：

- 超标量结构对程序员是透明的，处理机能自己检测下一条指令能否流出，不需要由编译器或专门的变换程序对程序中的指令进行重新排列；
- 即使是没有经过编译器针对超标量结构进行调度优化的代码或是旧的编译器生成的代码也可以运行，当然运行的效果不会很好。
 - 要想达到很好的效果，方法之一：使用动态超标量调度技术。

5.5 多指令流出技术

第 285 页

3. 下表列出了一些基本的多流出技术、这些技术的特点以及采用这些技术的处理机实例。

技 术	流出结构	冲突检测	调 度	主要特点	处理机实例
超标量 (静态)	动态	硬件	静态	按序执行	Sun UltraSPARC II /III
超标量 (动态)	动态	硬件	动态	部分乱序执行	IBM Power2
超标量 (猜测)	动态	硬件	带有前瞻的动态调度	带有前瞻的乱序执行	Pentium III/4, MIPS R10K, Alpha 21264, HP PA 8500, IBM RS64III
VLIW/LIW	静态	软件	静态	流出包之间没有冲突	Trimedia, i860
EPIC	主要是静态	主要是软件	主要是静态	相关性被编译器显式地标记出来	Itanium

5.5.1 基于静态调度的多流出技术

第 286 页

- 在典型的超标量处理器中，每个时钟周期可流出1到8条指令。
- 指令按序流出，在流出时进行冲突检测。
 - 由硬件检测当前流出的指令之间是否存在冲突以及当前流出的指令与正在执行的指令是否有冲突。

举例：一个4-流出的静态调度超标量处理机

- 在取指令阶段，流水线将从取指令部件收到1 ~ 4条指令（称为流出包）。
 - 在一个时钟周期内，这些指令有可能是全部都能流出，也可能是只有一部分能流出。
- 流出部件检测结构冲突或者数据冲突，一般分两阶段实现：
 - 第一段：进行流出包内的冲突检测，选出初步判定可以流出的指令；
 - 第二段：检测所选出的指令与正在执行的指令是否有冲突。

5.5.1 基于静态调度的多流出技术

第 287 页

MIPS处理机是怎样实现超标量的呢？

假设：每个时钟周期流出两条指令：1条整数型指令 + 1条浮点操作指令，其中：把load指令、store指令、分支指令归类为整数型指令。

1. 要求：

- 同时取两条指令（64位），译码两条指令（64位）。

2. 对指令的处理包括以下步骤：

- 从Cache中取两条指令；
- 确定那几条指令可以流出（0 ~ 2条指令）；
- 把它们发送到相应的功能部件。

5.5.1 基于静态调度的多流出技术

3. 双流出超标量流水线中指令执行的时空图

- 假设：所有的浮点指令都是加法指令，其执行时间为两个时钟周期。为简单起见，图中总是把整数指令放在浮点指令的前面。

指令类型	流水线工作情况							
整数指令	IF	ID	EX	MEM	WB			
浮点指令	IF	ID	EX	EX	MEM	WB		
整数指令		IF	ID	EX	MEM	WB		
浮点指令		IF	ID	EX	EX	MEM	WB	
整数指令			IF	ID	EX	MEM	WB	
浮点指令			IF	ID	EX	EX	MEM	WB
整数指令				IF	ID	EX	MEM	WB
浮点指令				IF	ID	EX	EX	MEM

5.5.1 基于静态调度的多流出技术

第 289 页

4. 采用 “1条整数型指令 + 1条浮点指令” 并行流出的方式，需要增加的硬件很少。
5. 浮点load或浮点store指令将使用整数部件，会增加对浮点寄存器的访问冲突。
 - 增设一个浮点寄存器的读/写端口。
6. 由于流水线中的指令多了一倍，定向路径也要增加。
7. 限制超标量流水线的性能发挥的障碍
 - load指令：load后续3条指令都不能使用其结果，否则就会引起停顿。
 - 分支延迟
 - 如果分支指令是流出包中的第一条指令，则其延迟是3个时钟周期；
 - 否则就是流出包中的第二条指令，其延迟就是两个时钟周期。

5.5.2 基于动态调度的多流出技术

第 290 页

■ 扩展Tomasulo算法：支持双流出超标量流水线

➤ 每个时钟周期流出两条指令：一条是整数指令，另一条是浮点指令。

1. 采用一种比较简单的方法：

➤ 指令按顺序流向保留站，否则会破坏程序语义。

➤ 将整数所用的表结构与浮点用的表结构分离开，分别进行处理，这样就可以同时地流出一条浮点指令和一条整数指令到各自的保留站。

2. 有两种不同的方法可以实现多流出，关键在于：对保留站的分配和对流水线控制表格的修改。

➤ 在半个时钟周期里完成流出步骤，这样一个时钟周期就能处理两条指令。

➤ 设置一次能同时处理两条指令的逻辑电路，现代的流出4条或4条以上指令的超标量处理机经常是两种方法都采用。

5.5.2 基于动态调度的多流出技术

第 291 页

例5.5 对于采用了Tomasulo算法和多流出技术的MIPS流水线，考虑以下简单循环的执行。该程序把F2中的标量加到一个向量的每个元素上。

```
Loop:  L.D      F0, 0 (R1)    // 取一个数组元素放入F0
        ADD.D   F4, F0, F2    // 加上在F2中的标量
        S.D     F4, 0 (R1)    // 存结果
        DADDIU  R1,R1, #-8    // 将指针减少8（每个数据占8个字节）
        BNE     R1,R2,Loop    // 若R1不等于R2，表示尚未结束，转移到Loop继续。
```

5.5.2 基于动态调度的多流出技术

第 292 页

现做以下假设：

1. 每个时钟周期能流出一条整数指令和一条浮点指令，即使它们相关也是如此。
2. 有一个整数部件，用于整数ALU运算和地址计算；并且对于每一种浮点操作类型都有一个独立的流水化了的浮点功能部件。
3. 指令流出和写结果各占用一个时钟周期。
4. 具有动态分支预测部件和一个独立的计算分支条件的功能部件。
5. 跟大多数动态调度处理器一样，写回段的存在意味着实际的指令延迟会比按序流动的简单流水线多一个时钟周期。
6. 所以，从产生结果数据的源指令到使用该结果数据的指令之间的延迟为：整数运算1个周期，load操作2个周期，浮点加法运算3个周期。

问题:

1. 请列出该程序前面3遍循环中各条指令的流出、开始执行和将结果写到CDB上的时间。
2. 如果分支指令单流出，没有采用延迟分支，但分支预测是完美的。请列出整数部件、浮点部件、数据Cache以及CDB的资源使用情况。

解：执行时，该循环将动态展开，并且只要可能就流出两条指令。表中列出了各指令执行到几个操作点的时间及资源的使用情况。

5.5.2 基于动态调度的多流出技术

第 294 页

遍数	指 令		流出	执行	访存	写CDB	说明
1	L.D	F0,0 (R1)	1	2	3	4	流出第一条指令
1	ADD.D	F4,F0,F2	1	5		8	等待L.D的结果
1	S.D	F4,0 (R1)	2	3	9		等待ADD.D的结果
1	DADDIU	R1,R1,#-8	2	4		5	等待ALU
1	BNE	R1,R2,Loop	3	6			等待DADDIU的结果
2	L.D	F0,0 (R1)	4	7	8	9	等待BNE完成
2	ADD.D	F4,F0,F2	4	10		13	等待L.D的结果
2	S.D	F4,0 (R1)	5	8	14		等待ADD.D的结果
2	DADDIU	R1,R1,#-8	5	9		10	等待ALU
2	BNE	R1,R2,Loop	6	11			等待DADDIU的结果
3	L.D	F0,0 (R1)	7	12	13	14	等待BNE完成
3	ADD.D	F4,F0,F2	7	15		18	等待L.D的结果
3	S.D	F4,0 (R1)	8	13	19		等待ADD.D的结果
3	DADDIU	R1,R1,#-8	8	14		15	等待ALU
3	BNE	R1,R2,Loop	9	16			等待DADDIU的结果

5.5.2 基于动态调度的多流出技术

第 295 页

遍数	指 令	执行
1	L.D F0,0 (R1)	2
1	ADD.D F4,F0,F2	5
1	S.D F4,0 (R1)	3
1	DADDIU R1,R1,#-8	4
1	BNE R1,R2,Loop	6
2	L.D F0,0 (R1)	7
2	ADD.D F4,F0,F2	10
2	S.D F4,0 (R1)	8
2	DADDIU R1,R1,#-8	9
2	BNE R1,R2,Loop	11
3	L.D F0,0 (R1)	12
3	ADD.D F4,F0,F2	15
3	S.D F4,0 (R1)	13
3	DADDIU R1,R1,#-8	14
3	BNE R1,R2,Loop	16

时钟 周期	整型ALU	浮点ALU	数据Cache	CDB
2	1/L.D			
3	1/S.D		1/L.D	
4	1/DADDIU			1/L.D
5		1/ADD.D		1/DADDIU
6				
7	2/L.D			
8	2/S.D		2/L.D	1/ADD.D
9	2/DADDIU		1/S.D	2/L.D
10		2/ADD.D		2/DADDIU
11				
12	3/L.D			

5.5.2 基于动态调度的多流出技术

第 296 页

遍数	指 令	执行
1	L.D F0,0 (R1)	2
1	ADD.D F4,F0,F2	5
1	S.D F4,0 (R1)	3
1	DADDIU R1,R1,#-8	4
1	BNE R1,R2,Loop	6
2	L.D F0,0 (R1)	7
2	ADD.D F4,F0,F2	10
2	S.D F4,0 (R1)	8
2	DADDIU R1,R1,#-8	9
2	BNE R1,R2,Loop	11
3	L.D F0,0 (R1)	12
3	ADD.D F4,F0,F2	15
3	S.D F4,0 (R1)	13
3	DADDIU R1,R1,#-8	14
3	BNE R1,R2,Loop	16

时钟 周期	整型ALU	浮点ALU	数据Cache	CDB
13	3/S.D		3/L.D	2/ADD.D
14	3/DADDIU		2/S.D	3/L.D
15		3/ADD.D		3/DADDIU
16				
17				
18				3/ADD.D
19			3/S.D	
20				

可以看出：

- 每3个时钟周期就执行一个新循环，每个循环5条指令。

$$\text{IPC} = 5/3 = 1.67 \text{ 条/拍}$$

- 虽然指令的流出率比较高，但是执行效率并不是很高。

- 16拍共执行15条指令，

- 平均指令执行速度为 $15/16 = 0.94$ 条/拍。

- 原因是浮点运算少，ALU部件成了瓶颈。

- 解决方法：增加一个加法器，把ALU功能和地址运算功能分开。

3. 上述双流出动态调度流水线的性能受限于以下3个因素：

- 整数部件和浮点部件的工作负载不平衡，没有充分发挥出浮点部件的作用。
 - 应该设法减少循环中整数型指令的数量。
- 每个循环叠代中的控制开销太大。
 - 5条指令中有两条指令是辅助指令；
 - 应该设法减少或消除这些指令。
- 控制相关使得处理机必须等到分支指令的结果出来后才能开始下一条L.D指令的执行。

5.5.3 超长指令字技术

第 299 页

1. 把能并行执行的多条指令组装成一条很长的指令；
(100多位到几百位)
2. 设置多个功能部件；
3. 指令字被分割成一些字段，每个字段称为一个操作槽，直接独立地控制一个功能部件；
4. VLIW处理机中，在指令流出时不需要进行复杂的冲突检测，而是依靠编译器全部安排好了。

并行位

I_1	0	I_2	0	I_3	1	I_4	0	I_5	1	I_6	0	I_7	0	I_8	1
-------	---	-------	---	-------	---	-------	---	-------	---	-------	---	-------	---	-------	---

5. VLIW存在的一些问题

- 程序代码长度增加了
 - 提高并行性而进行的大量的循环展开；
 - 指令字中的操作槽并非总能填满。

解决：采用指令共享立即数字段的方法，或者采用指令压缩存储、调入Cache或译码时展开的方法。

- 采用了锁步机制：任何一个操作部件出现停顿时，整个处理机都要停顿。
- 机器代码的不兼容性

5.5.4 多流出处理器受到的限制

第 301 页

指令多流出处理器受哪些因素的限制呢？

主要受以下3个方面的影响：

- 程序所固有的指令级并行性；
- 硬件实现上的困难；
- 超标量和超长指令字处理器固有的技术限制。

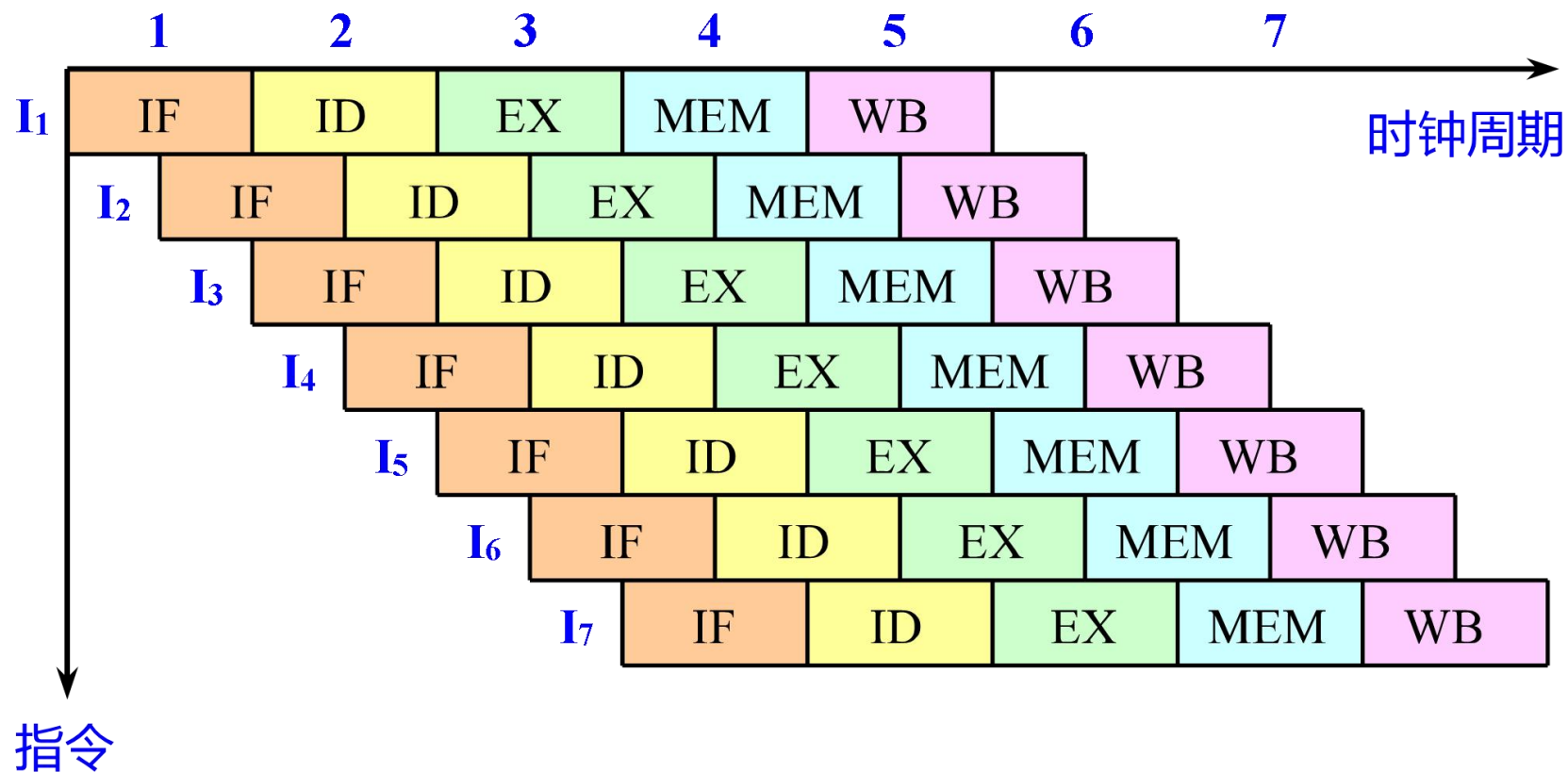
5.5.5 超流水线处理机

第 302 页

1. 将每个流水段进一步细分，这样在一个时钟周期内能够分时流出多条指令。这种处理机称为超流水线处理机。
2. 对于一台每个时钟周期能流出 n 条指令的超流水线计算机来说，这 n 条指令不是同时流出的，而是每隔 $1/n$ 个时钟周期流出一条指令。
 - 实际上该超流水线计算机的流水线周期为 $1/n$ 个时钟周期。
3. 一台每个时钟周期分时流出两条指令的超流水线计算机的时空图。
4. 在有的资料上，把指令流水线级数为8或8以上的流水线处理机称为超流水线处理机。

5.5.5 超流水线处理机

第 303 页



最大吞吐率:

$$TP_{\max} = \lim_{n \rightarrow \infty} \frac{n}{(m+n-1)\Delta t} = \frac{1}{\Delta t}$$

5.5.5 超流水线处理机

第 304 页

5. 典型的超流水线处理器：SGI公司的MIPS系列R4000

■ R4000微处理器芯片内有2个Cache：

- 指令Cache和数据Cache
- 容量都是8KB
- 每个Cache的数据宽度为64位

一个16×64位的浮点通用寄存器组。浮点通用寄存器组也可以设置成32个32位的浮点寄存器。

■ R4000的核心处理部件：整数部件

- 一个32×32位的通用寄存器组
- 一个算术逻辑部件（ALU）
- 一个专用的乘法/除法部件

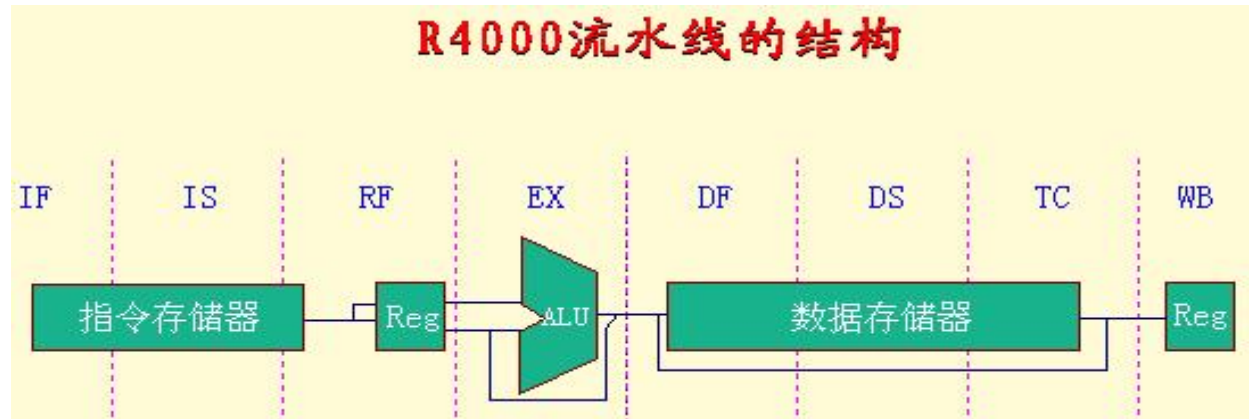
■ 浮点部件

- 一个执行部件
 - ✓ 浮点乘法部件
 - ✓ 浮点除法部件
 - ✓ 浮点加法/转换/求平方根部件（它们可以并行工作）

5.5.5 超流水线处理机

第 305 页

R4000的指令流水线有8级，各级的功能：



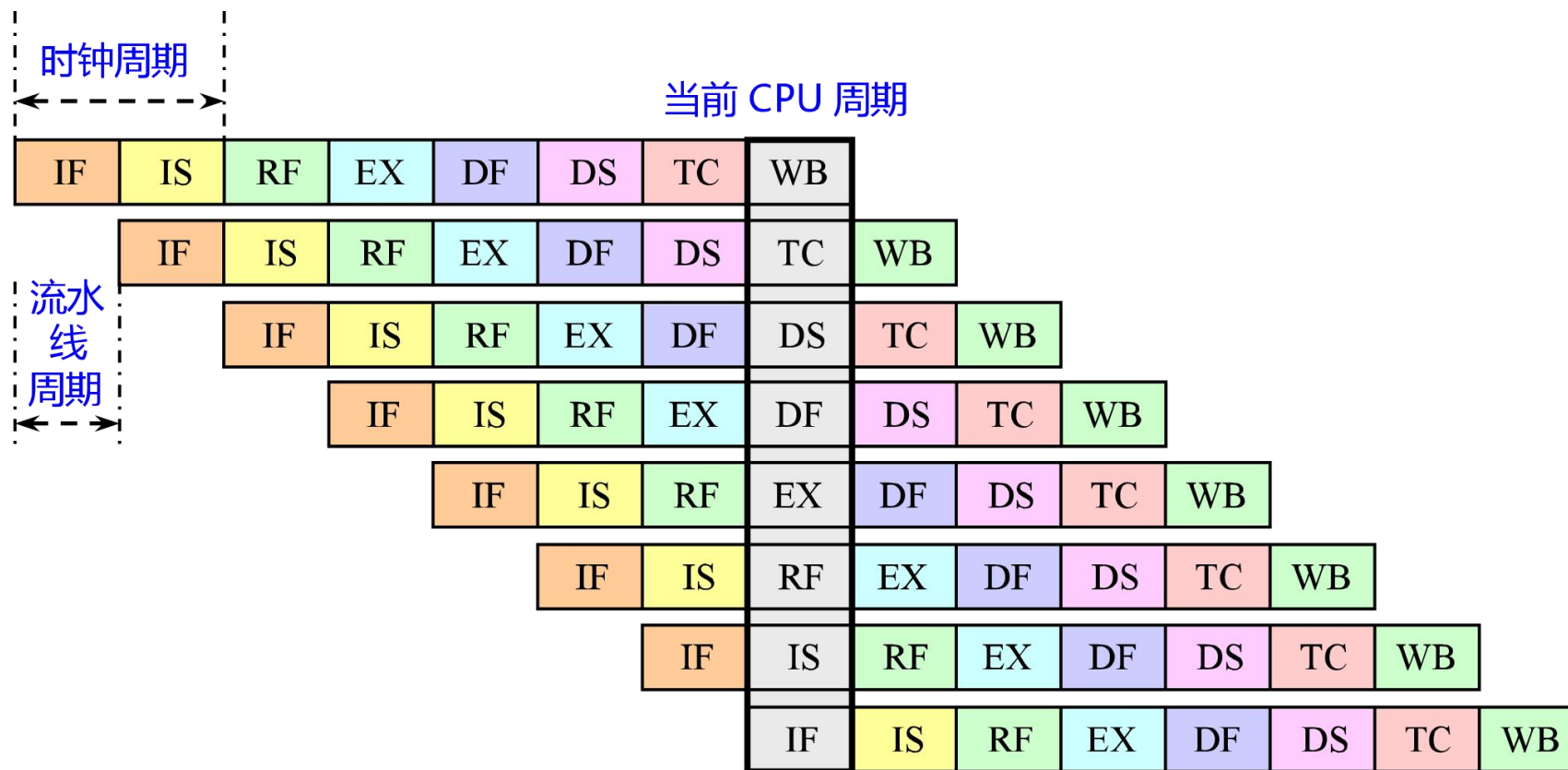
- **IF**：取指令的前半步，根据PC值去启动对指令Cache的访问。
- **IS**：取指令的后半步，在这一级完成对指令Cache的访问。
- **RF**：指令译码，访问寄存器组读取操作数，冲突检测，并判断指令Cache是否命中。
- **EX**：指令执行。包括：有效地址计算，ALU操作，分支目标地址计算，条件码测试。

- **DF**：取数据的前半步，启动对数据Cache的访问。
- **DS**：取数据的后半步，在这一级完成对数据Cache的访问。
- **TC**：标识比较，判断对数据Cache的访问是否命中。
- **WB**：load指令或运算型指令把结果写回寄存器组。

5.5.5 超流水线处理机

第 306 页

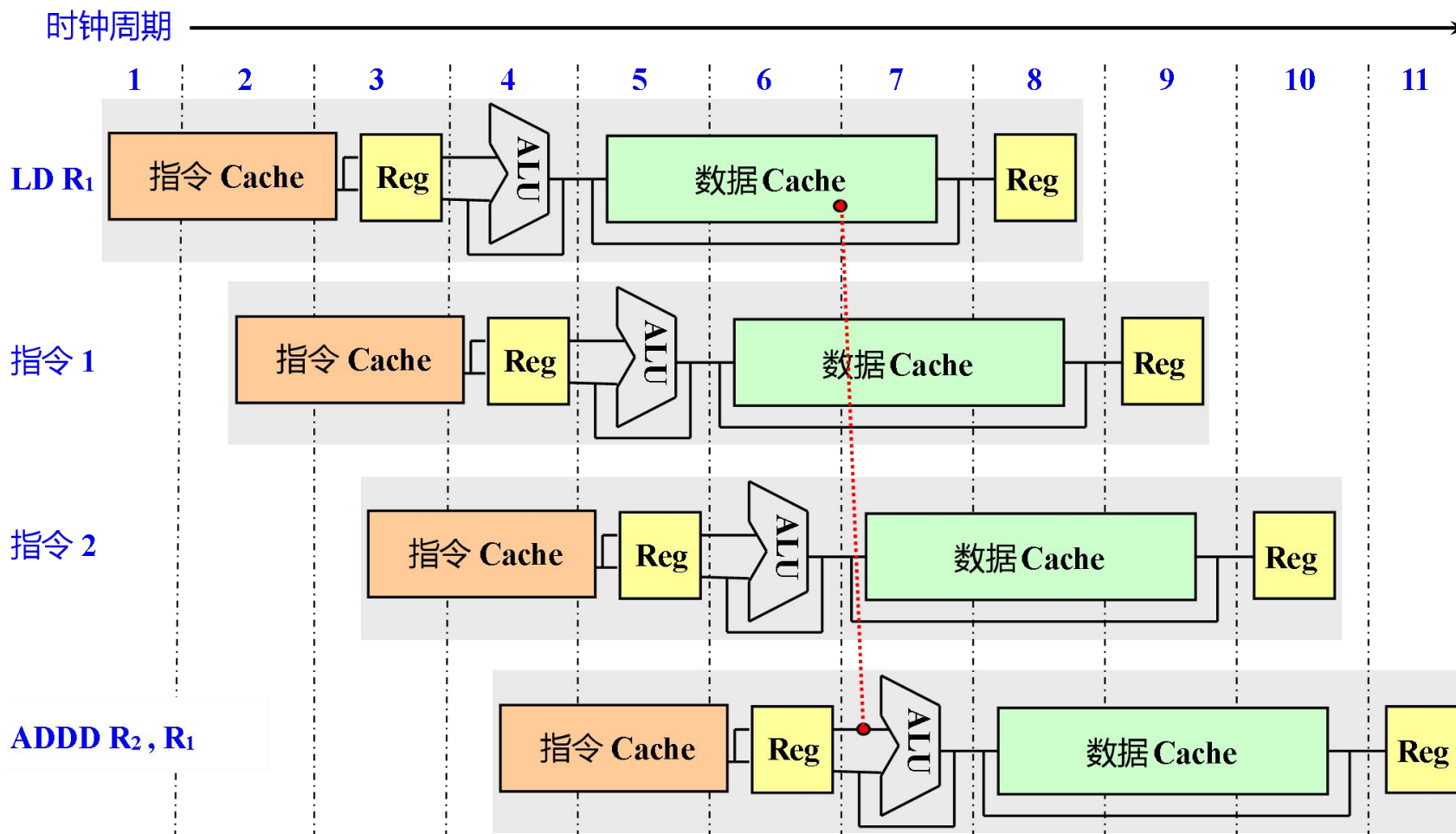
MIPS R4000指令流水线时空图



5.5.5 超流水线处理机

第 307 页

载入延迟为两个时钟周期



1. 指令级并行的概念：基于硬件的动态开发方法、基于软件的静态开发方法、CPI、IPC、循环展开
2. 相关与指令级并行：数据相关、名相关、控制相关，结构冲突、数据冲突、控制冲突，数据流与异常行为
3. 指令的动态调度：静态调度、动态调度、记分牌动态调度算法、Tomasulo算法
4. 动态分支预测技术：分支历史表BHT、分支目标缓冲器BTB、基于硬件的前瞻执行
5. 多指令流出技术：超标量、超长指令字VLIW、基于静态调度的多流出技术、基于动态调度的多流出技术

1、假设有多个加法器，不存在加法器的资源冲突。有3条连续指令组成的程序代码如下：

I ₁ :	ADD	R1, R2, R4	;R1←(R2)+(R4)
I ₂ :	ADD	R2, R1, 1	;R2←(R1)+1
I ₃ :	SUB	R1, R4, R5	;R1←(R4)+(R5)

- (1) 分析程序代码段中的数据相关；
- (2) 采用何种硬件技术可解决这些数据相关？请加以说明。

解：（1）指令I₁和I₂之间有RAW相关，I₁和I₂之间、I₂和I₃之间有WAR相关，I₁和I₃之间有WAW相关。

(2)

I1:	ADD	R1, R2, R4
I2:	ADD	R2', R1, 1
I3:	SUB	R1', R4, R5

I₁和I₂之间的RAW相关利用定向技术解决；
I₁和I₂之间、I₂和I₃之间有WAR相关以及I₁和I₃之间有WAW相关利用寄存器重命名技术R1→R1'、R2→R2'。

2、假设有一条长流水线，仅对条件转移指令使用分支目标缓冲。假设分支预测错误的开销为4个时钟周期，缓冲不命中的开销为3个时钟周期。假设：命中率为90%，预测精度为90%，分支频率为15%，没有分支的基本CPI为1。

- (1) 求程序执行的CPI;
- (2) 相对于采用固定的两个时钟周期延迟的分支处理，哪种方法程序执行速度更快?

解：(1) 程序执行的CPI=没有分支的基础CPI+分支带来的额外开销
分支带来的额外开销=15%×(90%命中×10%预测错误×4+10%没命中×3)=0.099
所以，程序执行的CPI=1+0.099=1.099

(2) 采用固定的两个时钟周期延迟的分支处理CPI=1+15%×2=1.3
由 (1) (2) 可知分支目标缓冲方法执行速度快。

指令在 BTB中?	预测	实际情 况	延迟周 期
是	成功	成功	0
是	成功	不成功	4
不是		成功	3
不是		不成功	0

3、下面的一段MIPS汇编程序是计算高斯消去法中的关键一步，用于完成下面公式的计算：

$$Y=a \times X+Y$$

整数指令均为一个时钟周期完成，浮点和整数部件均采用流水。整数操作之间以及与其他所有浮点操作之间的延迟为0，转移指令的延迟为0，X中的最后一个元素存放在存储器中的地址为DONE。

```
FOO:  L.D          F2, 0(R1)
      MUT.D       F4, F2, F0
      L.D          F6, 0(R2)
      ADD.D       F6, F4, F6
      S.D         F6, 0[R2]
      DADDIU      R1, R1, #8
      DADDIU      R2, R2, #8
      DSUBIU      R3, R1, #DONE
      BENZ
      FOO
```

指令的延迟如下表所示。

产生结果的指令	使用结果的指令	延迟（时钟周期数）
浮点计算	另一个浮点计算	3
浮点计算	浮点store（S.D）	2
浮点load（L. D）	浮点计算	1
浮点load（L. D）	浮点load（L. D）	0

(1) 对于标准的MIPS单流水线，上述循环计算一个Y值需要多少时间？其中有多少空转周期？

解： (1)

L.D	F2, 0(R1)	// 1
stall		
MUT.D	F4, F2, F0	// 2
L.D	F6, 0(R2)	// 3
stall		
stall		
ADD.D	F6, F4, F6	// 4
stall		
stall		
S.D	F6, 0[R2]	// 5
DADDIU	R1, R1, #8	// 6
DADDIU	R2, R2, #8	// 7
DSUBIU	R3, R1, #DONE	// 8
BNEZ	R3, FOO	// 9

共有14个时钟周期，其中有5个空转周期

(2) 对于标准的MIPS单流水线，将上述循环顺序展开4次，不进行任何指令调度，计算一个Y值平均需要多少时间？加速比是多少？

解：(2) 循环顺序展开4次，不进行任何指令调度，则指令1~5及其间的stall都是必要的，只是指令6~9只需执行一次，因此共有 $10 \times 4 + 4 = 44$ 个时钟周期，计算出4个Y值，所以计算一个Y值需要11个时钟周期，加速比为 $14/11 = 1.27$ 。

加速主要是来自减少控制开销，即减少对R1、R2的整数操作以及比较、分支指令而来的。

(3) 对于标准的MIPS单流水线，将上述循环顺序展开4次，优化和调度指令，使循环处理时间达到最优，计算一个Y值平均需要多少时间？加速比是多少？

解：(3) 循环顺序展开4次，优化和调度指令，如下：

L.D	F2, 0(R1)	ADD.D	F6, F4, F6
L.D	F8, 8(R1)	ADD.D	F12, F10, F12
L.D	F14, 16(R1)	ADD.D	F18, F16, F18
L.D	F20, 24(R1)	ADD.D	F24, F22, F24
MUT.D	F4, F2, F0	S.D	F6, 0[R2]
MUT.D	F10, F8, F0	S.D	F12, 8[R2]
MUT.D	F16, F14, F0	S.D	F18, 16[R2]
MUT.D	F22, F20, F0	S.D	F24, 24[R2]
L.D	F6, 0(R2)	DADDIU	R1, R1, #32
L.D	F12, 8(R2)	DADDIU	R2, R2, #32
L.D	F18, 16(R2)	DADDIU	R3, R1, #DONE
L.D	F24, 24(R2)	BNEZ	R3, FOO

共用了24个时钟周期，则计算一个Y值平均需要
 $24/4=6$ 个时钟周期。

加速比： $14/6=2.33$

(4) 对于两路超标量的MIPS流水线，设有两个指令流出部件，可以流出任意组合的指令，系统中的功能部件数量不受限制。将上述循环展开4次，优化和调度指令，使循环处理时间达到最优。计算一个Y值平均需要多少时间？加速比是多少？

整数指令	浮点指令	时钟周期数
L.D F2, 0(R1)		1
L.D F8, 8(R1)		2
L.D F14, 16(R1)	MUT.D F4, F2, F0	3
L.D F20, 24(R1)	MUT.D F10, F8, F0	4
L.D F6, 0(R2)	MUT.D F16, F14, F0	5
L.D F12, 8(R2)	MUT.D F22, F20, F0	6
L.D F18, 16(R2)	ADD.D F6, F4, F6	7
L.D F18, 24(R2)	ADD.D F12, F10, F12	8
DADDIU R1, R1, #32	ADD.D F18, F16, F18	9
S.D F6, 0(R2)	ADD.D F24, F22, F24	10
S.D F12, 8(R2)		11
S.D F18, 16(R2)		12
S.D F24, 24(R2)		13
DADDIU R2, R2, #32		14
DSUBIU R3, R1, #DONE		15
BNEZ R3, FOO		16

计算一个Y值需要16/4=4个时钟周期
加速比=14/4=3.5

应用题

第 316 页

(5) 对于超长指令字MIPS处理器，将上述循环展开4次，优化和调度指令，使循环处理时间达到最优。计算一个Y值平均需要多少时间？加速比是多少？

访存1	访存2	浮点指令1	浮点指令2	整数指令	时钟周期
L.D F2, 0(R1)	L.D F8, 8(R1)				1
L.D F14, 16(R1)	L.D F20, 24(R1)				2
L.D F6, 0(R2)	L.D F12, 8(R2)	MUT.D F4, F2, F0	MUT.D F10, F8, F0		3
L.D F18, 16(R2)	L.D F24, 24(R2)	MUT.D F16, F14, F0	MUT.D F22, F20, F0		4
		ADD.D F6, F4, F6	ADD.D F12, F10, F12		5
		ADD.D F18, F16, F18	ADD.D F24, F22, F24	DADDIU R1, R1, #32	6
				DADDIU R2, R2, #32	7
				DADDIU R3, R1, #32	8
				BNEZ R3, FOO	9
S.D F6, -32(R2)	S.D F12, -24(R2)				10
S.D F18, -16(R2)	S.D F24, -8(R2)				11

计算一个Y值平均需要11/4个时钟周期，加速比=14/(11/4)=56/11≈5.1

4、用一台每个时钟周期发射两条指令的超标量处理机运行下面一段程序。所有指令都要进行取指令（IF）、译码（ID）、执行（LS、MUL、ADD等）、写结果（WB）等4个阶段。其中，IF、ID、WB三个阶段各为一个流水段，其延迟时间均为10ns。在执行（EX）阶段，LOAD操作、AND操作各延迟10ns，ADD操作延迟20ns，MUL操作延迟30ns。这4种功能部件各设置一个，它们可以并行工作。ADD部件和MUL部件都采用流水结构，每一级流水线的延迟时间都是10ns。

(1)请列出程序代码中所有的数据相关及相关类型

I1	LOAD	R0, M(A)	; $R0 \leftarrow M(A)$
I2	ADD	R1, R0	; $R1 \leftarrow (R1) + (R0)$
I3	LOAD	R2, M(B)	; $R2 \leftarrow M(B)$
I4	MUL	R3, R4	; $R3 \leftarrow (R3) \times (R4)$
I5	AND	R4, R5	; $R4 \leftarrow (R4) \wedge (R5)$
I6	ADD	R2, R5	; $R2 \leftarrow (R2) + (R5)$

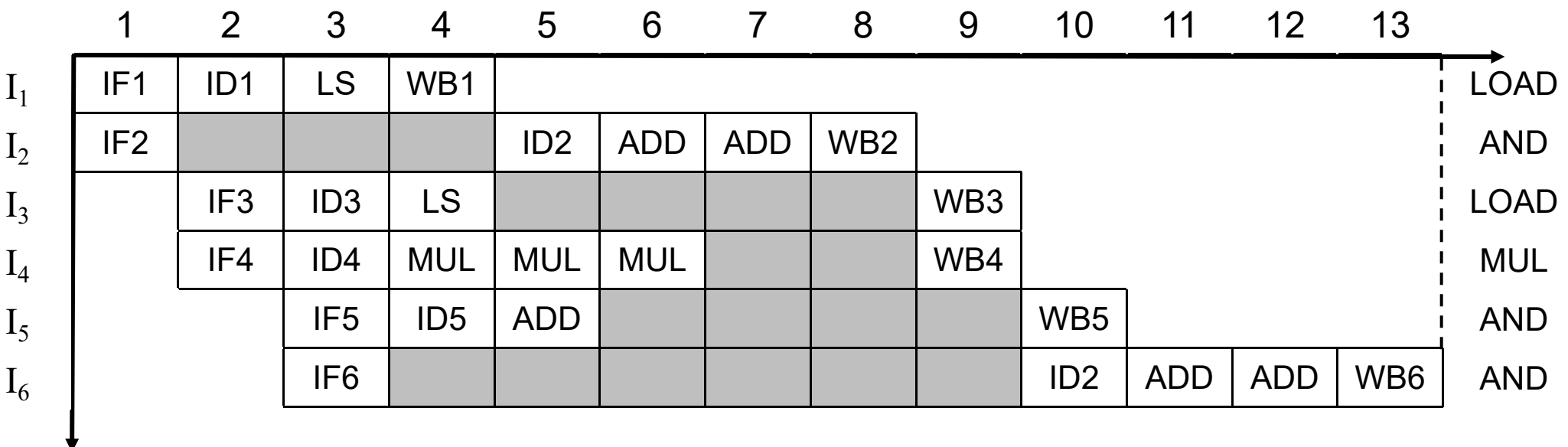
1. 指令I1、I2间有寄存器R0的RAW相关
2. 指令I3、I6间有寄存器R2的RAW相关
3. 指令I4、I5间有寄存器R4的WAR相关
4. 指令I3、I6间有寄存器R2的WAW相关

应用题

第 318 页

(2)假设所有运算型指令都在译码（ID）流水段读寄存器，在写结果（WB）流水段写寄存器，采用顺序发射顺序完成的调度方法。画出流水线的时空图；计算执行这个程序所用的时间。

```
I1  LOAD      R0, M(A)      ; R0←M(A)
I2  ADD       R1, R0        ; R1←(R1)+(R0)
I3  LOAD      R2, M(B)      ; R2←M(B)
I4  MUL       R3, R4        ; R3←(R3)×(R4)
I5  AND       R4, R5        ; R4←(R4)^(R5)
I6  ADD       R2, R5        ; R2←(R2)+(R5)
```



LOAD
AND
LOAD
MUL
AND
AND

这个程序所用的时间是
130ns

第6章 指令级并级及其开发——软件方法